微信小程序开发基础

第1章、小程序概述

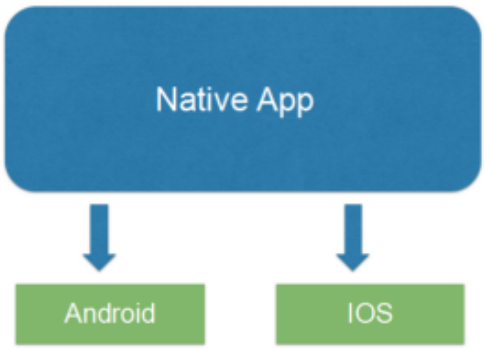
1.1节、什么是小程序

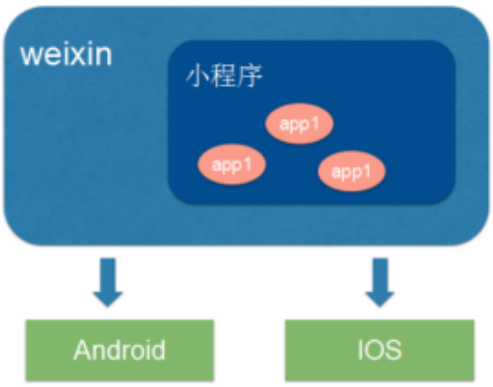
微信小程序，简称小程序，是一种不需要下载安装即可使用的应用，它实现了应用”触手可及”的梦想，用户扫一扫或搜一下即可打开应用。

说明:
1.小程序的使用：打开微信-扫一扫或者是搜索小程序 进入小程序
2.小程序是需要下载安装的，小程序大小占用小，用户感觉不到
3.2017年大小限制1M，目前单个文件最大2M （超过2M也是可以的 需要分包加载）
特点:
1.免安装
2.操作接近原生App
3.必须在微信中使用

1.2节、小程序的环境

微信小程序





1.3节、注册小程序的流程

小程序不注册也可以去学习去写小程序 ，但是不注册的小程序是本地的小程序，微信上看不到，只能本地打开你的微信开发工具查看。

注册小程序的目的是为了获取小程序的一个AppID账号，有了这个账号就可以发布自己开发的小程序，这样微信里面才可以搜索到。

注册小程序:

1. 公司注册 ---（公司内部已经注册完毕）给你小程序的appid账号

2. 学习中注册个人账号（appid）可以发布的
3. 个人和公司有区别 部分功能不对个人开放：比如 支付（结合百度） 转发

微信小程序官网： https://mp.weixin.qq.com/

1.3.1、注册账号



点击：**立即注册**---进入注册界面



填写：**账号（邮箱）、密码、验证码**等

1.3.2、邮箱验证



进入邮箱：**激活**



1.3.3、登记信息

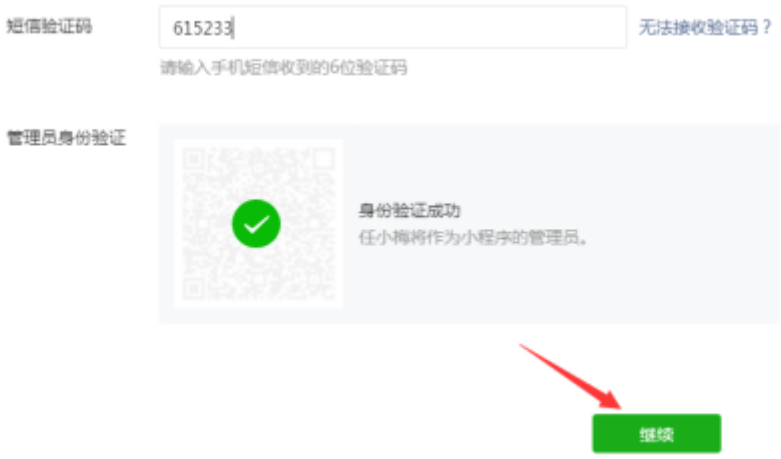


地区：**中国大陆**

主体：**个人**



填入：**姓名、身份证、手机号**



接着：**输入验证码、微信扫二维码**，最后提交



1.3.4、填写小程序



填写：**小程序名、简称、头像、介绍**等信息



需要注意的是**服务类目**、不要选择**游戏**

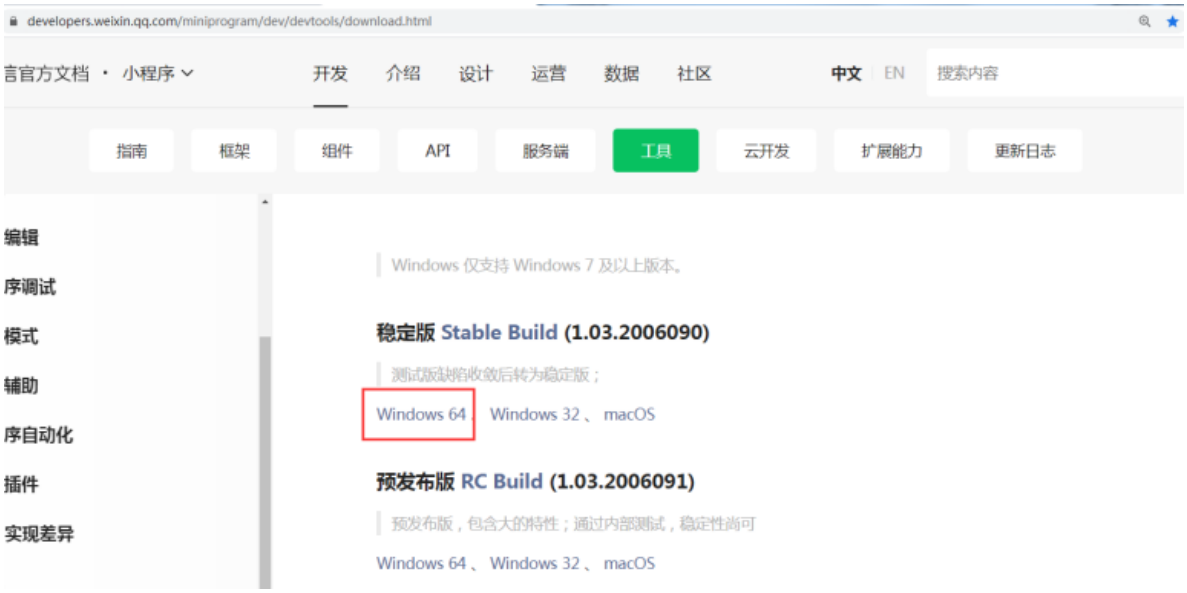


注册完成后生成：小程序个人AppID账号，我们可以开发个人小程序了

1.4节、综合案例-创建小程序

1.4.1、安装开发工具

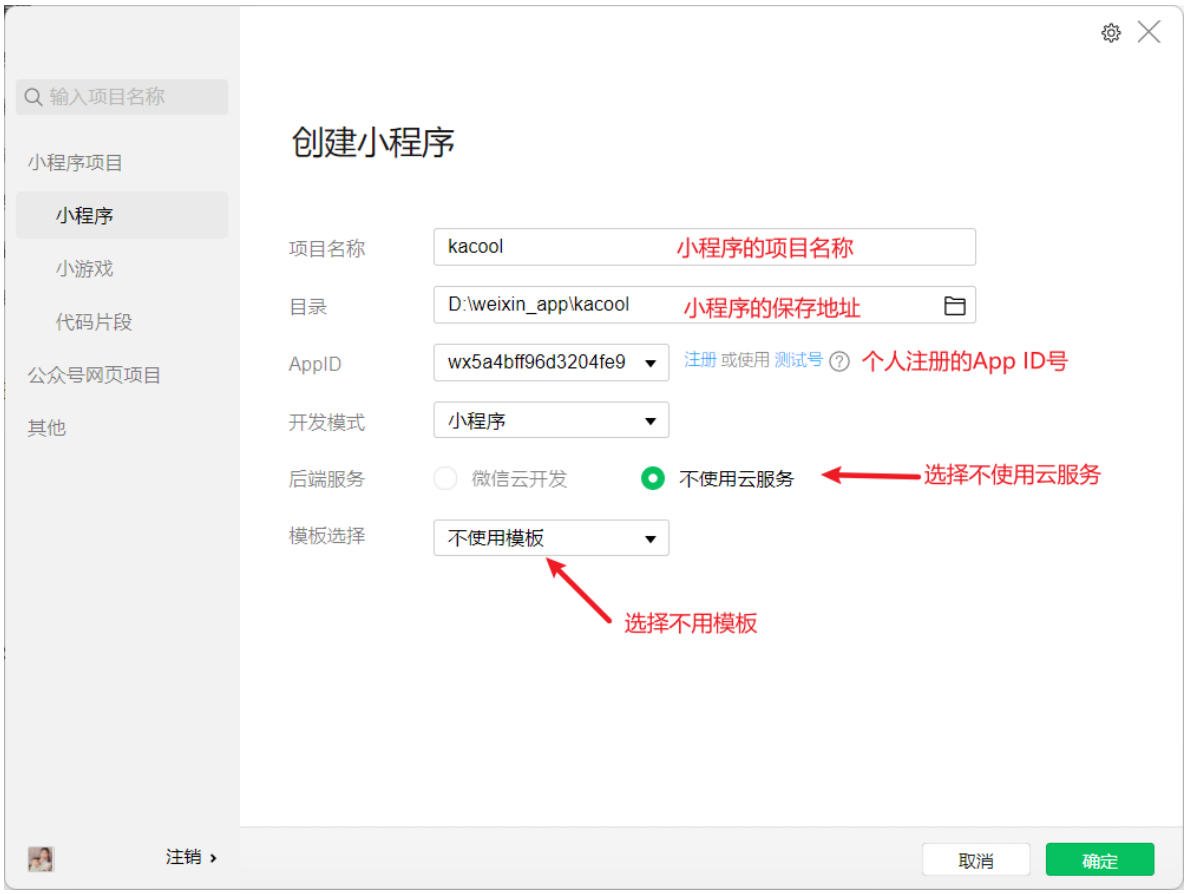
微信小程序开发工具下载地址：https://developers.weixin.qq.com/miniprogram/dev/devtools/download.html



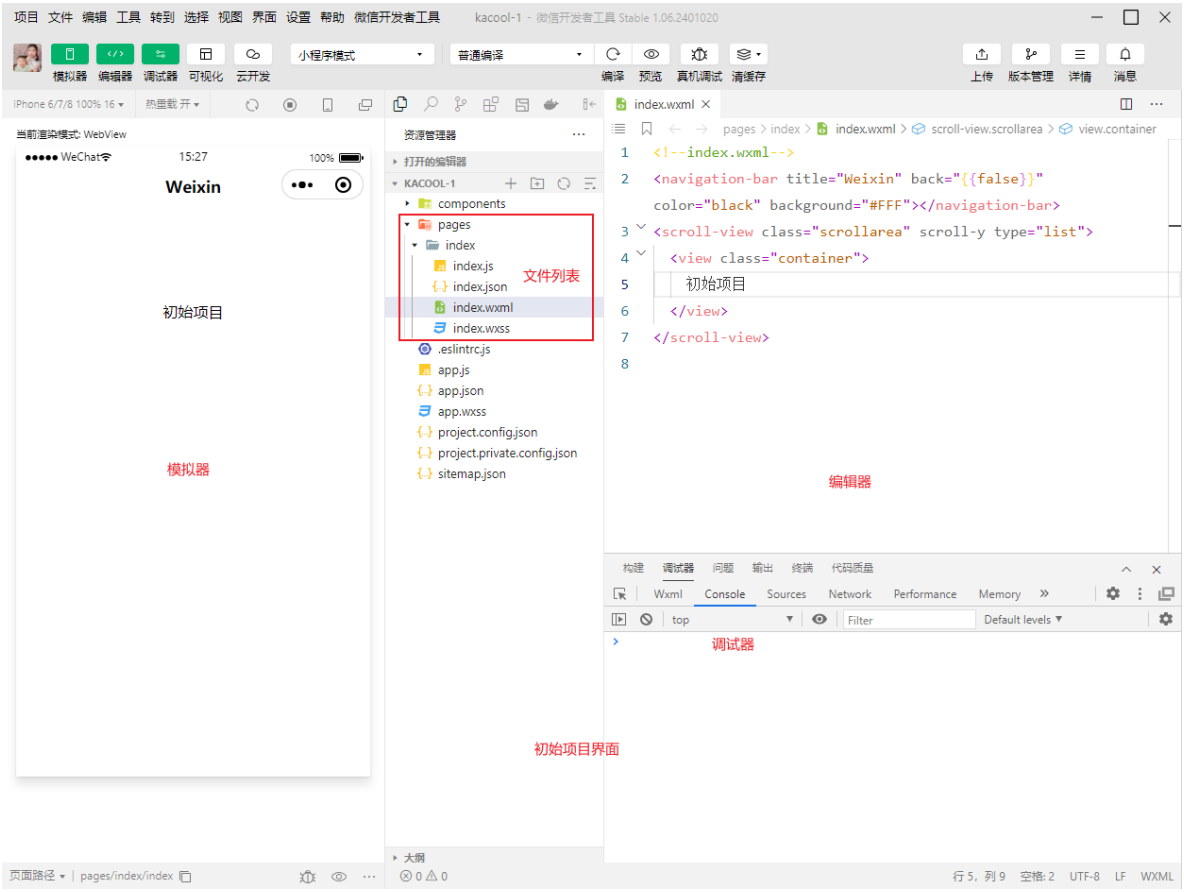
选择：**对应自己电脑系统的稳定版本**，下载并安装



1.4.2、创建小程序项目



点击：**确定**按钮



1.4.3、小程序目录结构

```
├──pages              //小程序页面目录
|   |──index          //小程序页面 1
|   |   ├──index.js     //页面逻辑文件
|   |   ├──index.json   //页面配置文件
|   |   ├──index.wxml   //页面结构文件，也就是 html
|   |   └──index.wxss   //页面样式文件，也就是 css
|   └──log            //小程序页面 2，初始化可删
├──utils              //工具集合目录，初始化可删
├──app.js             //小程序的逻辑
|──app.json           //小程序的公共配置
|──app.wxss           //小程序的公共样式
|──project.config.json      //工具配置文件
└──sitemap.json             //索引配置
```

第2章、WXML语法

WXML（WeiXin Markup Language）是框架设计的一套标签语言，结合基础组件、事件系统，可以构建出页面的结构。

在WXML中，其知识点主要涵盖：

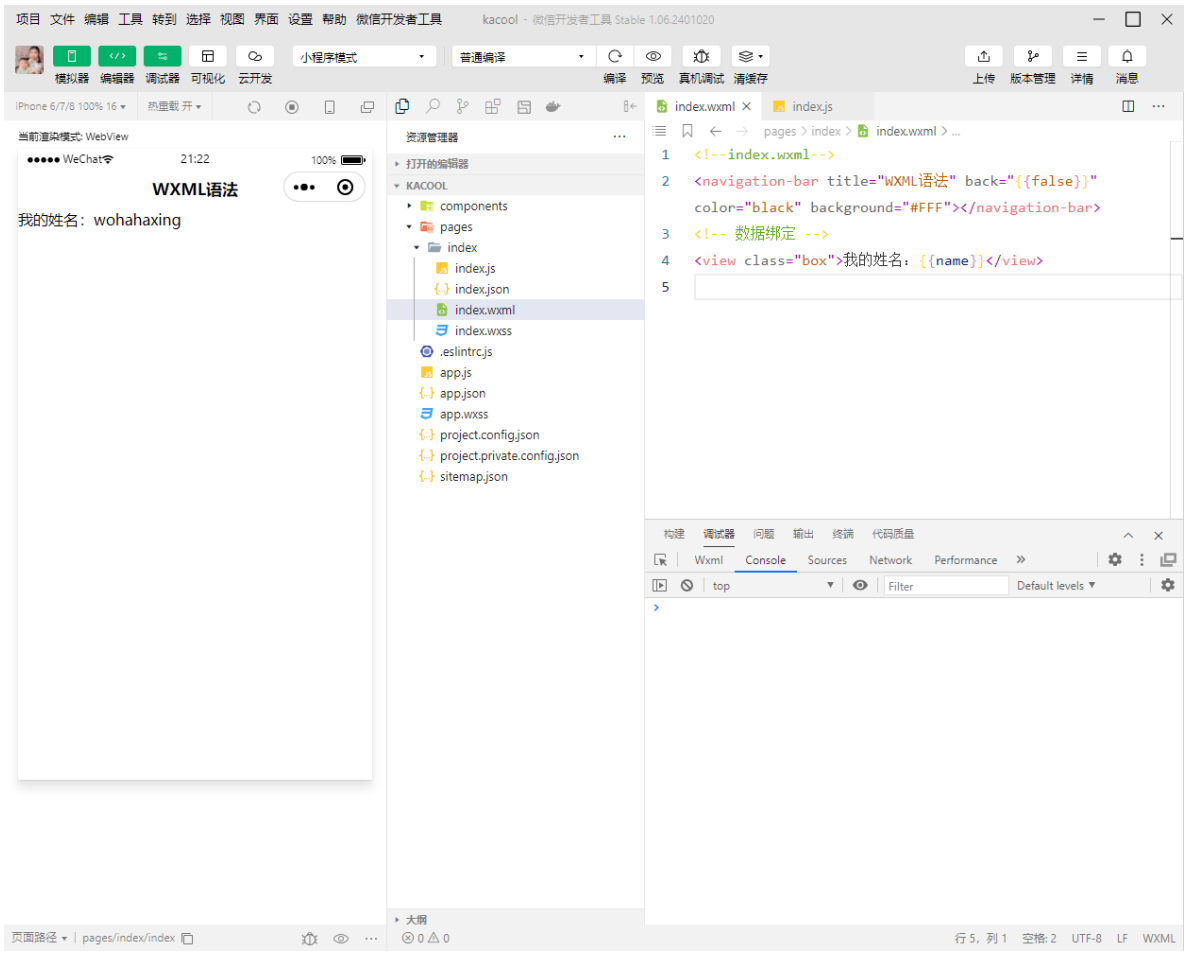
1. 数据绑定
2. 属性绑定
3. 事件绑定
4. 条件渲染
5. 列表渲染

2.1节、数据绑定

数据绑定就是使用“{{}}”插值语法，将data中的数据，直接展示在页面结构中。

```
<!-- 数据绑定 -->
<view class="box">我的姓名：{{name}}</view>
```

```
/**
* 页面的初始数据
*/
data: {
    name:'wohahaxing'
}
```



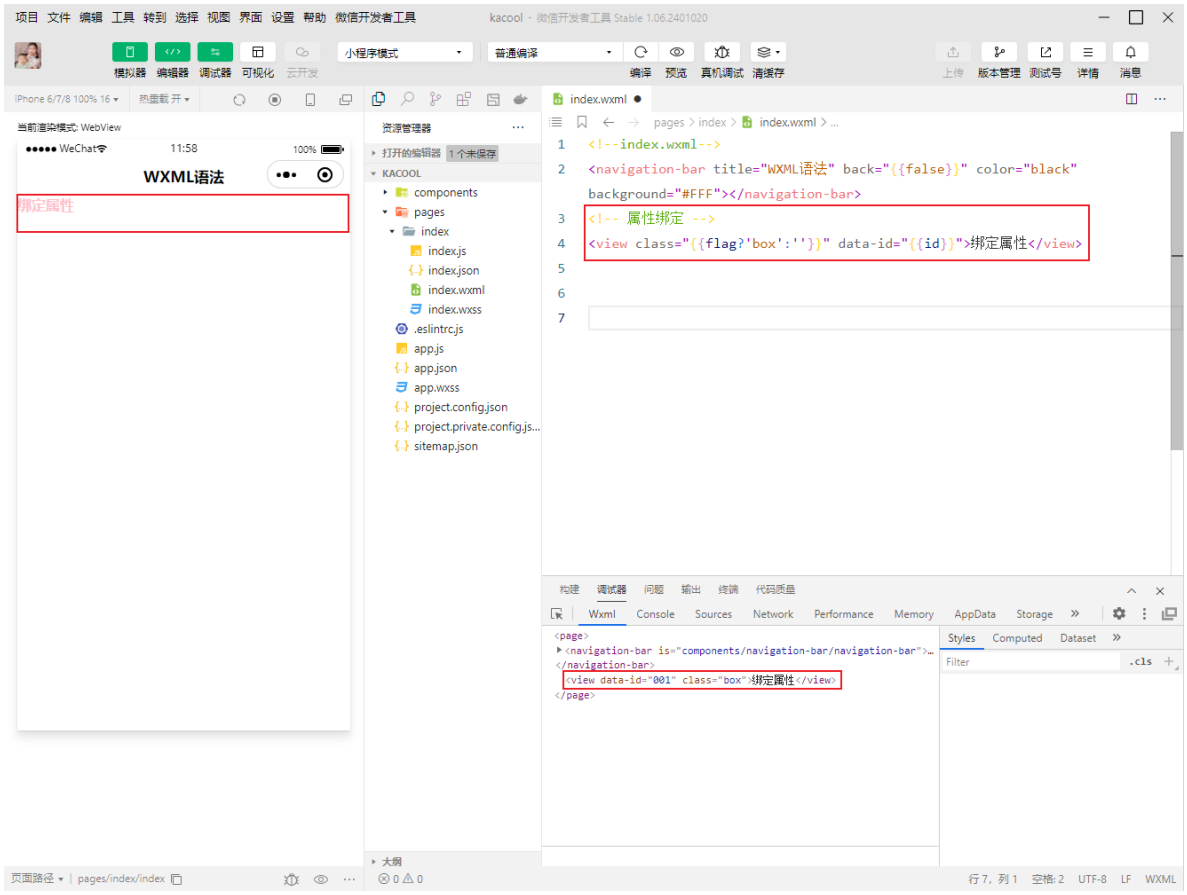
2.2节、属性绑定

属性绑定是指把data中的数据绑定给组件中的某一个属性，经常用于样式动态控制或者传递数据。

```
<!-- 属性绑定 -->
<view class="{{flag?'box':''}}" data-id="{{id}}">绑定属性</view>
```

```
/**
* 页面的初始数据
*/
data: {
    id:"001",
    flag:true
}
```

```
/**index.wxss**/
.box{
  color: pink;
  font-weight: 700;
}
```



2.3节、事件绑定

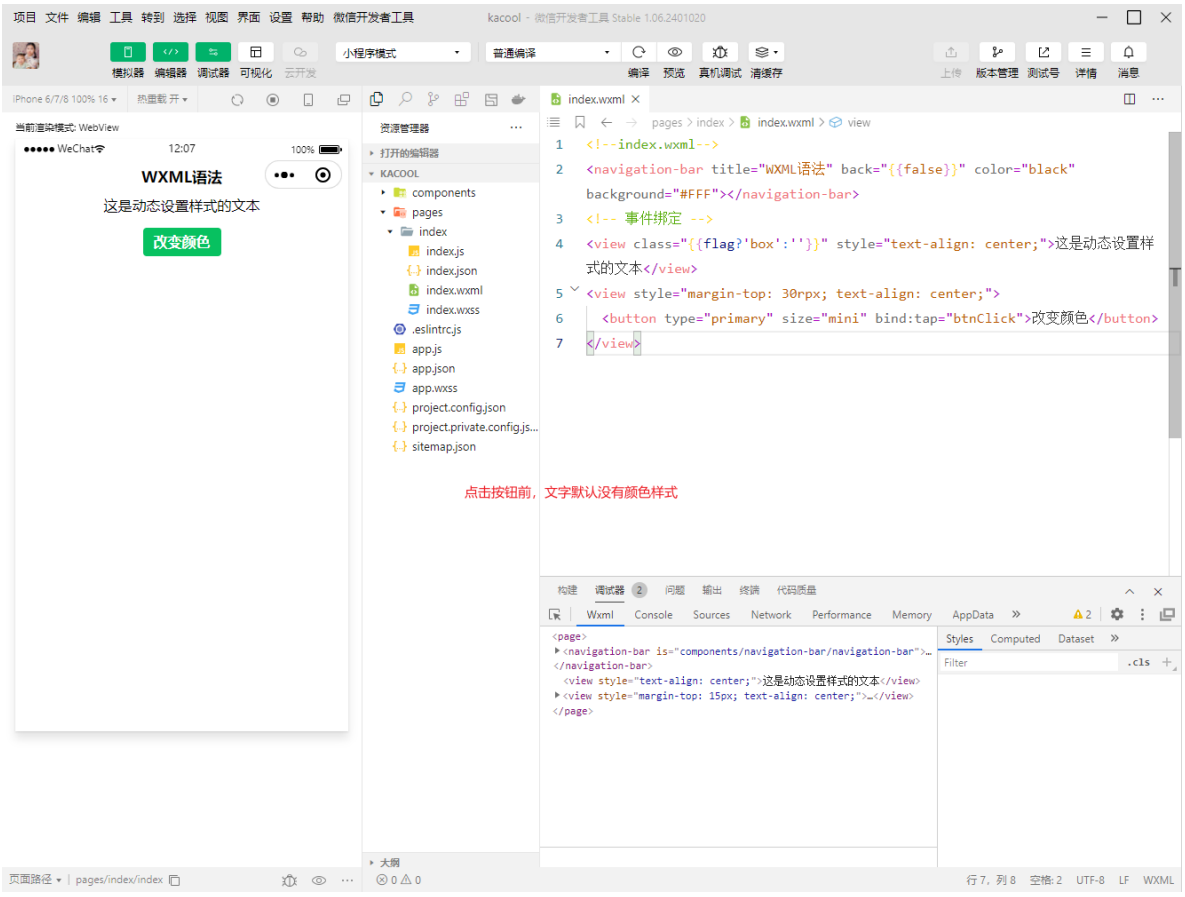
事件在小程序的交互中，使用非常频繁。跟网页中的事件有所不同的是，小程序中的事件通过`bind:事件名`这种方式来绑定。

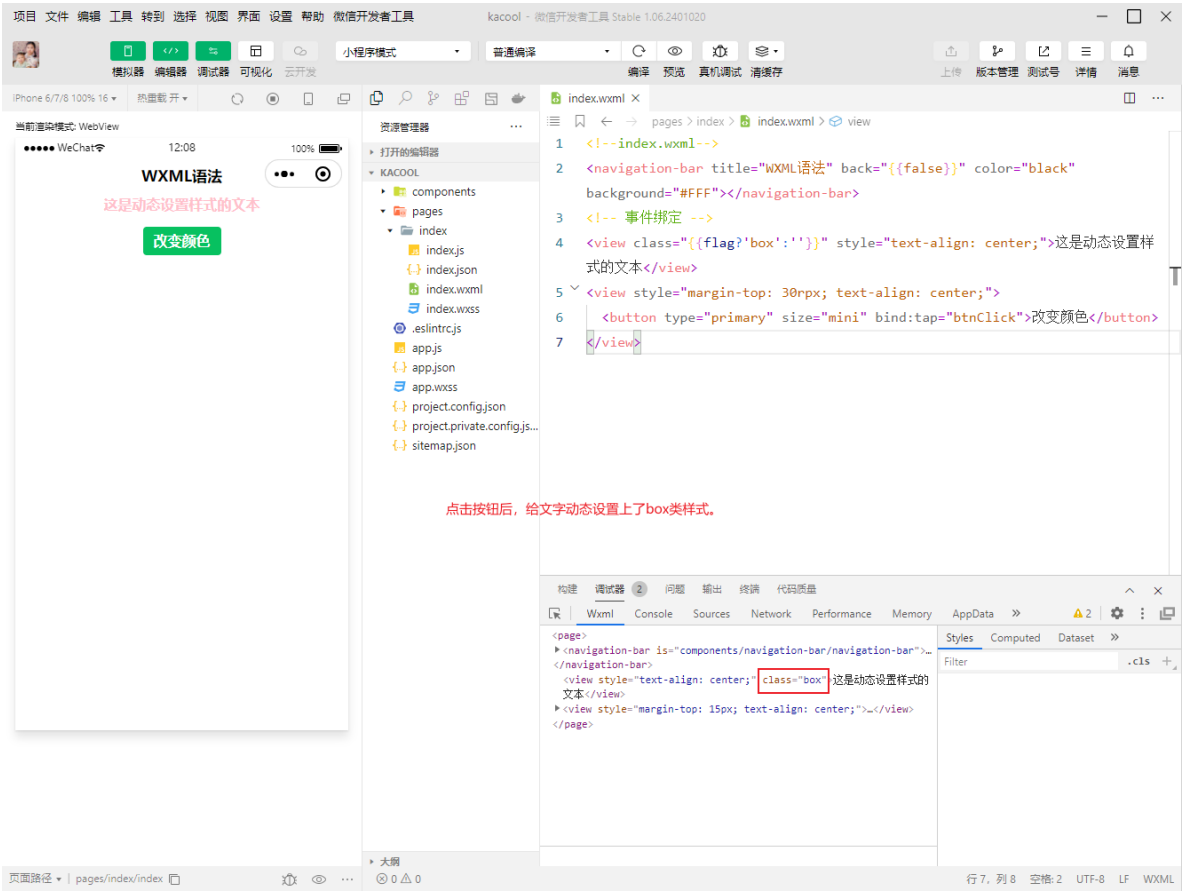
```wxml
<!-- 事件绑定 -->
<view class="{{flag?'box':''}}" style="text-align: center;">这是动态设置样式的文本</view>
<view style="margin-top: 30rpx; text-align: center;">
  <button type="primary" size="mini" bind:tap="btnClick">改变颜色</button>
</view>
```

```js
/**
* 页面的初始数据
*/
data: {
    flag:false
},

//自定义事件
btnClick(){
    this.setData({
        flag:!this.data.flag
    })
}
```

```wxss
/**index.wxss**/
.box{
  color: pink;
  font-weight: 700;
}
```





说明：

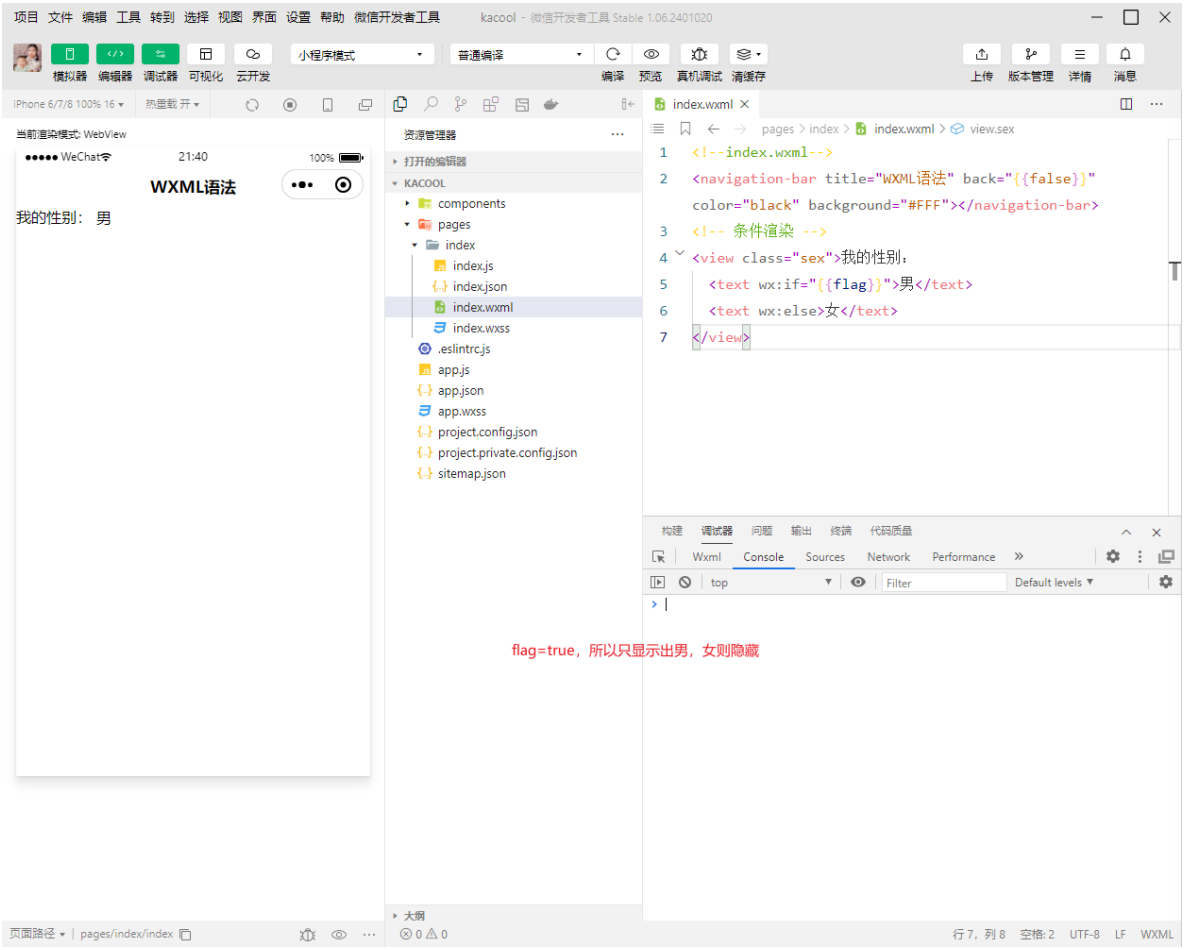
在微信小程序中，不同的组件，都拥有各自的事件，所以我们在组件学习中会看到通过bind方式来绑定其它事件。

2.4节、条件渲染

在框架中，使用 `wx:if=""` 来判断是否需要渲染该代码块，与之对应的有 `wx:elif=""` 和 `wx:else` 。

```
<!-- 条件渲染 -->
<view class="sex">我的性别:
    <text wx:if="{{flag}}">男</text>
    <text wx:else>女</text>
</view>
```

```
/**
* 页面的初始数据
*/
data: {
    flag:true
}
```



2.5节、列表渲染

在组件上使用 `wx:for` 控制属性绑定一个数组，即可使用数组中各项的数据重复渲染该组件。

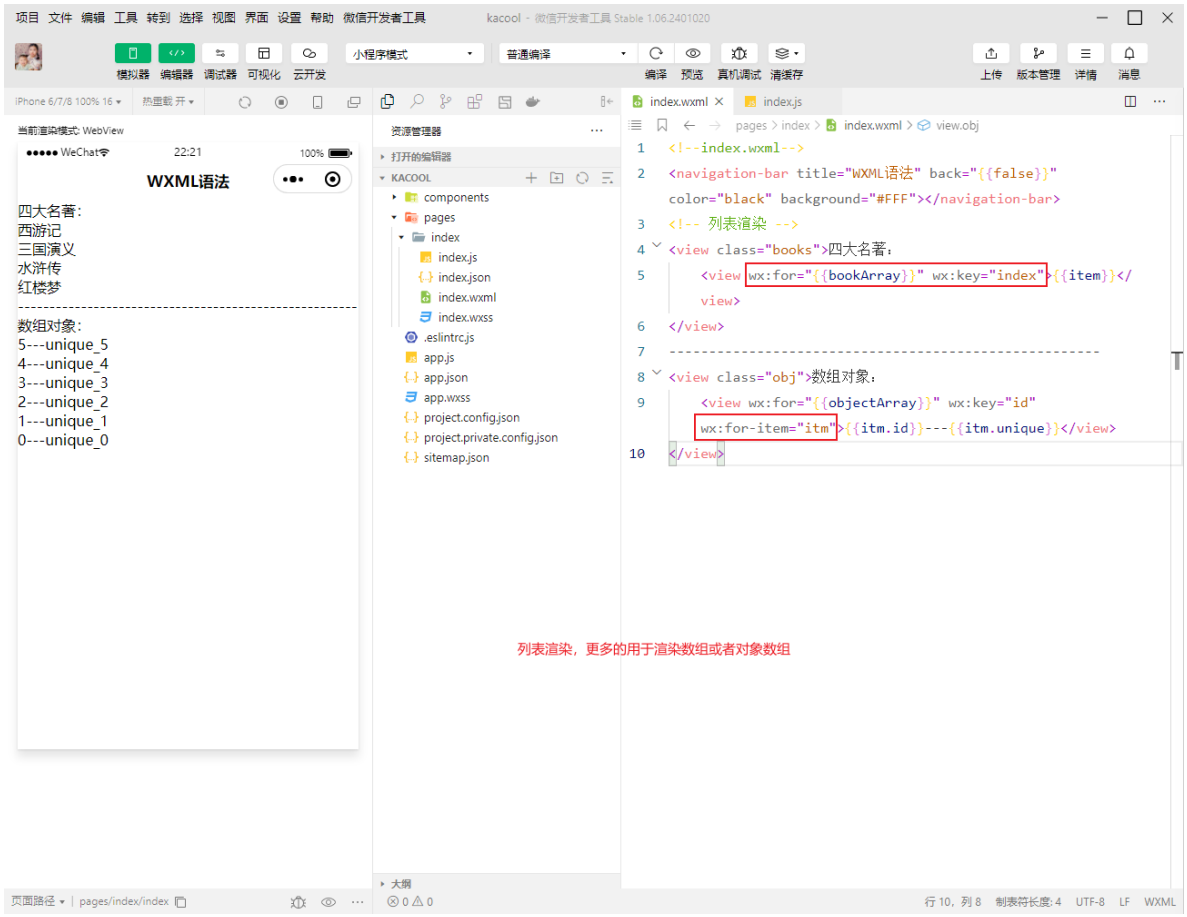
默认数组的当前项的下标变量名默认为 `index`，数组当前项的变量名默认为 `item`;使用 `wx:for-item` 可以指定数组当前元素的变量名，使用 `wx:for-index` 可以指定数组当前下标的变量名。

需要使用 `wx:key` 来指定列表中项目的唯一的标识符

```
<!-- 列表渲染 -->
<view class="books">四大名著：
    <view wx:for="{{bookArray}}" wx:key="index">{{item}}</view>
</view>
-----------------------------------------------------
<view class="obj">数组对象：
    <view wx:for="{{objectArray}}" wx:key="id" wx:for-item="itm">{{itm.id}}---{{itm.unique}}</view>
</view>
```

```
/**
* 页面的初始数据
*/
data: {
    bookArray: ["西游记", "三国演义", "水浒传", "红楼梦"],
```

```
    objectArray: [
        {id: 5, unique: 'unique_5'},
        {id: 4, unique: 'unique_4'},
        {id: 3, unique: 'unique_3'},
        {id: 2, unique: 'unique_2'},
        {id: 1, unique: 'unique_1'},
        {id: 0, unique: 'unique_0'},
    ]
}
```



更多知识学习：https://developers.weixin.qq.com/miniprogram/dev/reference/wxml

2.6节、综合案例-词条

实现效果：

●●●●● WeChat 14:54 100%

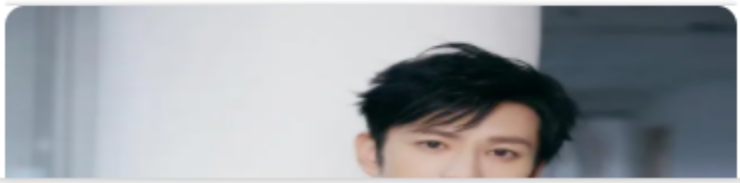
综合案例-词条



杨幂　女

健康女神

在2023年，中国健康女神杨幂以她的美丽和健康形象，成为了全国人民的健康榜样。她始终坚持健康的生活方式，并通过自己的努力，将健康理念传递给更多的人。杨幂，你的健康形象感动了亿万人民，你是我们心中的健康女神！

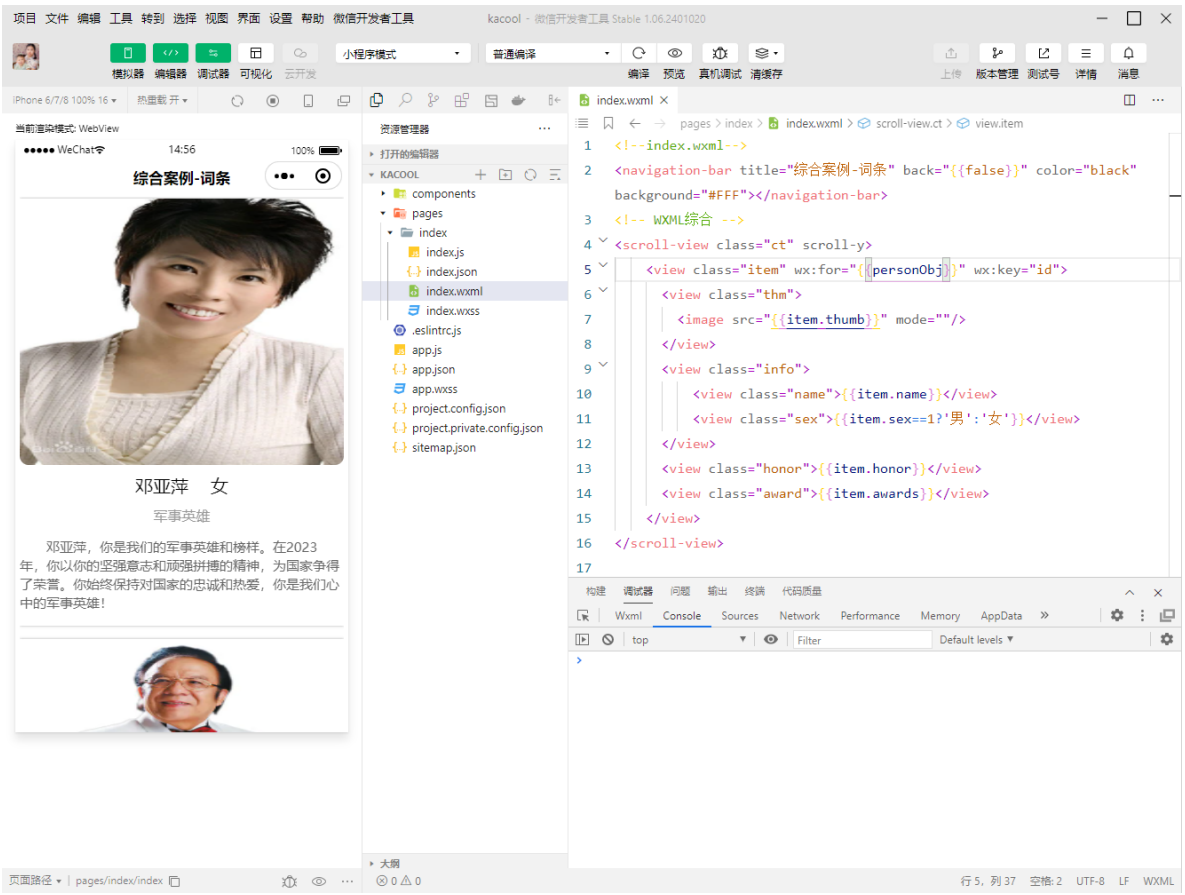


●●●●● WeChat 14:56 100%

综合案例-词条







核心代码：

```
<!--index.wxml-->
<navigation-bar title="综合案例-词条" back="{{false}}" color="black" background="#FFF"></navigation-bar>
<!-- WXML综合 -->
<scroll-view class="ct" scroll-y>
    <view class="item" wx:for="{{personObj}}" wx:key="id">
      <view class="thm">
        <image src="{{item.thumb}}" mode=""/>
      </view>
      <view class="info">
          <view class="name">{{item.name}}</view>
          <view class="sex">{{item.sex==1?'男':'女'}}</view>
```

```
        </view>
        <view class="honor">{{item.honor}}</view>
        <view class="award">{{item.awards}}</view>
      </view>
</scroll-view>

```

```
/**index.wxss**/
.ct{
  box-sizing: border-box;
  padding: 0rpx 10rpx;
  height: 100vh;
  padding-bottom: 120rpx;
}
.ct .item{
  box-shadow: 0rpx 0rpx 6rpx #aaa;
  font-size: 40rpx;
}

.ct .thm{
  width: 100%;
  height: 600rpx;
}
.ct .thm image{
  width: 100%;
  height: 100%;
  border-radius: 20rpx;
}
.ct .info{
  display: flex;
  justify-content: center;
  margin: 20rpx 0rpx;
}
.ct .sex{
  margin-left: 50rpx;
}
.ct .honor{
  text-align: center;
  font-size: 32rpx;
  color: #999;
}
.ct .award{
  margin: 30rpx 0rpx;
  font-size: 30rpx;
  color: #666;
  text-indent: 2em;
  line-height: 1.4em;
  padding-bottom: 30rpx;
}
```

```
/**
 * 页面的初始数据
 */
  data: {
    flag:false,
    title:"2023年度感动中国十大人物及颁奖词",
```

```
    personObj:[
      {
        id:"001",
        name:"杨幂",
        sex:0,
        honor:"健康女神",
        awards:"在2023年，中国健康女神杨幂以她的美丽和健康形象，成为了全国人民的健康榜样。她始终坚持健康的生活方式，并通过自己的努力，将健康理念传递给更多的人。杨幂，你的健康形象感动了亿万人民，你是我们心中的健康女神！",
        thumb:"https://bkimg.cdn.bcebos.com/pic/203fb80e7bec54e736d1538552708c504fc2d5623c5d?x-bce-process=image/format,f_auto/resize,m_lfit,limit_1,h_801"
      },
      {
        id:"002",
        name:"黄晓明",
        sex:1,
        honor:"公益明星",
        awards:"在2023年，黄晓明以他的公益行动，成为了全国人民的公益明星。他参与并组织了多项公益活动，为社会做出了巨大的贡献。他用自己的力量和影响力，为社会弱势群体提供帮助，他是我们心中的公益明星！",
        thumb:"https://bkimg.cdn.bcebos.com/pic/cb8065380cd7912397dd9e20bf6c4e82b2b7d0a22a15?x-bce-process=image/format,f_auto/watermark,image_d2F0ZXIvYmFpa2UyNzI,g_7,xp_5,yp_5,P_20/resize,m_lfit,limit_1,h_1080"
      },
      {
        id:"003",
        name:"邓亚萍",
        sex:0,
        honor:"军事英雄",
        awards:"邓亚萍，你是我们的军事英雄和榜样。在2023年，你以你的坚强意志和顽强拼搏的精神，为国家争得了荣誉。你始终保持对国家的忠诚和热爱，你是我们心中的军事英雄！",
        thumb:"https://bkimg.cdn.bcebos.com/pic/d8f9d72a6059252d4c079d96309b033b5bb5b914?x-bce-process=image/format,f_auto/watermark,image_d2F0ZXIvYmFpa2UyNzI,g_7,xp_5,yp_5,P_20/resize,m_lfit,limit_1,h_1080"
      },
      {
        id:"004",
        name:"杨洪基",
        sex:1,
        honor:"青年领袖",
        awards:"杨洪基，你是我们的青年领袖和榜样。在2023年，你以你的才华和努力，成为了全国青年的代表和引领者。你始终关注青年群体的成长和发展，为他们提供了榜样，你是我们心中的青年领袖！",
```

```
38
     thumb:"https://bkimg.cdn.bcebos.com/pic/a9d3fd1f4134970a59da63c193cad1c8a6865db6?x-bce-process=image/format,f_auto/watermark,image_d2F0ZXIvYmFpa2UyNzI,g_7,xp_5,yp_5,P_20/resize,m_lfit,limit_1,h_1080"
39        }
40      ]
41    }
```

说明：

在实际开发中，词条中所展示的数据，应该是来自于后端数据库的，由于暂未对数据查询进行相关学习，此处是直接对其进行了简化。

第3章、小程序组件

3.1节、基础组件

1 在小程序中，组件共分为：视图容器、基础内容、图像组件、导航、表单组件、多媒体等。

3.1.1、视图容器

1、view组件

1 小程序中使用最平凡的组件，可以理解为HTML中的DIV，独占一行。

属性说明

属性	类型	默认值	必填	说明	最低版本
hover-class	string	none	否	指定按下去的样式类。当 `hover-class="none"` 时，没有点击态效果	1.0.0
hover-stop-propagation	boolean	false	否	指定是否阻止本节点的祖先节点出现点击态	1.5.0
hover-start-time	number	50	否	按住后多久出现点击态，单位毫秒	1.0.0
hover-stay-time	number	400	否	手指松开后点击态保留时间，单位毫秒	1.0.0

更多学习资料：https://developers.weixin.qq.com/miniprogram/dev/component/view.html

```wxml
<!--pages/view/view.wxml-->
<view hover-class="hv1">
    视图容器1
    <view hover-class="hv2" hover-stop-propagation="{{flag}}">
        视图容器2
    </view>
</view>

```

```wxss
/* pages/view/view.wxss */
.hv1{
  font-size: 24px;
  color: red;
}
.hv2{
  font-size: 24px;
  color: blue;
}
```

```
<!-- data数据 -->
data: {
    flag:true
}
```

实现效果:



视图容器1

视图容器2

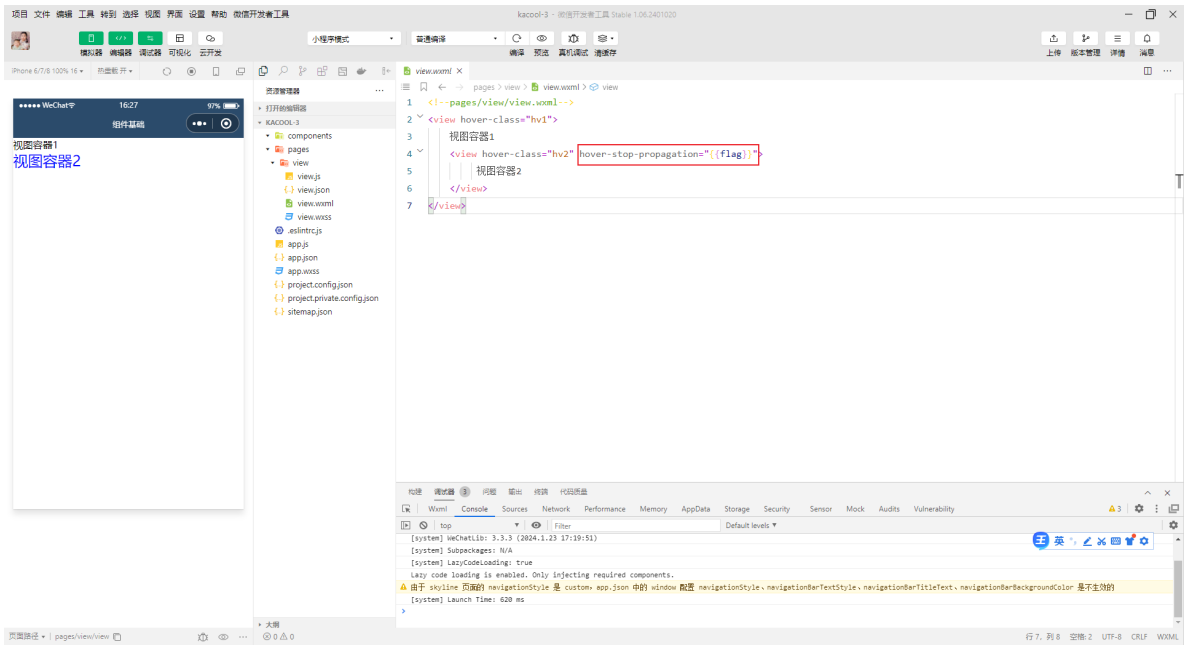
点击外部大容器，内部容器文字同时改变颜色



视图容器1

视图容器2

点击内部容器，阻止外部样式变化(阻止冒泡)



说明：

在实际使用中，需要留意的是，hover-stop-propagation属性值直接写`true`或者`false`都会当成**"真"**，需要采用**插值语法**，当然也可以不写，只需要添加该属性即可。

2、scroll-view组件

1 可滚动视图组件。使用竖向滚动时，需要给scroll-view一个固定高度，通过 WXSS 设置 height。组件属性的长度单位默认为px，2.4.0起支持传入单位(rpx/px)。

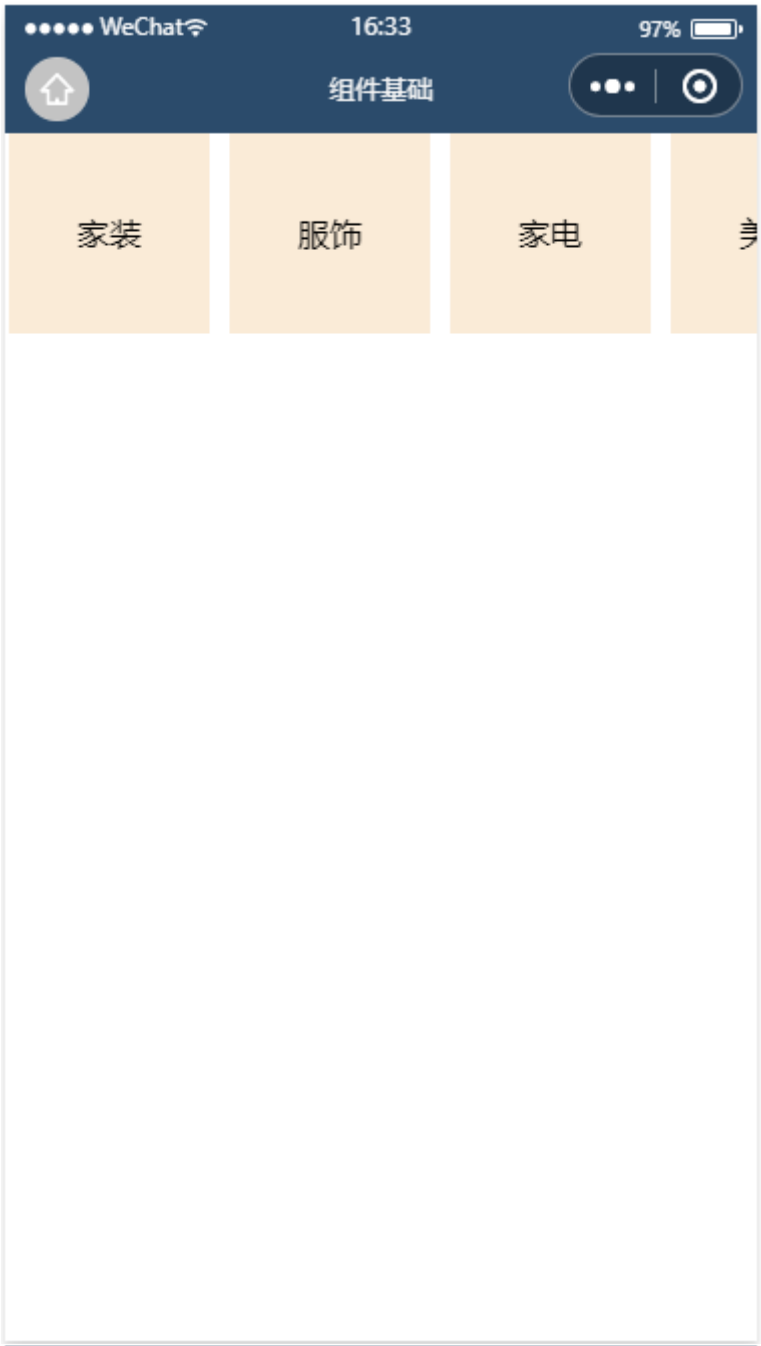
属性	类型	默认值	必填	说明	最低版本
scroll-x	boolean	false	否	允许横向滚动	1.0.0
scroll-y	boolean	false	否	允许纵向滚动	1.0.0
upper-threshold	number/string	50	否	距顶部/左边多远时，触发 scrolltoupper 事件	1.0.0
lower-threshold	number/string	50	否	距底部/右边多远时，触发 scrolltolower 事件	1.0.0
scroll-top	number/string		否	设置竖向滚动条位置	1.0.0
scroll-left	number/string		否	设置横向滚动条位置	1.0.0
scroll-into-view	string		否	值应为某子元素id（id不能以数字开头）。设置哪个方向可滚动，则在哪个方向滚动到该元素	1.0.0
scroll-with-animation	boolean	false	否	在设置滚动条位置时使用动画过渡	1.0.0

```
<!--pages/scroll-view/scroll-view.wxml-->
<scroll-view class="sv" scroll-x="{{flag}}" upper-threshold="100" bindscrolltoupper="scrollTop" scroll-into-view="{{viewId}}" bindscrolltolower="scrollBom">
   <view class="sv-item" wx:for="{{scrollList}}" wx:key="key">
      {{item}}
   </view>
</scroll-view>
```

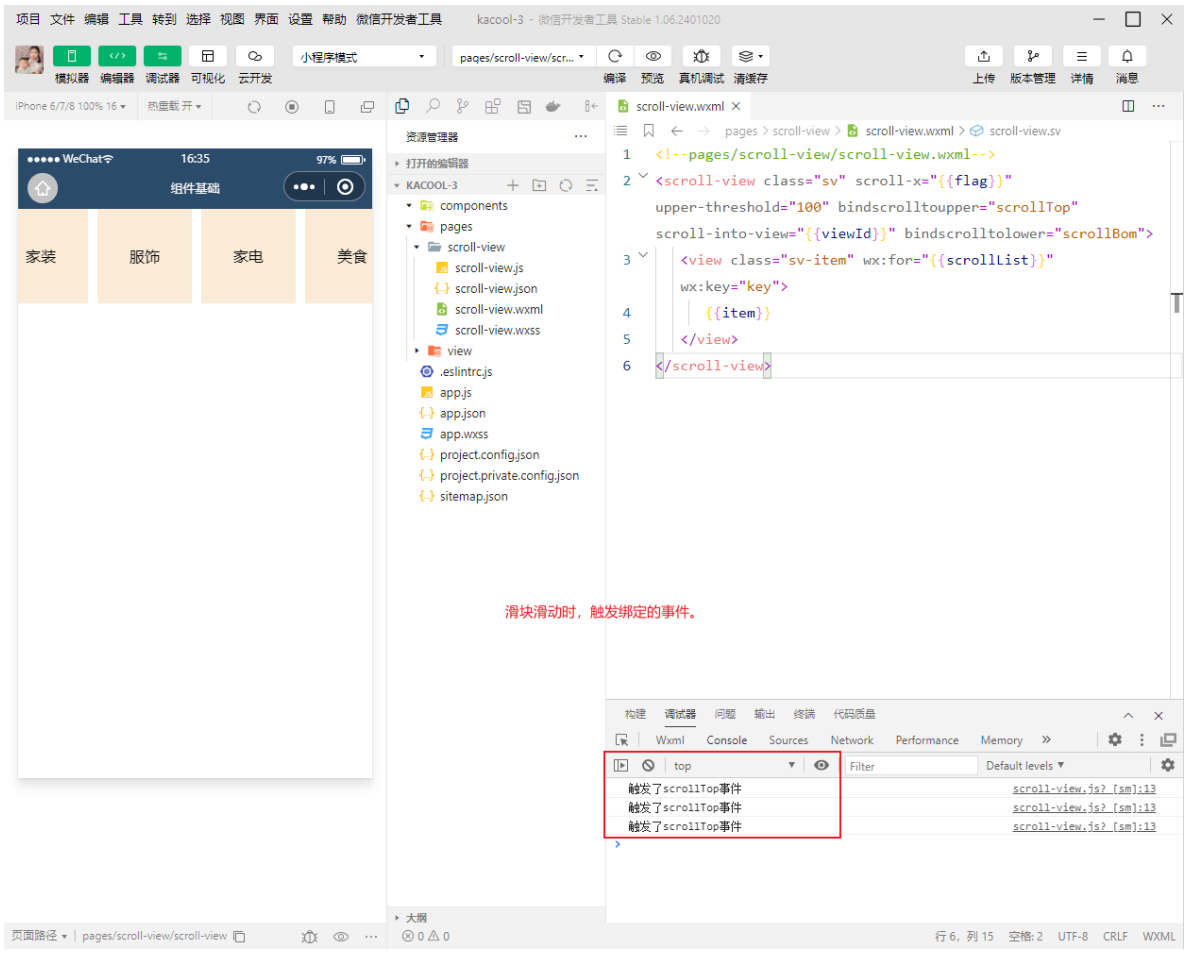
```
/* pages/scroll-view/scroll-view.wxss */
.sv{
  width: 750rpx;
  height: 200rpx;
  white-space: nowrap;
}
.sv-item{
  display: inline-block;
  width: 200rpx;
  height: 200rpx;
  line-height: 100px;
  text-align: center;
  margin: 0rpx 10rpx;
  background-color: antiquewhite;
}
```

```
<!-- scroll-view数据及事件 -->
  data: {
    flag:true,
    scrollList:["家装","服饰","家电","美食","特产"],
  },
  // 自定义scrollTop触顶事件
  scrollTop(){
     console.log("触发了scrollTop事件");
  },
  // 自定义scrollTop触底事件
  scrollBom(){
    console.log("触发了scrollBom事件");
 }
```

实现效果:


WeChat
16:33
97%
组件基础
家装
服饰
家电


WeChat
16:34
97%
组件基础
家电
美食
特产



这里的属性特别多，详细介绍地址：https://developers.weixin.qq.com/miniprogram/dev/component/scroll-view.html

3、swiper组件

1. 滑块视图容器，也称为轮播组件。其内部只可放置swiper-item组件，否则会导致未定义的行为。
2. swiper组件具有默认的宽度和高度样式，在实际使用时，需要将其合理设置。

属性	类型	默认值	必填	说明	最低版本
indicator-dots	boolean	false	否	是否显示面板指示点	1.0.0
indicator-color	color	rgba(0, 0, 0, .3)	否	指示点颜色	1.1.0
indicator-active-color	color	#000000	否	当前选中的指示点颜色	1.1.0
autoplay	boolean	false	否	是否自动切换	1.0.0
current	number	0	否	当前所在滑块的 index	1.0.0
interval	number	5000	否	自动切换时间间隔	1.0.0
duration	number	500	否	滑动动画时长	1.0.0
circular	boolean	false	否	是否采用衔接滑动	1.0.0

```
<!-- swiper及swiper-item组件 -->
<view class="sw">
  <swiper autoplay="{{flag}}" interval="2000" duration="500" circular="{{flag }}" bindchange="currentChange">
    <swiper-item wx:for="{{swiperList}}" wx:key="id" class="sw-item">
      <image src="{{item.imgUrl}}"/>
      <view>
        {{item.title}}
      </view>
    </swiper-item>
  </swiper>
  <view class="sw-tl">
      <text>{{currt}}</text> / <text>{{count}}</text>
  </view>
</view>
```

```
<!-- swiper及swiper-item样式 -->
.sw{
  position: relative;
}
.sw swiper{
  height: 270px;
  text-align: center;
}
.sw-item{
  width: 100%;
  height: 300px;
```

```
}
.sw-item view{
  font-size: 14px;
  color: #333;
  letter-spacing: 3px;
}
.sw-tl{
  position: absolute;
  right: 30px;
  bottom: 0px;
}
.sw-tl text{
  font-size: 14px;
  color: #333;
}
```

```
<!-- data数据及事件 -->
 /**
   * 页面的初始数据
   */
  data: {
    flag: true,
    currt: 1,
    count: 0,
    swiperList: [
      {
        id: 1,
        imgUrl: '../../images/m.jpg',
        title: "小米手机，旗舰选择"
      },
      {
        id: 2,
        imgUrl: '../../images/h.jpg',
        title: "京东美妆，靓丽你我"
      },
      {
        id: 3,
        imgUrl: '../../images/q.jpg',
        title: "京东好货，值得购买"
      },
      {
        id: 4,
        imgUrl: '../../images/z.jpg',
        title: "京东好纸，干净卫生"
      }
    ]
  },
  // 自定义轮播图改变事件
  currentChange(e) {
    this.setData({
      currt: e.detail.current + 1
    })
  },

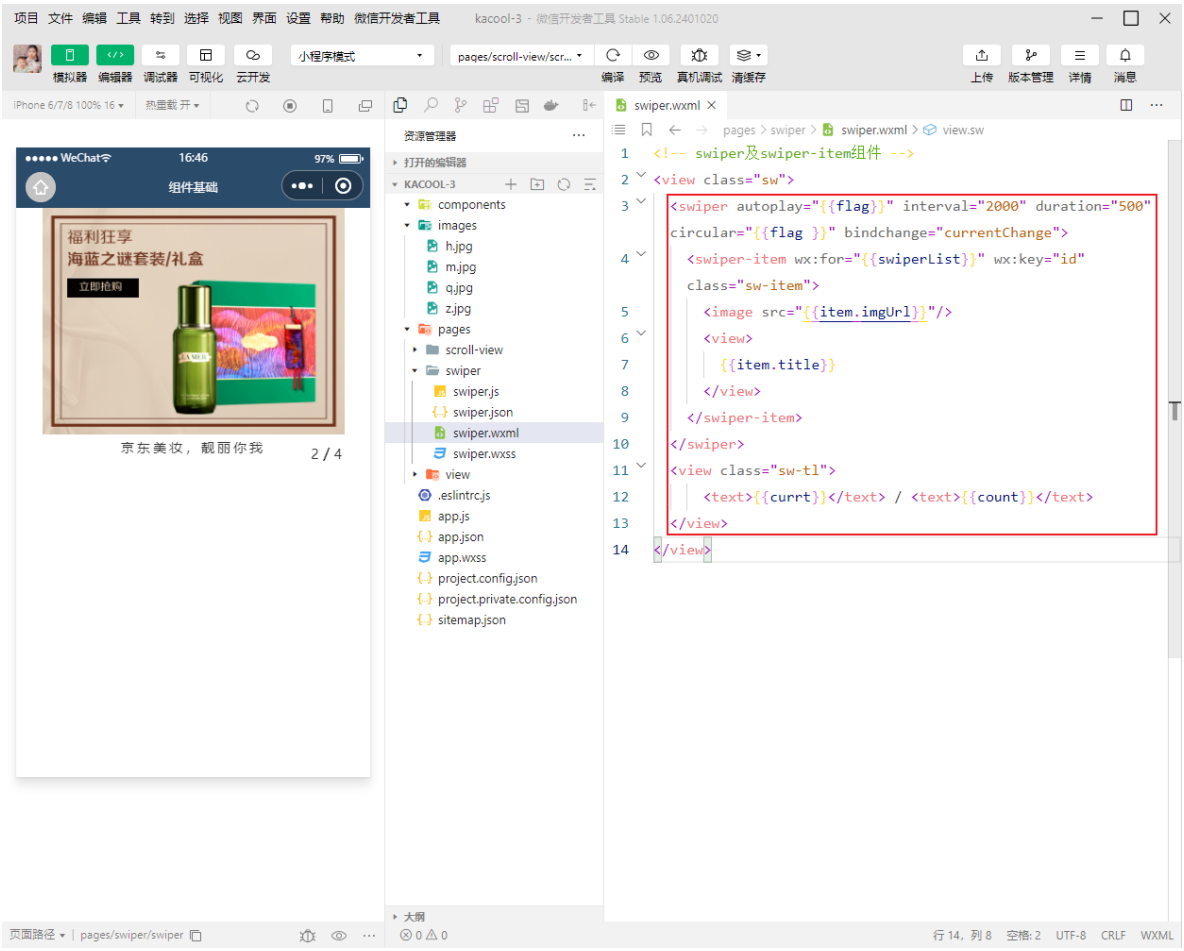
  /**
   * 生命周期函数--监听页面加载
```

```
41      */
42     onLoad(options) {
43        this.setData({
44          count:this.data.swiperList.length
45        })
46     }
```

实现效果：


WeChat
16:45
97%
组件基础
购过瘾
小米手机
立即抢购
小米手机，旗舰选择
1 / 4


WeChat
16:45
97%
组件基础
京选好货
你值得拥有
年的味道
京东好货，值得购买
3 / 4



更多学习资料：https://developers.weixin.qq.com/miniprogram/dev/component/swiper.html

3.1.2、基础内容

1、text组件

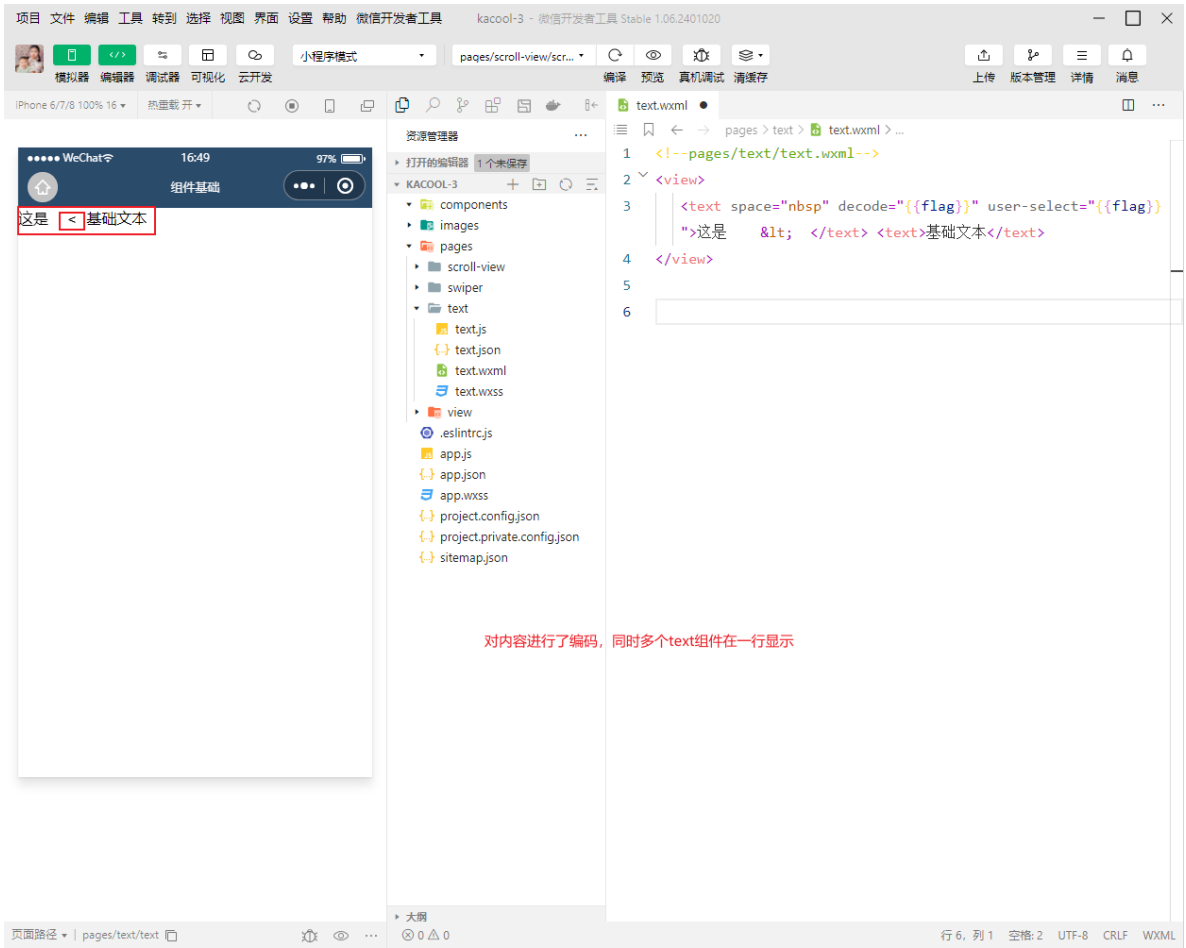
1 类似于HTML中的span标签。

属性	类型	默认值	必填	说明	最低版本
user-select	boolean	false	否	文本是否可选，该属性会使文本节点显示为 inline-block	2.12.1
space	string		否	显示连续空格	1.4.0
decode	boolean	false	否	是否解码	1.4.

```
<!-- text基础内容组件 -->
<view>
    <text space="nbsp" decode="{{flag}}" user-select="{{flag}}">这是    &lt;    基础文本</text>
</view>
```

```
<!-- text组件数据-->
data{
  flag:true
}
```

实现效果：



2、icon组件

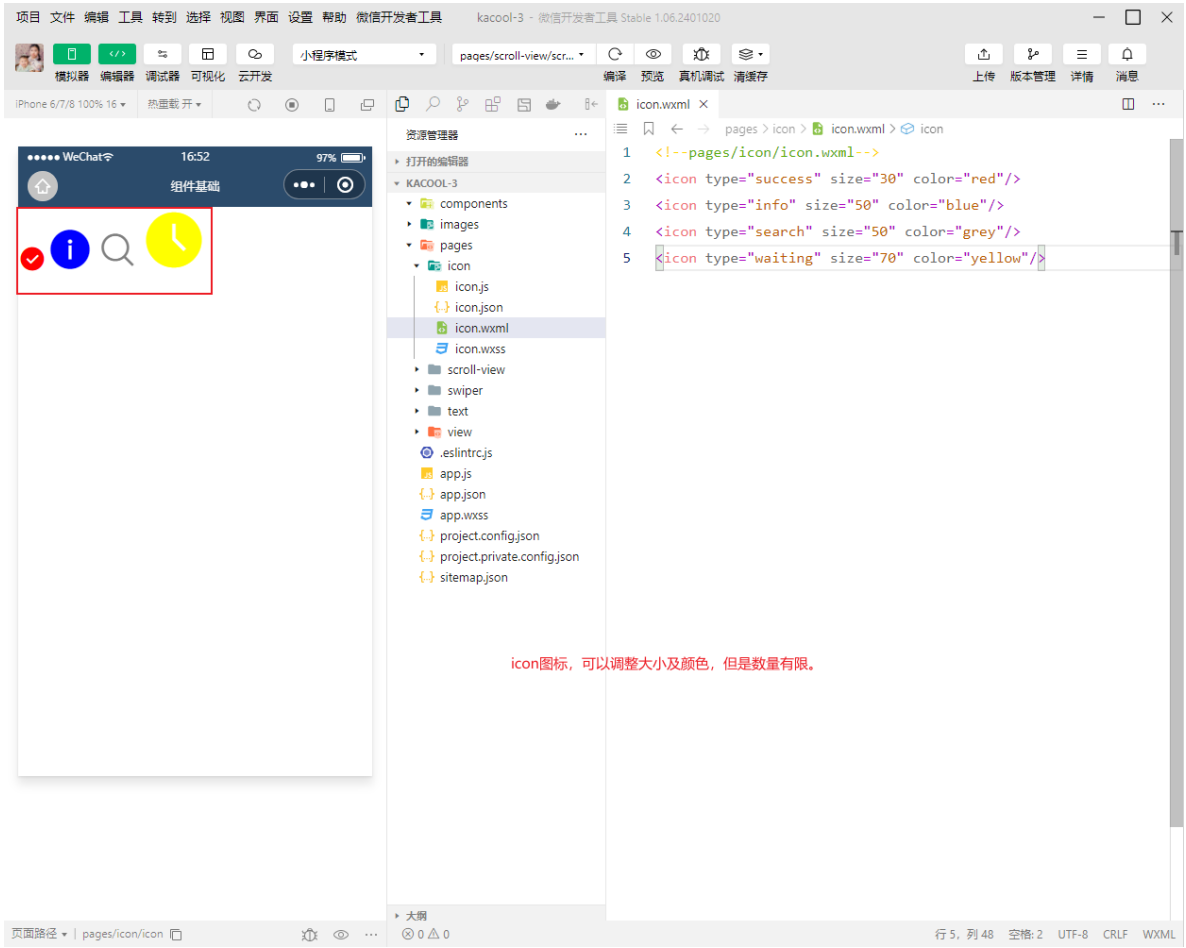
```
小程序内置的图标组件，数量少。
```

属性	类型	默认值	必填	说明	最低版本
type	string		是	icon的类型，有效值：success，success_no_circle，info，warn，waiting，cancel，download，search，clear	1.0.0
size	number/string	23	否	icon的大小，单位默认为px，2.4.0起支持传入单位(rpx/px)，2.21.3起支持传入其余单位(rem 等)。	1.0.0
color	string		否	icon的颜色，同 css 的color	1.0.

更多详细资料：https://developers.weixin.qq.com/miniprogram/dev/component/icon.html

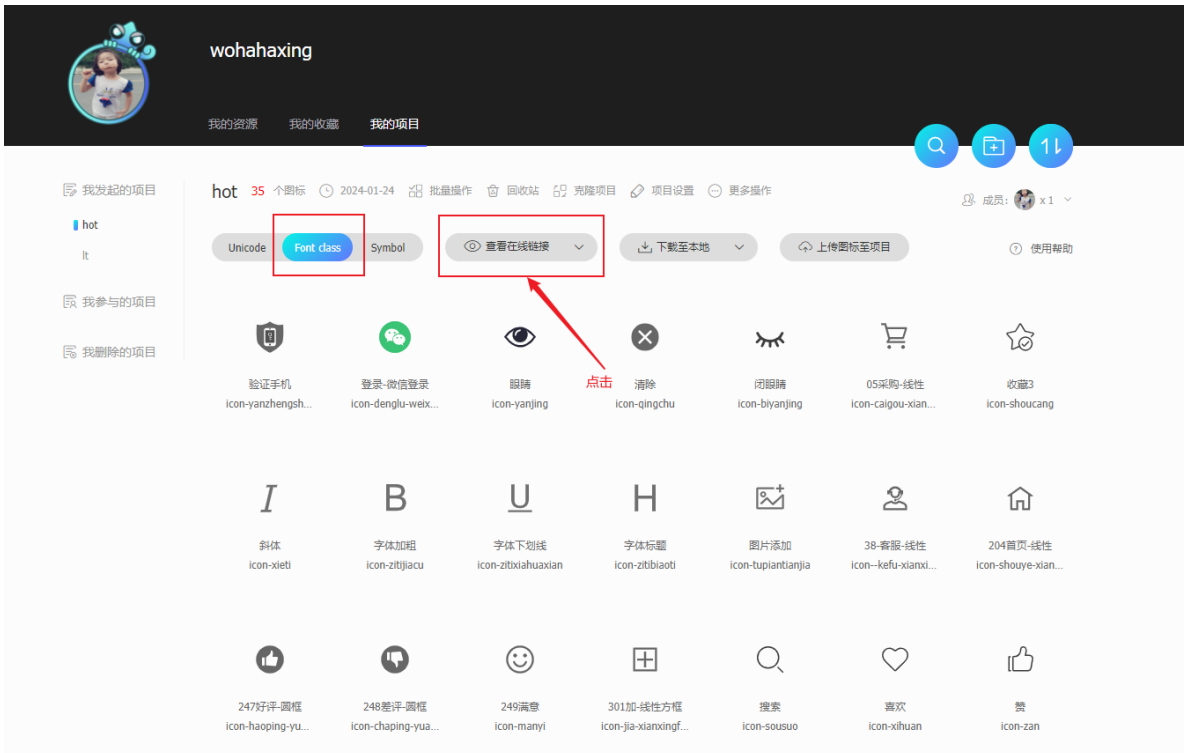
```
<!-- icon组件 -->
<icon type="success" size="30" color="red"/>
<icon type="info" size="50" color="blue"/>
<icon type="search" size="50" color="grey"/>
<icon type="waiting" size="70" color="yellow"/>
```

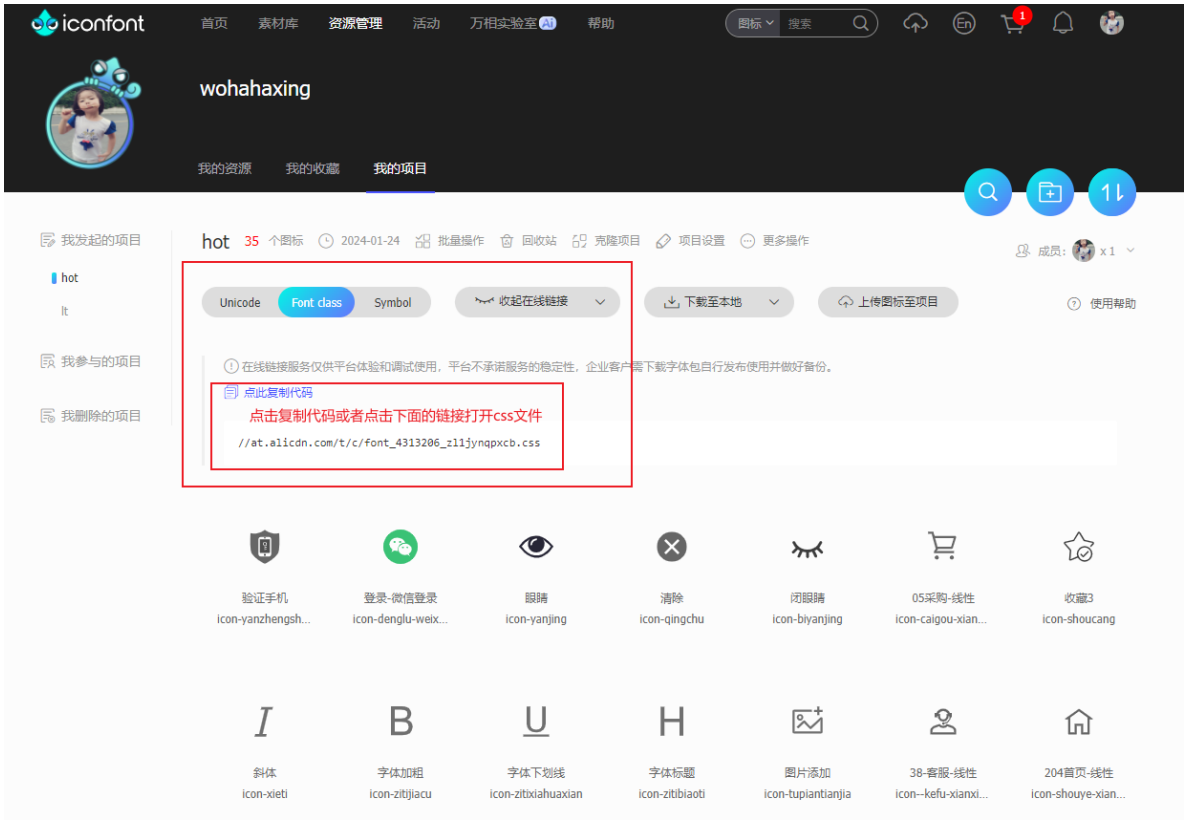
实现效果：



由于字体图标的便捷性，因此在App中使用较多，我们经常会引入外部的字体图标库(iconfont字体图标库：https://www.iconfont.cn/)，使用方式：

1. 打开网站，注册账号
2. 选择图标，并添加至项目
3. 点击 查看在线链接





4. 将复制出来的代码，粘贴到app.wxss全局样式文件中。
5. 使用iconfont中的字体图标

```
/**app.wxss**/
@font-face {
  font-family: "iconfont"; /* Project id 4313206 */
  src: url('//at.alicdn.com/t/c/font_4313206_zl1jynqpxcb.woff2?t=1706066506809') format('woff2'),
       url('//at.alicdn.com/t/c/font_4313206_zl1jynqpxcb.woff?t=1706066506809') format('woff'),
       url('//at.alicdn.com/t/c/font_4313206_zl1jynqpxcb.ttf?t=1706066506809') format('truetype');
}

.iconfont {
  font-family: "iconfont" !important;
  font-size: 16px;
  font-style: normal;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

.icon-yanzhengshouji:before {
  content: "\e634";
}

.icon-denglu-weixindenglu:before {
  content: "\e629";
}

.icon-yanjing:before {
  content: "\e60c";
```

```
}

.icon-qingchu:before {
  content: "\e616";
}

.icon-biyanjing:before {
  content: "\e901";
}

.icon-caigou-xianxing:before {
  content: "\e887";
}

.icon-shoucang:before {
  content: "\e906";
}

.icon-xieti:before {
  content: "\e648";
}

.icon-zitijiacu:before {
  content: "\e64c";
}

.icon-zitixiahuaxian:before {
  content: "\e64d";
}

.icon-zitibiaoti:before {
  content: "\e64e";
}

.icon-tupiantianjia:before {
  content: "\e656";
}

.icon--kefu-xianxing:before {
  content: "\e8ab";
}

.icon-shouye-xianxing:before {
  content: "\e8b9";
}

.icon-haoping-yuankuang:before {
  content: "\e8d8";
}

.icon-chaping-yuankuang:before {
  content: "\e8d9";
}

.icon-manyi:before {
  content: "\e8db";
}
```

```
84
85  .icon-jia-xianxingfangkuang:before {
86    content: "\e8de";
87  }
88
89  .icon-sousuo:before {
90    content: "\e86e";
91  }
92
93  .icon-xihuan:before {
94    content: "\e86f";
95  }
96
97  .icon-zan:before {
98    content: "\e870";
99  }
100
101 .icon-jingxuan:before {
102   content: "\e880";
103 }
104
105 .icon-zengjia:before {
106   content: "\e889";
107 }
108
109 .icon-gengduo:before {
110   content: "\e88b";
111 }
112
113 .icon-toutiao:before {
114   content: "\e892";
115 }
116
117 .icon-gengduo1:before {
118   content: "\e8a6";
119 }
120
121 .icon-biaodankongjianlianxiren:before {
122   content: "\e8c1";
123 }
124
125 .icon-shanchu:before {
126   content: "\e8c2";
127 }
128
129 .icon-biaodankongjiandanxuan:before {
130   content: "\e8c5";
131 }
132
133 .icon-xiugai:before {
134   content: "\e8cf";
135 }
136
137 .icon-tubiao-:before {
138   content: "\e651";
139 }
140
```

```
.icon-dibiao:before {
  content: "\e8e6";
}

.icon-lingquanzhongxin:before {
  content: "\e8e7";
}

.icon-remen:before {
  content: "\e8f4";
}

.icon-zulin:before {
  content: "\e900";
}

```

```
<!--pages/icon/icon.wxml-->
<icon type="success" size="30" color="red" />
<icon type="info" size="50" color="blue" />
<icon type="search" size="50" color="grey" />
<icon type="waiting" size="70" color="yellow" />
<!-- 字体图标 -->
<view style="padding: 40rpx;border: 2rpx solid green;">
  <view>
    <icon class="iconfont icon-denglu-weixindenglu" style="font-size: 120rpx;color:red"></icon>
    <icon class="iconfont icon-caigou-xianxing" style="font-size: 120rpx;color:green"></icon>
    <icon class="iconfont icon-shoucang" style="font-size: 120rpx;color:blue"></icon>
    <icon class="iconfont icon-haoping-yuankuang" style="font-size: 120rpx;color:pink"></icon>
    <icon class="iconfont icon-shanchu" style="font-size: 120rpx;color:purple"></icon>
  </view>
</view>
```



3、progress组件

```
1    进度条组件。组件属性的长度单位默认为px，2.4.0起支持传入单位(rpx/px)
```

属性	类型	默认值	必填	说明	最低版本
percent	number		否	百分比0~100	1.0.0
show-info	boolean	false	否	在进度条右侧显示百分比	1.0.0
border-radius	number/string	0	否	圆角大小	2.3.1
font-size	number/string	16	否	右侧百分比字体大小	2.3.1
stroke-width	number/string	6	否	进度条线的宽度	1.0.0
color	string	#09BB07	否	进度条颜色（请使用activeColor）	1.0.0
activeColor	string	#09BB07	否	已选择的进度条的颜色	1.0.0
backgroundColor	string	#EBEBEB	否	未选择的进度条的颜色	1.0.0
active	boolean	false	否	进度条从左往右的动画	1.0.0
active-mode	string	backwards	否	backwards：动画从头播；forwards：动画从上次结束点接着播	1.7.0
duration	number	30	否	进度增加1%所需毫秒数	2.8.2
bindactiveend	eventhandle		否	动画完成事件	2.4

```
<!-- progress组件 -->
<progress percent="60" show-info="{{flag}}" font-size="16px" activeColor="green" active="{{flag}}"/>

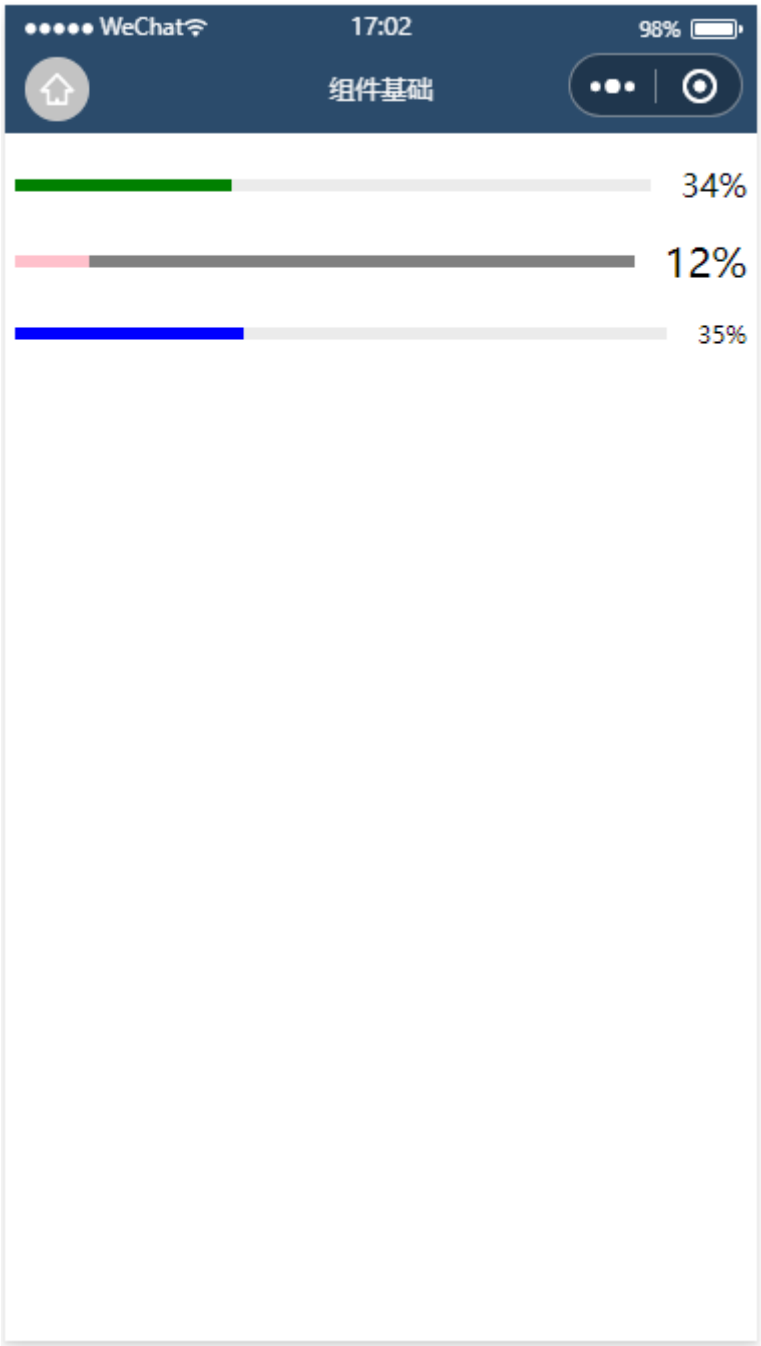
<progress percent="40" show-info="{{flag}}" font-size="20px" activeColor="pink"  backgroundColor="grey" duration="100" active="{{flag}}"/>

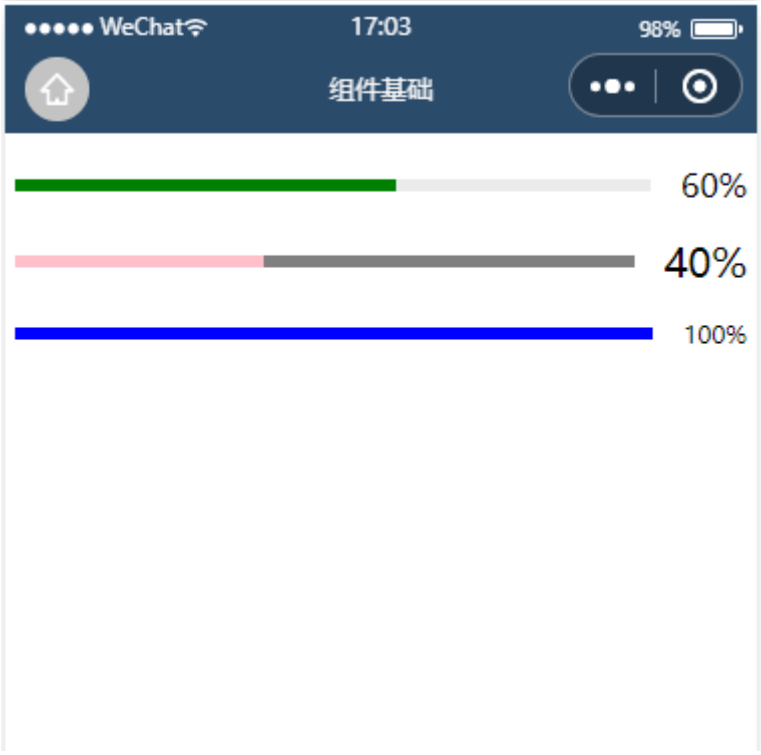
<progress percent="100" show-info="{{flag}}" font-size="12px" activeColor="blue" active="{{flag}}" bindactiveend="proFnish"/>
```

```
//data数据及事件
data: {
    flag:true
},
// 自定义滚动条完成事件
proFnish(e){
   console.log("加载完成...");
}
```

```
/* pages/progress/progress.wxss */
.pg progress{
  margin: 30rpx 10rpx;
}
```

实现效果：


WeChat
17:02
98%
组件基础
34%
12%
35%


WeChat
17:03
98%
组件基础
60%
40%
100%

更多学习资料：https://developers.weixin.qq.com/miniprogram/dev/component/progress.html

3.2节、image组件

> image组件，用于展示图像，默认存在宽和高，分别是：320px*240px

属性	类型	默认值	必填	说明	最低版本
src	string		否	图片资源地址	1.0.0
mode	string	scaleToFill	否	图片裁剪、缩放的模式	1.0.0
lazy-load	boolean	false	否	图片懒加载，在即将进入一定范围（上下三屏）时才开始加载	1.5.0
show-menu-by-longpress	boolean	false	否	长按图片显示发送给朋友、收藏、保存图片、搜一搜、打开名片/前往群聊/打开小程序（若图片中包含对应二维码或小程序码）的菜单。	2.7.0

mode属性：

合法值	说明	最低版本
scaleToFill	缩放模式，不保持纵横比缩放图片，使图片的宽高完全拉伸至填满 image 元素	
aspectFit	缩放模式，保持纵横比缩放图片，使图片的长边能完全显示出来。也就是说，可以完整地将图片显示出来。	
aspectFill	缩放模式，保持纵横比缩放图片，只保证图片的短边能完全显示出来。也就是说，图片通常只在水平或垂直方向是完整的，另一个方向将会发生截取。	
widthFix	缩放模式，宽度不变，高度自动变化，保持原图宽高比不变	
heightFix	缩放模式，高度不变，宽度自动变化，保持原图宽高比不变	2.10.3

代码：

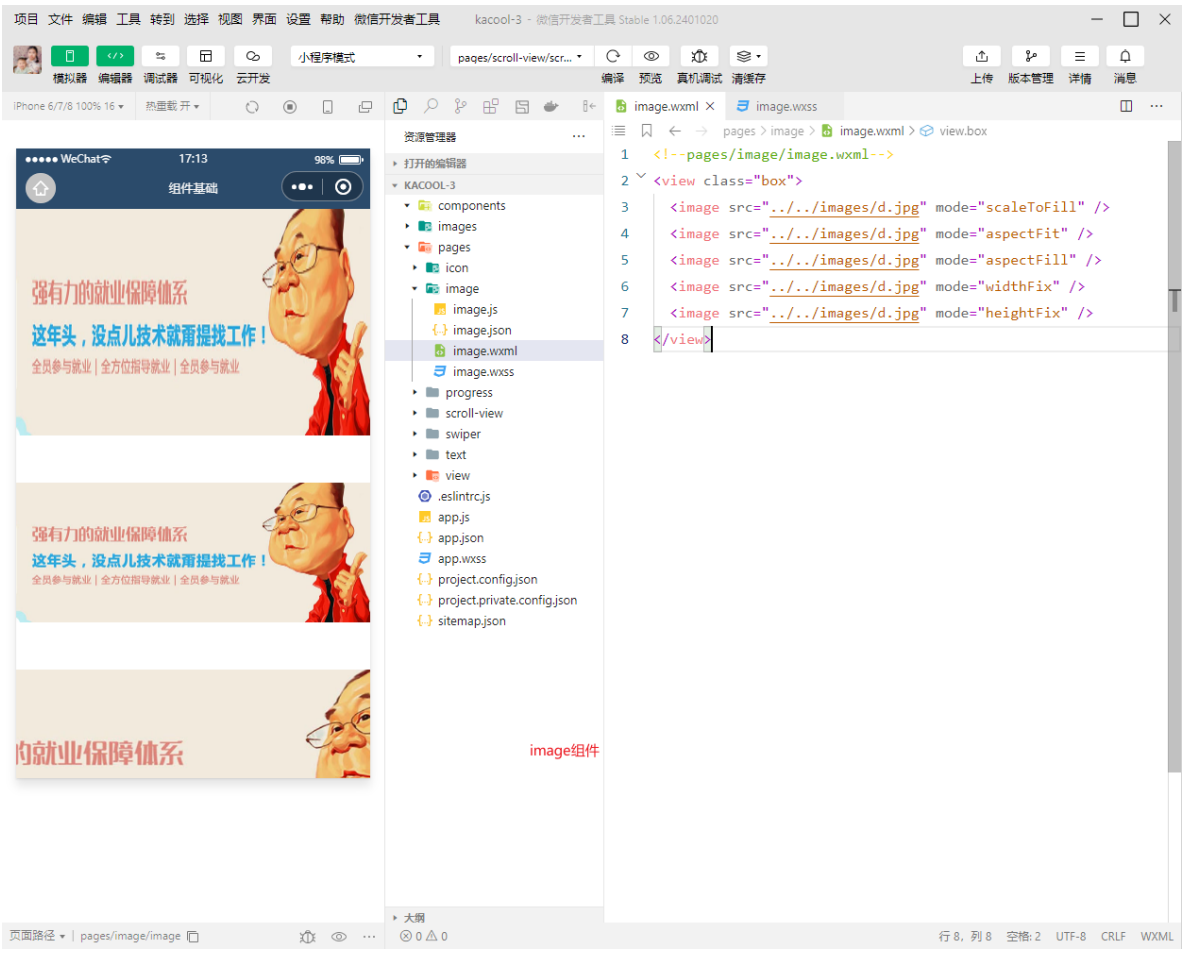
```
<!-- image组件 -->
<scroll-view class="box" scroll-y="true" style="height:1200rpx">
    <image src="../../images/d.jpg" mode="scaleToFill" />
    <image src="../../images/d.jpg" mode="aspectFit" />
    <image src="../../images/d.jpg" mode="aspectFill" />
    <image src="../../images/d.jpg" mode="widthFix" />
    <image src="../../images/d.jpg" mode="heightFix" />
</scroll-view>
```

```
样式：
.box{
  width: 750rpx;
  image{
    width: 100%;
  }
}
```

效果：


●●●●● WeChat
17:07
98%
组件基础
scaleToFill：不保持比例，自动拉升至元素宽高
强有力的就业保障体系
这年头，没点儿技术就甭提找工作！
全员参与就业 | 全方位指导就业 | 全员参与就业
aspectFit：保持比例，长边显示
强有力的就业保障体系
这年头，没点儿技术就甭提找工作！
全员参与就业 | 全方位指导就业 | 全员参与就业
aspectFill：保持比例，短边显示
的就业保障体系
●●●●● WeChat
17:11
98%
组件基础
aspectFill：保持比例，短边显示
的就业保障体系
，没点儿技术就甭提找工作！
业 | 全方位指导就业 | 全员参与就业
widthFix：保持比例，宽度显示，高度自适应
强有力的就业保障体系
这年头，没点儿技术就甭提找工作！





更多学习资料：https://developers.weixin.qq.com/miniprogram/dev/component/image.html

3.3节、navigator组件

1 页面超级链接，既可以链接到普通页面，也可以导航到TabBar(底部导航栏)页面。

属性	类型	默认值	必填	说明	最低版本
target	string	self	否	在哪个目标上发生跳转，默认当前小程序	2.0.7
url	string		否	当前小程序内的跳转链接	1.0.0
open-type	string	navigate	否	跳转方式	1.0.0
delta	number	1	否	当 open-type 为 'navigateBack' 时有效，表示回退的层数	1.0.0
app-id	string		否	当`target="miniProgram"`且`open-type="navigate"`时有效，要打开的小程序 appId	2.0.7
path	string		否	当`target="miniProgram"`且`open-type="navigate"`时有效，打开的页面路径，如果为空则打开首页	2.0.7
hover-class	string	navigator-hover	否	指定点击时的样式类，当`hover-class="none"`时，没有点击态效果	1.0.0

target属性：

合法值	说明
self	当前小程序
miniProgram	其它小程序

open-type属性：

合法值	说明	最低版本
navigate	对应 wx.navigateTo 或 wx.navigateToMiniProgram 的功能	
redirect	对应 wx.redirectTo 的功能	
switchTab	对应 wx.switchTab 的功能	
reLaunch	对应 wx.reLaunch 的功能	1.1.0
navigateBack	对应 wx.navigateBack 或 wx.navigateBackMiniProgram （基础库 2.24.4 版本支持）的功能	1.1.0
exit	退出小程序，`target="miniProgram"`时生效	2.1.0

代码：

```
<!--pages/navigator/navigator.wxml-->
跳转到:
<navigator open-type="navigate" url="/pages/icon/icon">图标</navigator>
<navigator open-type="navigate" url="/pages/swiper/swiper" open-type="navigate">轮播图</navigator>
<navigator open-type="navigate" url="/pages/scroll-view/scroll-view" open-type="navigate">滑块</navigator>
```

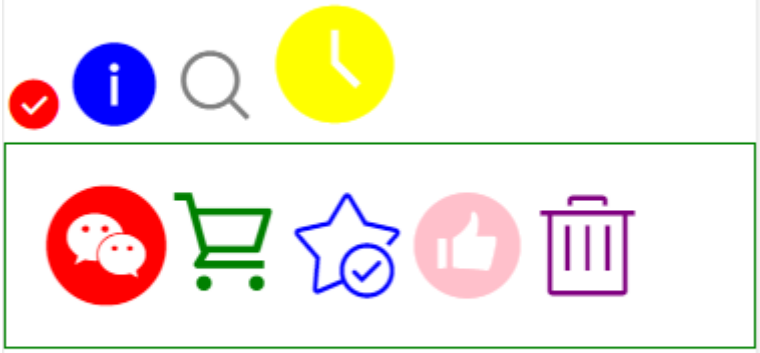
```
<!-- navigator组件:scroll-view页面 -->
<navigator open-type="navigateBack" delta="1">返回</navigator>
```

效果:



跳转到:
图标
轮播图
滑块



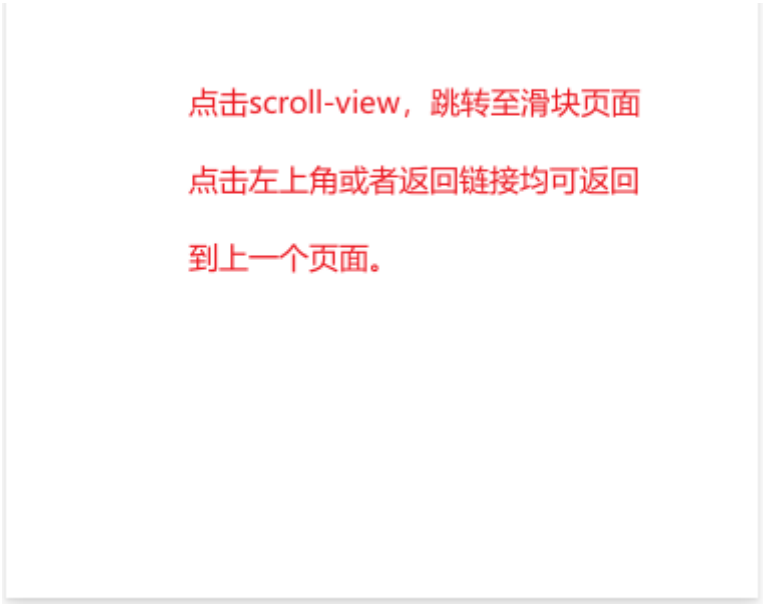


点击icon链接，跳转至icon页面，点击左箭头可以返回上一个页面。



点击swiper，跳转至轮播图页面，同样左上角可以点击返回。





需要注意的是，目前展现的是普通页面之间的跳转动作，如果想要跳转至底部导航栏页面，open-type属性值则为不同。

更多学习资料：https://developers.weixin.qq.com/miniprogram/dev/component/navigator.html

3.4节、tabBar组件

```
底部导航栏组件
```

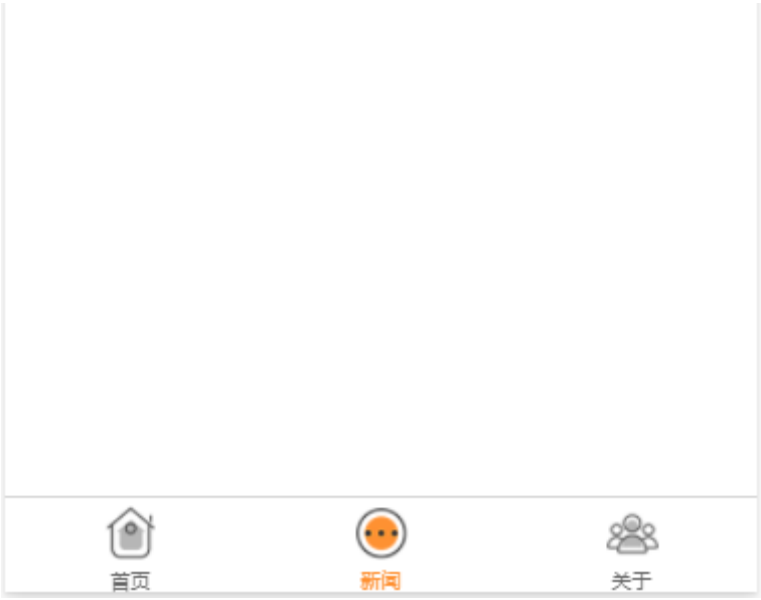
```
app.json页面
"tabBar": {
    "color": "#666",
    "selectedColor": "#fe9132",
    "list": [
      {
        "pagePath": "pages/index/index",
        "text": "首页",
        "iconPath": "/images/home.png",
        "selectedIconPath": "/images/home-h.png"
      },
      {
        "pagePath": "pages/swiper/swiper",
        "text": "新闻",
        "iconPath": "/images/list.png",
        "selectedIconPath": "/images/list-h.png"
      },
      {
        "pagePath": "pages/icon/icon",
        "text": "关于",
        "iconPath": "/images/user.png",
        "selectedIconPath": "/images/user-h.png"
      }
    ]
  }
```

```
<!-- tabBar组件:index页面 -->
跳转至:
<navigator open-type="switchTab" url="/pages/swiper/swiper">新闻</navigator>
<navigator open-type="navigate" url="/pages/scroll-view/scroll-view">滑块</navigator>
<navigator open-type="switchTab" url="/pages/icon/icon">关于</navigator>
```

效果:

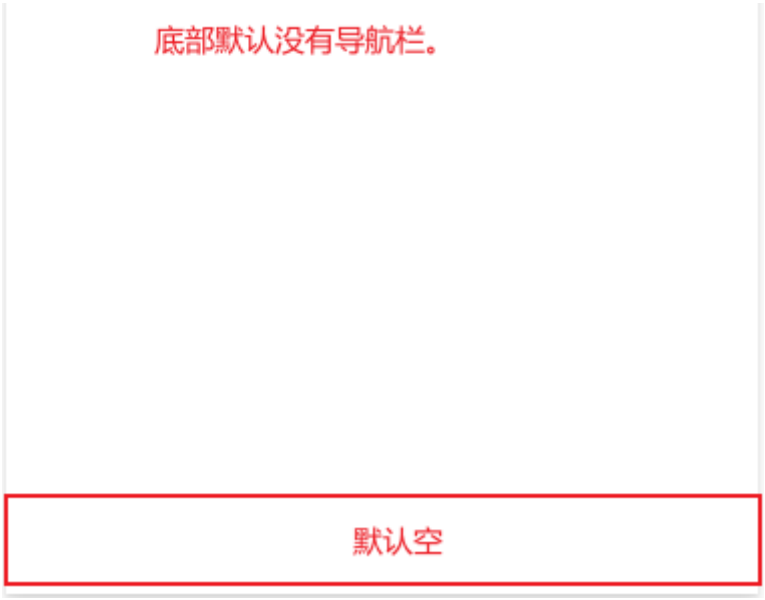

●●●●● WeChat
17:39
98%
组件基础
跳转至:
新闻
滑块
关于
首页
新闻
关于


●●●●● WeChat
17:39
98%
组件基础
福利狂享
海蓝之谜套装/礼盒
立即抢购
LA MER
京东美妆，靓丽你我
2 / 4


首页
新闻
关于


WeChat
17:42
98%
组件基础
所有的tabBar页面，左上角没有返回箭头，但是底部均有导航栏。
首页
新闻
关于


WeChat
17:41
98%
组件基础
家装
服饰
家电
返回
所有非tabBar页面，左上角有返回箭头



更多学习资料：https://developers.weixin.qq.com/miniprogram/dev/reference/configuration/app.html#tabBar

3.5节、表单组件

3.5.1、button组件

> button按钮，类似于HTML中的按钮标签。

属性	类型	默认值	必填	说明	最低版本
size	string	default	否	按钮的大小	1.0.0
type	string	default	否	按钮的样式类型	1.0.0
plain	boolean	false	否	按钮是否镂空，背景色透明	1.0.0
disabled	boolean	false	否	是否禁用	1.0.0
loading	boolean	false	否	名称前是否带 loading 图标	1.0.0
form-type	string		否	用于 form 组件，点击分别会触发 form 组件的 submit/reset 事件	1.0.0
open-type	string		否	微信开放能力	1.1.0
hover-class	string	button-hover	否	指定按钮按下去的样式类。当 hover-class="none" 时，没有点击态效果	1.0.0

> type的属性值：

type	string	default	否	按钮的样式类型	1.0.0

合法值说明	primary绿色 default白色 warn红色				
type	string	default	否	按钮的样式类型	1.0.0

form-type的属性值:

form-type	string		否	用于 form 组件，点击分别会触发 form 组件的 submit/reset 事件	1.0.0
合法值说明	submit提交表单 reset重置表单				

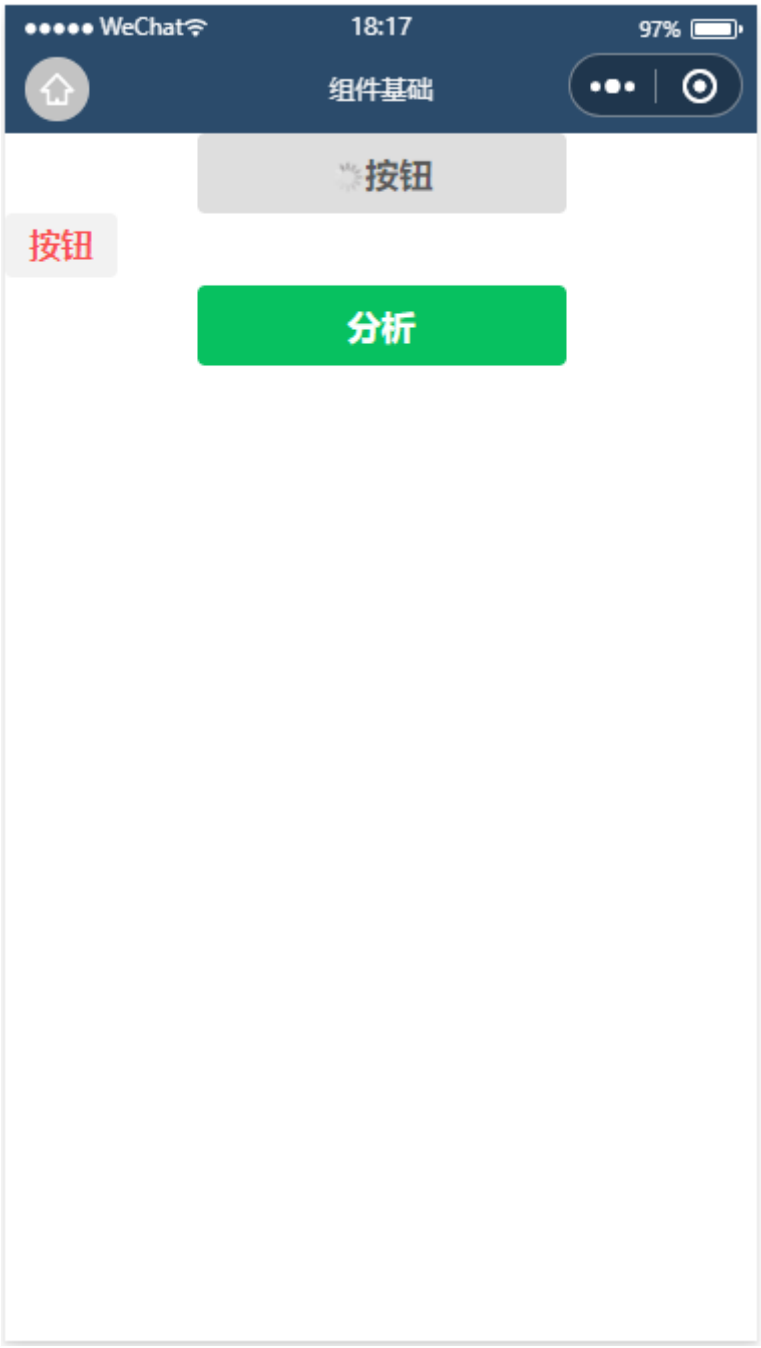
open-type的属性值:

合法值	说明	最低版本
contact	打开客服会话，如果用户在会话中点击消息卡片后返回小程序，可以从 bindcontact 回调中获得具体信息，具体说明	1.1.0
liveActivity	通过前端获取新的一次性订阅消息下发机制使用的 code	2.26.2
share	触发用户转发，使用前建议先阅读使用指引	1.2.0
getPhoneNumber	手机号快速验证，向用户申请，并在用户同意后，快速填写和验证手机，具体说明 （*小程序插件中不能使用*）	1.2.0
getRealtimePhoneNumber	手机号实时验证，向用户申请，并在用户同意后，快速填写和实时验证手机号。具体说明（*小程序插件中不能使用*）	2.24.4
getUserInfo	获取用户信息，可以从bindgetuserinfo回调中获取到用户信息 （*小程序插件中不能使用*）	1.3.0
launchApp	打开APP，可以通过app-parameter属性设定向APP传的参数具体说明	1.9.5
openSetting	打开授权设置页	2.0.7
feedback	打开“意见反馈”页面，用户可提交反馈内容并上传日志，开发者可以登录小程序管理后台后进入左侧菜单“客服反馈”页面获取到反馈内容	2.1.0
chooseAvatar	获取用户头像，可以从bindchooseavatar回调中获取到头像信息	2.21.2
agreePrivacyAuthorization	用户同意隐私协议按钮。用户点击一次此按钮后，所有已声明过的隐私接口可以正常调用。可通过 bindagreeprivacyauthorization 监听用户同意隐私协议事件。隐私合规开发指南详情可见《小程序隐私协议开发指南》	2.32.3

```
<!-- button组件 -->
<button type="default" loading="{{flag}}">按钮</button>
<button type="warn" size="mini" form-type="submit">按钮</button>
<button type="primary" open-type="share">分析</button>
```

```
data数据:
data{
  flag:true
}
```

实现效果：


WeChat
18:17
97%
组件基础
按钮
按钮
分析


WeChat
18:18
97%
组件基础
按钮
按钮
发送给:
虚拟好友
kacool-3



> 说明：
>
> 在实际开发中，按钮除了用于点击事件，还主要用于表单数据的提交。

更多属性学习地址：https://developers.weixin.qq.com/miniprogram/dev/component/button.html

3.5.2、checkbox组件

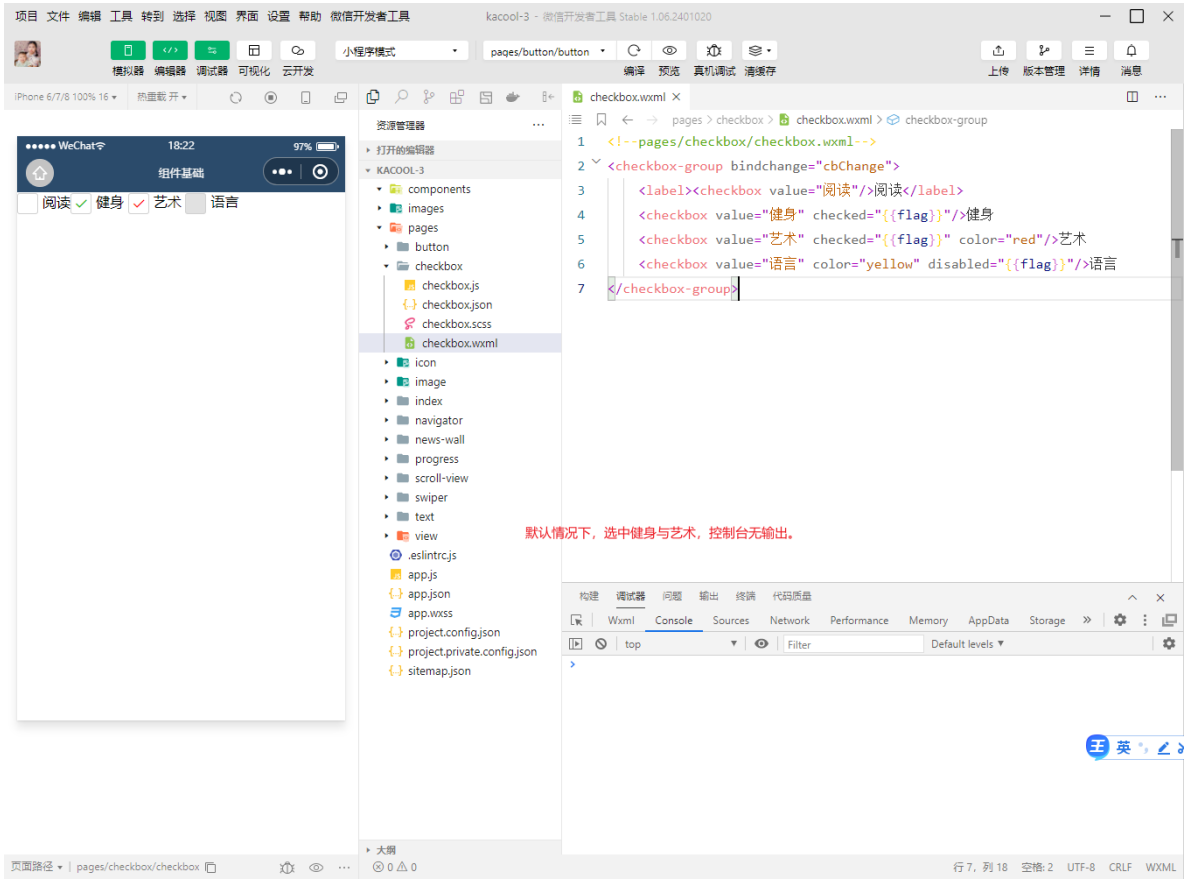
```
1  复选框组件。通常与label组件、checkbox-group组件配合使用。
```

属性	类型	默认值	必填	说明	最低版本
value	string		否	checkbox标识，选中时触发checkbox-group的change 事件，并携带checkbox 的 value	1.0.0
disabled	boolean	false	否	是否禁用	1.0.0
checked	boolean	false	否	当前是否选中，可用来设置默认选中	1.0.0
color	string	#09BB07	否	checkbox的颜色，同 css 的color	1.0.

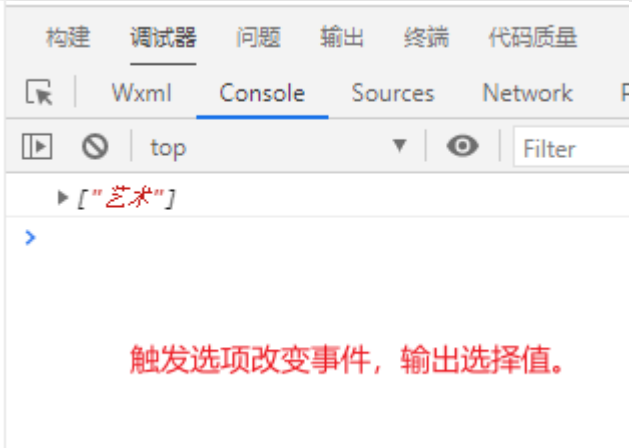
```
1  <!-- checkbox-group、label、checkbox组件 -->
2  <checkbox-group bindchange="cbChange">
3      <label><checkbox value="阅读"/>阅读</label>
4      <checkbox value="健身" checked="{{flag}}"/>健身
5      <checkbox value="艺术" checked="{{flag}}" color="red"/>艺术
6      <checkbox value="语言" color="yellow" disabled="{{flag}}"/>语
   言
7  </checkbox-group>
```

```
data数据及事件：
data: {
    flag:true
},
// 自定义复选框选中改变事件
cbChange(e){
  console.log(e.detail.value);
}
```

实现效果：

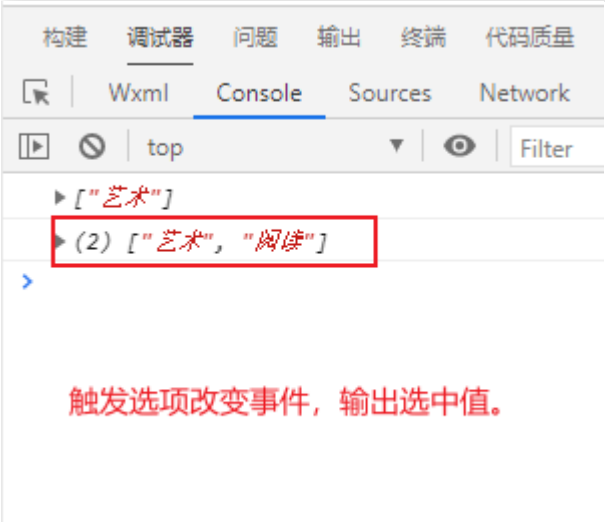






触发选项改变事件，输出选择值。





触发选项改变事件，输出选中值。

更多属性学习地址：https://developers.weixin.qq.com/miniprogram/dev/component/checkbox.html

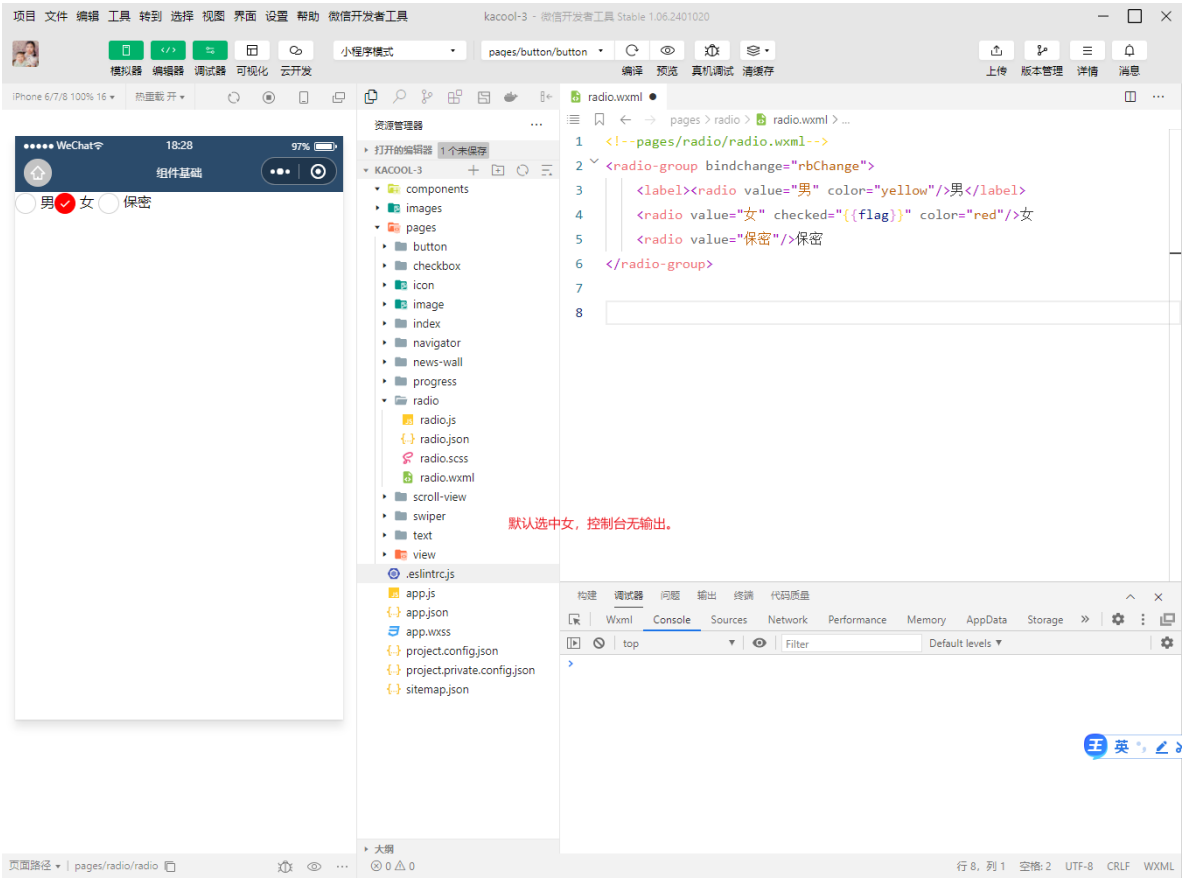
3.5.3、radio组件

单选按钮，跟复选框类似，通常结合radio-group、label组件一起使用。

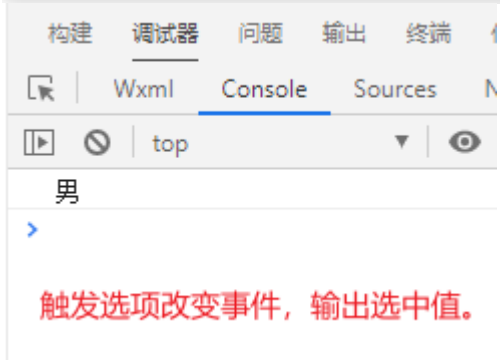
```
<!-- radio-group、label、radio组件 -->
<radio-group bindchange="rbChange">
    <label><radio value="男" color="yellow"/>男</label>
    <radio value="女" checked="{{flag}}" color="red"/>女
    <radio value="保密"/>保密
</radio-group>
```

```
data数据及事件：
data: {
    flag:true
},
// 自定义复选框选中改变事件
rbChange(e){
  console.log(e.detail.value);
}
```

实现效果：

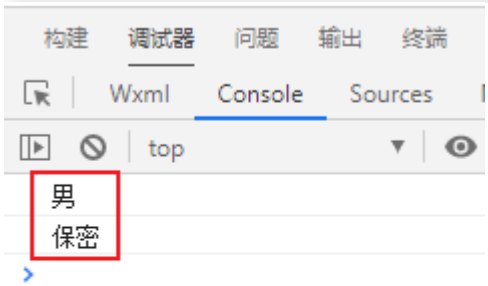






触发选项改变事件，输出选中值。





触发选项改变事件，输出选中值。

更多属性学习地址：https://developers.weixin.qq.com/miniprogram/dev/component/radio.html

3.5.4、switch组件

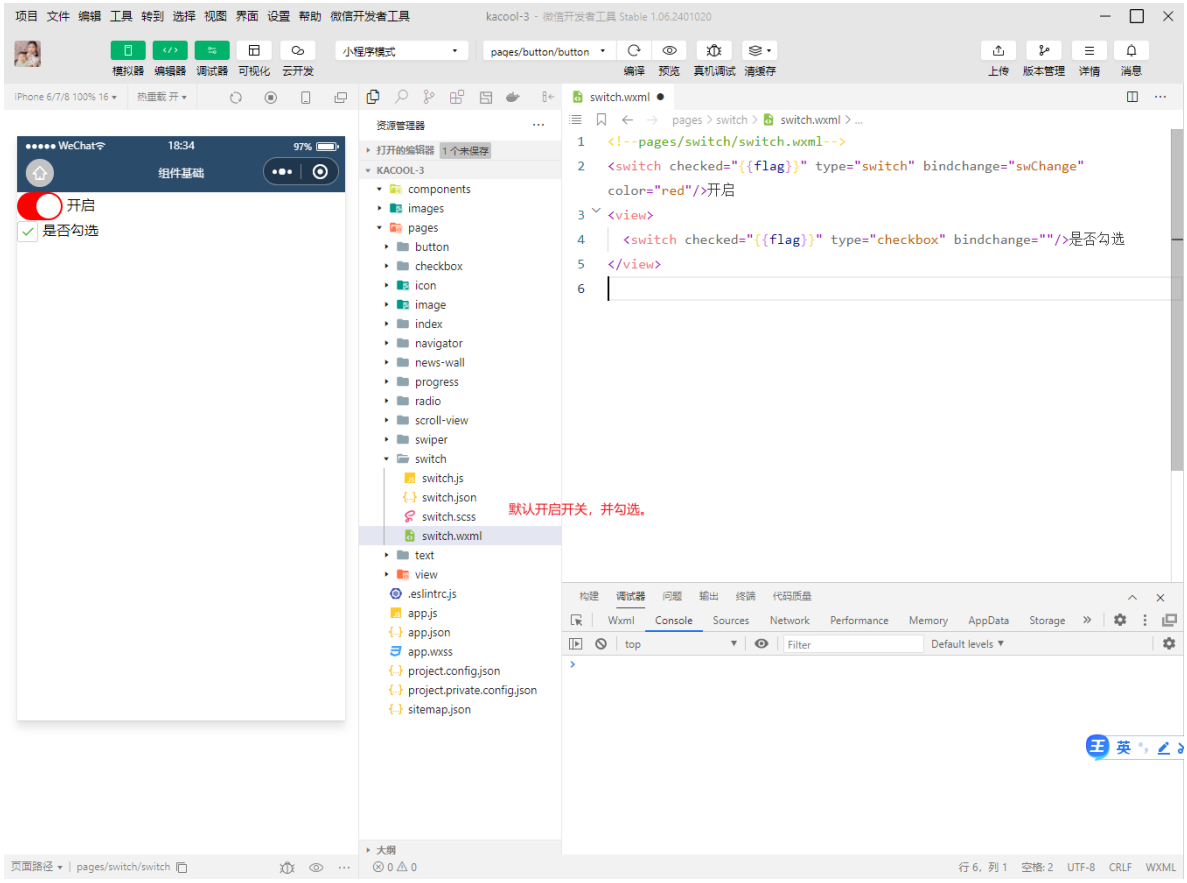
```
1  开关选择器组件。
```

属性	类型	默认值	必填	说明	最低版本
checked	boolean	false	否	是否选中	1.0.0
disabled	boolean	false	否	是否禁用	1.0.0
type	string	switch	否	样式，有效值：switch，checkbox	1.0.0
color	string	#04BE02	否	switch 的颜色，同 css 的 color	1.0.0
bindchange	eventhandle		否	点击导致 checked 改变时会触发 change 事件，event.detail={ value}	1.0.

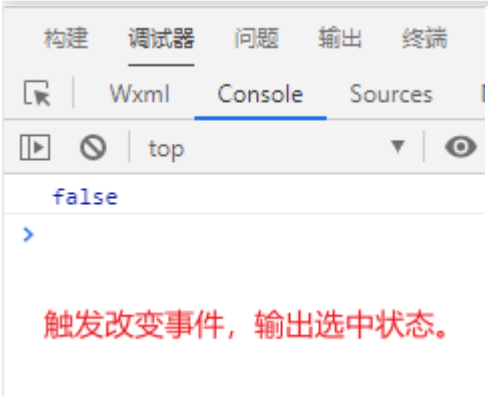
```
<!-- switch组件 -->
<switch checked="{{flag}}" type="switch" bindchange="swChange" color="red"/>开启
<view>
  <switch checked="{{flag}}" type="checkbox" bindchange=""/>是否勾选
</view>
```

```
data数据及事件：
data: {
    flag:true
},
// 自定义开关改变事件
swChange(e){
  console.log(e.detail.value);
}
```

实现效果：


项目 文件 编辑 工具 转到 选择 视图 界面 设置 帮助 微信开发者工具
kacool-3 - 微信开发者工具 Stable 1.06.2401020
模拟器 编辑器 调试器 可视化 云开发
小程序模式
pages/button/button
编译 预览 真机调试 清缓存
上传 版本管理 详情 消息
iPhone 6/7/8 100% 16
热重载 开
WeChat 18:34 97%
组件基础
开启
是否勾选
资源管理器
打开的编辑器 1 个未保存
KACOOL-3
components
images
pages
button
checkbox
icon
image
index
navigator
news-wall
progress
radio
scroll-view
swiper
switch
switch.js
switch.json
switch.scss
switch.wxml
text
view
.eslintrc.js
app.js
app.json
app.wxss
project.config.json
project.private.config.json
sitemap.json
switch.wxml
pages > switch > switch.wxml > ...
1 <!--pages/switch/switch.wxml-->
2 <switch checked="{{flag}}" type="switch" bindchange="swChange"
color="red"/>开启
3 <view>
4 <switch checked="{{flag}}" type="checkbox" bindchange=""/>是否勾选
5 </view>
6
默认开启开关，并勾选。
构建 调试器 问题 输出 终端 代码质量
Wxml Console Sources Network Performance Memory AppData Storage
top
Filter
Default levels
大纲
页面路径 pages/switch/switch
行 6，列 1 空格: 2 UTF-8 CRLF WXML





更多属性学习地址：https://developers.weixin.qq.com/miniprogram/dev/component/switch.html

3.5.5、slider组件

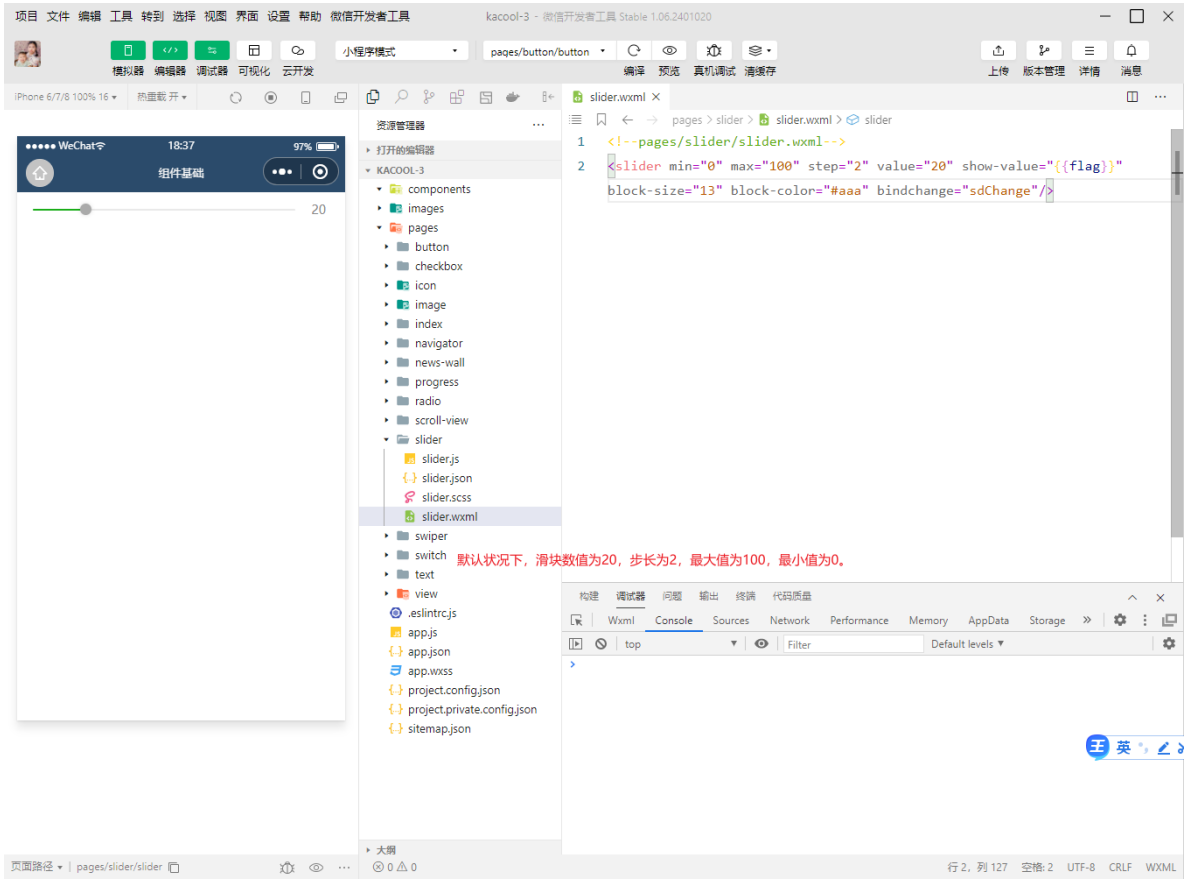
```
1    滑动块组件。
```

属性	类型	默认值	必填	说明	最低版本
min	number	0	否	最小值	1.0.0
max	number	100	否	最大值	1.0.0
step	number	1	否	步长，取值必须大于 0，并且可被(max - min)整除	1.0.0
disabled	boolean	false	否	是否禁用	1.0.0
value	number	0	否	当前取值	1.0.0
color	color	#e9e9e9	否	背景条的颜色（请使用 backgroundColor）	1.0.0
selected-color	color	#1aad19	否	已选择的颜色（请使用 activeColor）	1.0.0
activeColor	color	#1aad19	否	已选择的颜色	1.0.0
backgroundColor	color	#e9e9e9	否	背景条的颜色	1.0.0
block-size	number	28	否	滑块的大小，取值范围为 12 - 28	1.9.0
block-color	color	#ffffff	否	滑块的颜色	1.9.0
show-value	boolean	false	否	是否显示当前 value	1.0.0
bindchange	eventhandle		否	完成一次拖动后触发的事件，event.detail = {value}	1.0.0
bindchanging	eventhandle		否	拖动过程中触发的事件，event.detail = {value}	1.7.0

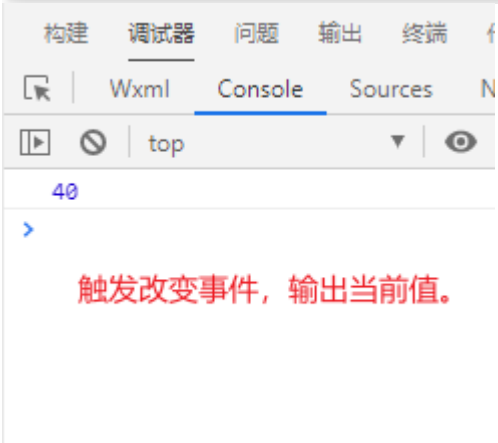
```
<!-- slider组件 -->
<slider min="0" max="100" step="2" value="20" show-value="{{flag}}" block-size="13" block-color="#aaa" bindchange="sdChange"/>
```

```
data数值及事件：
data: {
    flag:true
},
// 自定义滑块数值改变事件
sdChange(e){
  console.log(e.detail.value);
}
```

实现效果：


项目 文件 编辑 工具 转到 选择 视图 界面 设置 帮助 微信开发者工具
kacool-3 - 微信开发者工具 Stable 1.06.2401020
模拟器 编辑器 调试器 可视化 云开发
小程序模式
pages/button/button
编译 预览 真机调试 清缓存
上传 版本管理 详情 消息
iPhone 6/7/8 100% 16
热重载 开
WeChat 18:37 97%
组件基础
20
资源管理器
打开的编辑器
KACOOL-3
components
images
pages
button
checkbox
icon
image
index
navigator
news-wall
progress
radio
scroll-view
slider
slider.js
slider.json
slider.scss
slider.wxml
swiper
switch
text
view
.eslintrc.js
app.js
app.json
app.wxss
project.config.json
project.private.config.json
sitemap.json
slider.wxml
pages > slider > slider.wxml > slider
1 <!--pages/slider/slider.wxml-->
2 <slider min="0" max="100" step="2" value="20" show-value="{{flag}}"
block-size="13" block-color="#aaa" bindchange="sdChange"/>
默认状况下，滑块数值为20，步长为2，最大值为100，最小值为0。
构建 调试器 问题 输出 终端 代码质量
Wxml Console Sources Network Performance Memory AppData Storage
top
Filter
Default levels
大纲
页面路径 pages/slider/slider
行 2，列 127 空格: 2 UTF-8 CRLF WXML





更多属性学习地址：https://developers.weixin.qq.com/miniprogram/dev/component/slider.html

3.5.6、input组件

```
1    输入框组件。
```

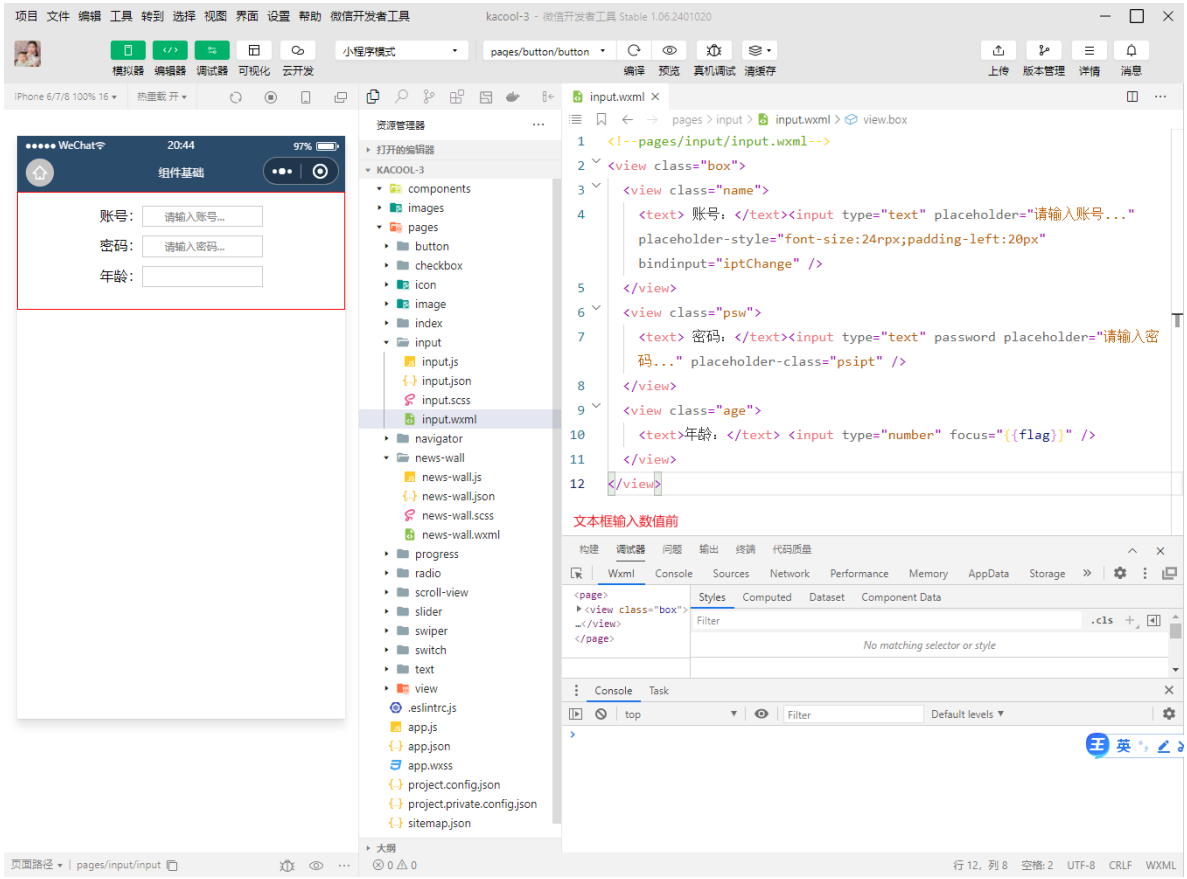
属性	类型	默认值	必填	说明	最低版本
value	string		是	输入框的初始内容	1.0.0
type	string	text	否	input 的类型（text、number、idcard、digit、safe-password、nickname。真机测试情况下，显示的输入键盘类型）	1.0.0
password	boolean	false	否	是否是密码类型	1.0.0
placeholder	string		是	输入框为空时占位符	1.0.0
placeholder-style	string		是	指定 placeholder 的样式	1.0.0
placeholder-class	string	input-placeholder	否	指定 placeholder 的样式类	1.0.0
disabled	boolean	false	否	是否禁用	1.0.0
maxlength	number	140	否	最大输入长度，设置为 -1 的时候不限制最大长度	1.0.0
cursor-spacing	number	0	否	指定光标与键盘的距离，取 input 距离底部的距离和 cursor-spacing 指定的距离的最小值作为光标与键盘的距离	1.0.0
focus	boolean	false	否	获取焦点	1.0.0
cursor	number		是	指定 focus 时的光标位置	1.0.0
bindinput	eventhandle		是	键盘输入时触发，event.detail = {value, cursor, keyCode}，keyCode 为键值，2.1.0 起支持，处理函数可以直接 return 一个字符串，将替换输入框的内容。	1.0.0
bindfocus	eventhandle		是	输入框聚焦时触发，event.detail = { value, height }，height 为键盘高度，在基础库 1.9.90 起支持	1.0.0
bindblur	eventhandle		是	输入框失去焦点时触发，event.detail = { value, encryptedValue, encryptError }	1.0.0
bindconfirm	eventhandle		是	点击完成按钮时触发，event.detail = { value }	1.0.0

```wxml
<!-- input组件 -->
<view>
  账号：<input type="text" placeholder="请输入账号..." placeholder-style="font-size:12px;padding-left:20px" bindinput="iptChange"/>
  密码：<input type="text" password="{{flag}}" placeholder="请输入密码..." placeholder-class="psipt"/>
  年龄：<input type="number" focus="{{flag}}"/>
</view>
```

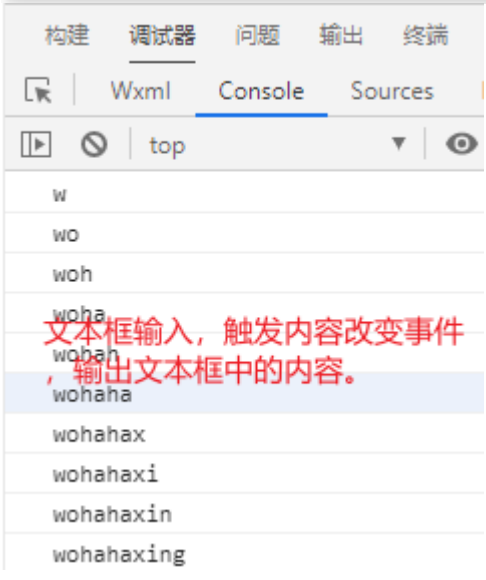
```
样式：
.psipt{
  font-size: 14px;
  text-align: center;
}
```

```
data数据及事件：
data: {
    flag:true
},
// 自定义输入框内容改变事件
iptChange(e){
  console.log(e.detail.value);
}
```

实现效果：







更多属性学习网址：https://developers.weixin.qq.com/miniprogram/dev/component/input.html

3.5.7、textarea组件

1 多行输入框

属性	类型	默认值	必填	说明	最低版本
value	string		否	输入框的内容	1.0.0
placeholder	string		否	输入框为空时占位符	1.0.0
placeholder-style	string		否	指定 placeholder 的样式，目前仅支持color,font-size和font-weight	1.0.0
placeholder-class	string	textarea-placeholder	否	指定 placeholder 的样式类	1.0.0
disabled	boolean	false	否	是否禁用	1.0.0
auto-height	boolean	false	否	是否自动增高，设置auto-height时，style.height不生效	1.0.0
bindinput	eventhandle		否	当键盘输入时，触发 input 事件，event.detail = {value, cursor, keyCode}，keyCode 为键值，目前工具还不支持返回keyCode参数。**bindinput 处理函数的返回值并不会反映到 textarea 上**	1.0.0

更多属性学习网址：https://developers.weixin.qq.com/miniprogram/dev/component/textarea.html

3.5.8、picker组件

1 从底部弹起的滚动选择器。

属性	类型	默认值	必填	说明	最低版本
header-text	string		否	选择器的标题，仅安卓可用	2.11.0
mode	string	selector	否	选择器类型	1.0.0
disabled	boolean	false	否	是否禁用	1.0.0
bindcancel	eventhandle		否	取消选择时触发	1.9.90

根据不同的mode数值，起类型有多种：

合法值	说明
selector	普通选择器
multiSelector	多列选择器
time	时间选择器
date	日期选择器
region	省市区选择器

普通选择器：mode = selector

属性名	类型	默认值	说明	最低版本
range	array/object array	[]	mode 为 selector 或 multiSelector 时，range 有效	
range-key	string		当 range 是一个 Object Array 时，通过 range-key 来指定 Object 中 key 的值作为选择器显示内容	
value	number	0	表示选择了 range 中的第几个（下标从 0 开始）	
bindchange	eventhandle		value 改变时触发 change 事件，event.detail = {value}	

代码：

```
<!-- picker组件 -->
<picker mode="selector" range="{{localArr}}" value="1" bindchange="selChange">
    <view>
     当前位置：<text style="color: red;">{{localArr[index]}}</text>
    </view>
</picker>
```

```
data: {
    localArr:["北京","广州","上海","深圳"],
    index:0
},
// 自定义选择函数
selChange(e){
  this.setData({
    index:e.detail.value
  })

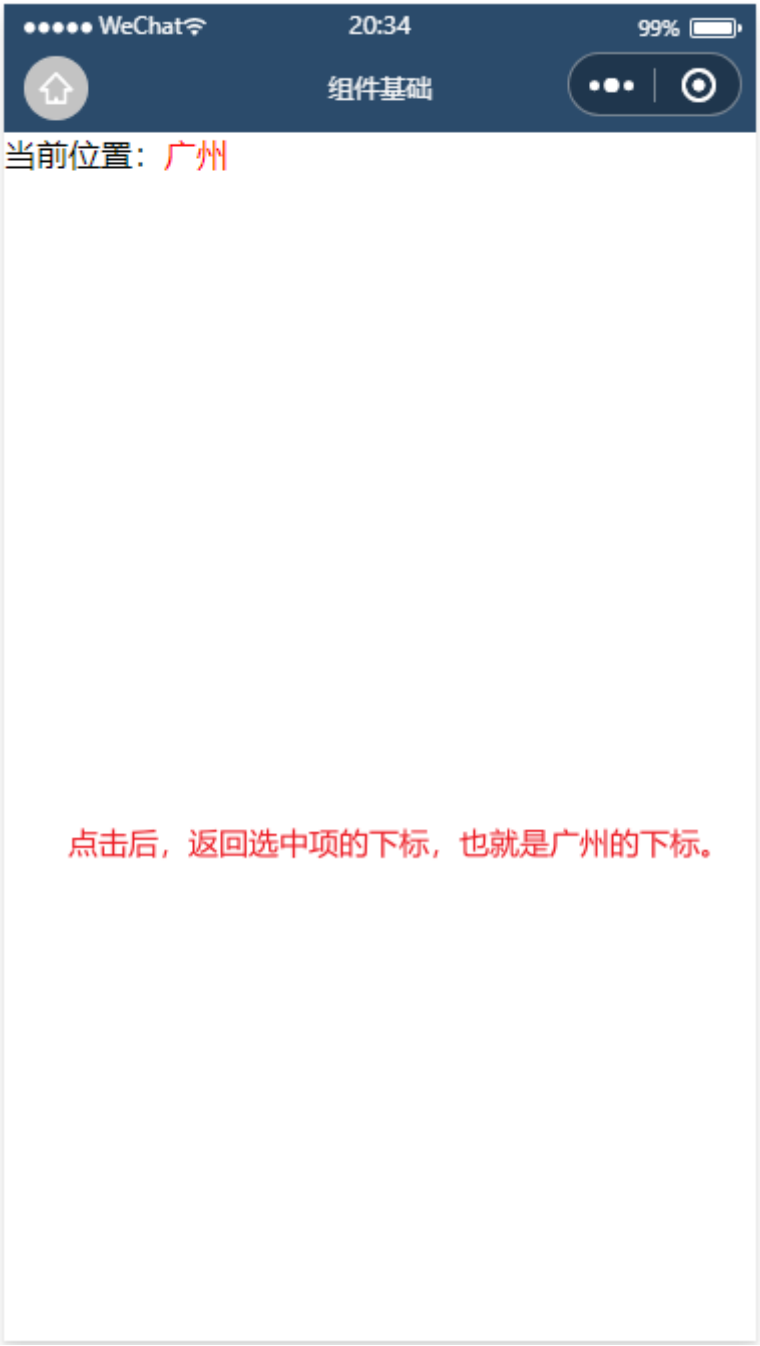
}
```

效果:


WeChat
20:33
99%
组件基础
当前位置：北京
默认区域为：北京


WeChat
20:33
99%
组件基础
当前位置：北京


取消
确定
点击“北京”，弹出picker，选中“广州”
北京
广州
上海
深圳


WeChat
20:34
99%
组件基础
当前位置：广州
点击后，返回选中项的下标，也就是广州的下标。

时间选择器：mode = time

属性名	类型	默认值	说明	最低版本
value	string		表示选中的时间，格式为"hh:mm"	
start	string		表示有效时间范围的开始，字符串格式为"hh:mm"	
end	string		表示有效时间范围的结束，字符串格式为"hh:mm"	
bindchange	eventhandle		value 改变时触发 change 事件，event.detail = {value}	

代码：

```
<!-- picker组件 -->
<picker mode="time" value="hh:mm" start="00:00" end="23:59" bindchange="selChange">
  <view>
    {{value}}
  </view>
</picker>
```

```
data: {
    value:"01:00"
},
// 自定义选择函数
selChange(e){
  this.setData({
    value:e.detail.value
  })

}
```

日期选择器：mode = date

属性名	类型	默认值	说明	最低版本
value	string	当天	表示选中的日期，格式为"YYYY-MM-DD"	
start	string		表示有效日期范围的开始，字符串格式为"YYYY-MM-DD"	

end	string	默	表示有效日期范围的结束，字符串格式为"YYYY-MM-DD"	最低
属性名	类型	认值	说明	版本
bindchange	eventhandle		value 改变时触发 change 事件，event.detail = {value}	

代码：

```
<!-- picker组件 -->
<picker mode="date" value="{{date}}" start="2015-09-01" end="2017-09-01" bindchange="selChange">
  <view>
    当前选择: {{date}}
  </view>
</picker>
```

```
data: {
    date:"2007-06-30"
},
// 自定义选择函数
selChange(e){
  this.setData({
    date:e.detail.value
  })

}
```

省市区选择器：mode = region

属性名	类型	默认值	说明	最低版本
value	array	[]	表示选中的省市区，默认选中每一列的第一个值	
custom-item	string		可为每一列的顶部添加一个自定义的项	1.5.0
level	string	region	选择器层级	2.21.1
bindchange	eventhandle		value 改变时触发 change 事件，event.detail = {value, code, postcode}，其中字段 code 是统计用区划代码，postcode 是邮政编码	

代码：

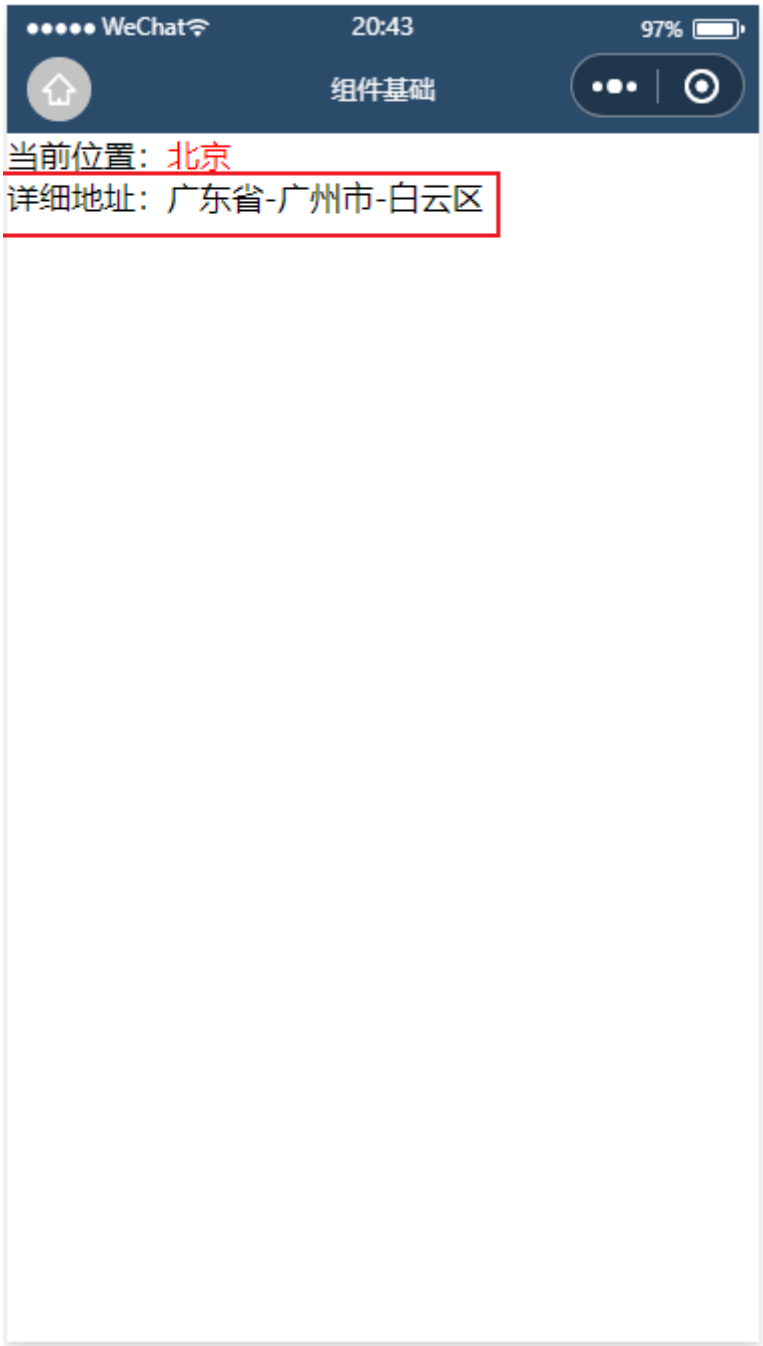
```
<!-- picker组件 -->
<picker mode="region" bindchange="selChange" value="{{region}}" custom-item="{{customItem}}">
    <view class="picker">
      当前选择：{{region[0]}}-{{region[1]}}-{{region[2]}}
    </view>
</picker>
```

```
data: {
    region: ['广东省', '广州市', '海珠区'],
    customItem: '全部'
},
// 自定义选择函数
selChange(e){
  this.setData({
    region: e.detail.value
  })
}
```

效果：


WeChat
20:42
97%
组件基础
当前位置：北京
详细地址：广东省-广州市-海珠区
默认显示：广东 广州 海珠
WeChat
20:43
97%
组件基础
当前位置：北京
详细地址：广东省-广州市-海珠区


取消
确定
湖北省
海珠区
湖南省
全部
天河区
广东省
广州市
白云区
广西壮族自治区
韶关市
黄埔区
海南省
深圳市
番禺区


WeChat
20:43
97%
组件基础
当前位置：北京
详细地址：广东省-广州市-白云区

更多知识学习：https://developers.weixin.qq.com/miniprogram/dev/component/picker.html

3.5.9、form组件

表单。将组件内的用户输入的switch input checkbox slider radio picker 提交。

当点击 form 表单中 form-type 为 submit 的 button 组件时，会将表单组件中的 value 值进行提交，需要在表单组件中加上 name 来作为 key。

属性	类型	默认值	必填	说明	最低版本
report-submit	boolean	false	否	是否返回 formId 用于发送模板消息	1.0.0
report-submit-timeout	number	0	否	等待一段时间（毫秒数）以确认 formId 是否生效。如果未指定这个参数，formId 有很小的概率是无效的（如遇到网络失败的情况）。指定这个参数将可以检测 formId 是否有效，以这个参数的时间作为这项检测的超时时间。如果失败，将返回 requestFormId:fail 开头的 formId	2.6.2
bindsubmit	eventhandle		否	携带 form 中的数据触发 submit 事件，event.detail = {value : {'name': 'value'} , formId: ''}	1.0.0
bindreset	eventhandle		否	表单重置时会触发 reset 事件	1.0.0

代码：

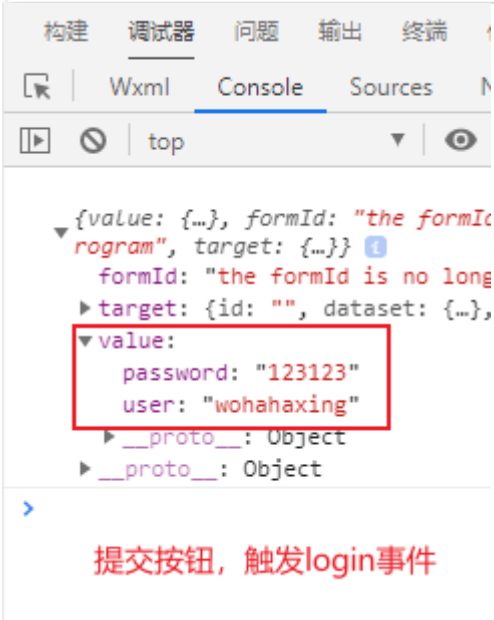
```
<!-- form组件 -->
<form  report-submit="{{flag}}" bindsubmit="login">
   账号：<input type="text" name="user" placeholder="请输入账号..."/>
   密码：<input type="password" name="password" placeholder="请输入密码..."/>

   <button form-type="submit" type="primary" size="mini">提交</button>
   <button form-type="reset" size="mini">重置</button>
</form>
```

```
data: {
    flag:false
},
// 自定义登录函数
login(e){
   console.log(e.detail);
}
```

效果:





更多知识学习：https://developers.weixin.qq.com/miniprogram/dev/component/form.html

3.6节、综合案例-新闻墙

实现效果：


WeChat
18:09
97%
新闻墙
1、昨天的信息太多。
2、今天的事情有点急。
3、明天是否会如此?
请输入新闻
提交
当前存在 3 条新闻

新闻墙默认状态


WeChat
18:11
97%
新闻墙
1、昨天的信息太多。
2、今天的事情有点急。
3、明天是否会如此?
4、晚饭吃什么?
请输入新闻
提交
当前存在 4 条新闻

添加一条新闻



删除新闻



新闻全部删除完毕

```
news-wall.json配置标题

{
  "usingComponents": {},
  "navigationBarTitleText": "新闻墙"
}
```

新闻墙涉及的代码：

```
<!--pages/news-wall/news-wall.wxml-->
<view class="box">
    <block wx:if="{{newsArr.length}}">
        <view class="new-list" wx:for="{{newsArr}}" wx:key="id">
            <view class="item"> <text>{{index + 1 + "、"}}{{item.title}}</text>
                <icon type="clear" data-idx="{{index}}" size="20" color="#666" bind:tap="delNews" />
            </view>
        </view>
    </block>
    <view wx:else class="no">暂无信息，等待添加</view>
    <view class="add"><input type="text" placeholder="请输入新闻" placeholder-style="font-size:24rpx;padding-left:10rpx" bindinput="addTitle" value="{{newTitle}}" focus /> <button type="primary" size="mini" bind:tap="addNews">提交</button></view>
    <view class="cut">当前存在<text style="color:deeppink;margin:0rpx 10rpx;">{{newsArr.length}}</text> 条新闻</view>
</view>
```

```
/**news-wall.wxss**/
.box{
  width: 100%;
  padding: 20rpx 30rpx;
  box-sizing: border-box;
  .new-list{
```

```
    margin: 20rpx;
    padding-bottom: 10rpx;
    border-bottom: 2rpx dotted #ccc;
    .item{
      display: flex;
      flex-direction: row;
      justify-content: space-between;
    }
  }
  .add{
    border: 2rpx solid #eee;
    display: flex;
    padding: 30rpx 0rpx;
    justify-content: space-between;
    input{
      flex-grow: 1;
      border: 1rpx dotted #ddd;
      font-size: 36rpx;
      margin-left: 10rpx;
      height: 62rpx;
      padding-left: 10rpx;
    }
    button{
      margin: 0rpx 20rpx;
    }
  }
  .no{
    color: #aaa;
    text-align: center;
    padding: 20rpx;
  }
  .cut{
    text-align: center;
    color: #ccc;
    font-size: 24rpx;
    margin-top: 20rpx;
  }
}
```

```
//news-wall.js
Page({

  /**
   * 页面的初始数据
   */
  data: {
      newTitle:'',
      newsArr:[{
        id:1,
        title:"昨天的信息太多。"
      },
      {
        id:2,
        title:"今天的事情有点急。"
      },
      {
```

```
        id:3,
        title:"明天是否会如此？"
      }]
  },

  //自定义文本框输入方法
  addTitle:function(e){
     this.setData({
        newTitle:e.detail.value
     })
  },

  //自定义添加新闻方法
  addNews:function(){
    let id = Date.now()
    let oldArr = this.data.newsArr
    oldArr.push({id,"title":this.data.newTitle})
    this.setData({
      newsArr:oldArr,
      newTitle:''
    })

  },

  //自定义删除新闻方法
  delNews:function(e) {
    let oldArr = this.data.newsArr
    let index = e.currentTarget.dataset.idx
    oldArr.splice(index,1)
    this.setData({
      newsArr:oldArr
    })
  },
  /**
   * 生命周期函数--监听页面加载
   */
  onLoad: function (options) {

  },

  /**
   * 生命周期函数--监听页面初次渲染完成
   */
  onReady: function () {

  },

  /**
   * 生命周期函数--监听页面显示
   */
  onShow: function () {

  },

  /**
   * 生命周期函数--监听页面隐藏
   */
```

```
75      onHide: function () {
76
77      },
78
79      /**
80       * 生命周期函数--监听页面卸载
81       */
82      onUnload: function () {
83
84      },
85
86      /**
87       * 页面相关事件处理函数--监听用户下拉动作
88       */
89      onPullDownRefresh: function () {
90
91      },
92
93      /**
94       * 页面上拉触底事件的处理函数
95       */
96      onReachBottom: function () {
97
98      },
99
100     /**
101      * 用户点击右上角分享
102      */
103     onShareAppMessage: function () {
104
105     }
106 })
107
```

总结:

在该综合案例中，巩固了页面组件布局，scss样式设计，Data数据访问，自定义方法编写等一系列综合知识点。

第4章、小程序API

4.1节、交互API

4.1.1、wx.showToast

```
1   消息提示框
```

属性	类型	默认值	必填	说明	最低版本
title	string		是	提示的内容	
icon	string	success	否	图标	
image	string		否	自定义图标的本地路径，image 的优先级高于 icon	1.1.0
duration	number	1500	否	提示的延迟时间	
mask	boolean	false	否	是否显示透明蒙层，防止触摸穿透	
success	function		否	接口调用成功的回调函数	
fail	function		否	接口调用失败的回调函数	
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）	

icon属性：

合法值	说明	最低版本
success	显示成功图标，此时 title 文本最多显示 7 个汉字长度	
error	显示失败图标，此时 title 文本最多显示 7 个汉字长度	2.14.1
loading	显示加载图标，此时 title 文本最多显示 7 个汉字长度	

```
<!-- API交互 -->
<button bindtap="showTost">点击显示消失框</button>
```

```
// 自定义消失函数
showTost(){
  wx.showToast({
    title: '消失框',
    icon: 'success',
    duration: 1000
  })
}
```

效果：



4.1.2、wx.showModal

```
显示模态框
```

属性	类型	默认值	必填	说明	最低版本
title	string		否	提示的标题	
content	string		否	提示的内容	
showCancel	boolean	true	否	是否显示取消按钮	
cancelText	string	取消	否	取消按钮的文字，最多 4 个字符	
cancelColor	string	#000000	否	取消按钮的文字颜色，必须是 16 进制格式的颜色字符串	
confirmText	string	确定	否	确认按钮的文字，最多 4 个字符	
confirmColor	string	#576B95	否	确认按钮的文字颜色，必须是 16 进制格式的颜色字符串	
editable	boolean	false	否	是否显示输入框	2.17.1
placeholderText	string		否	显示输入框时的提示文本	2.17.1
success	function		否	接口调用成功的回调函数	
fail	function		否	接口调用失败的回调函数	

属性	类型	默认值	必填	说明	最低版本
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）	

代码:

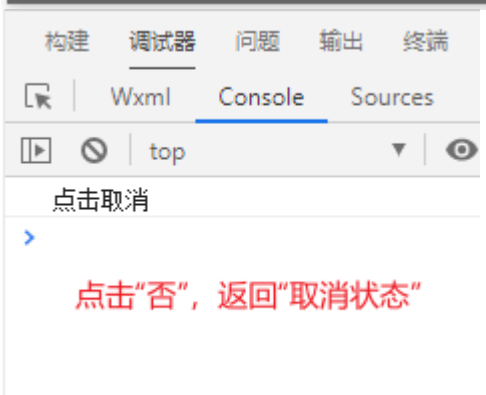
```
<!-- API交互 -->
<button bindtap="showMoTai">点击显示模态框</button>
```

```
//自定义函数
showMoTai(){
  wx.showModal({
    title: '标题',
    content: '这是内容...',
    cancelText:"否",
    confirmText:"是",
    confirmColor:"purple",
    complete: (res) => {
      if (res.cancel) {
         console.log("点击取消");
      }

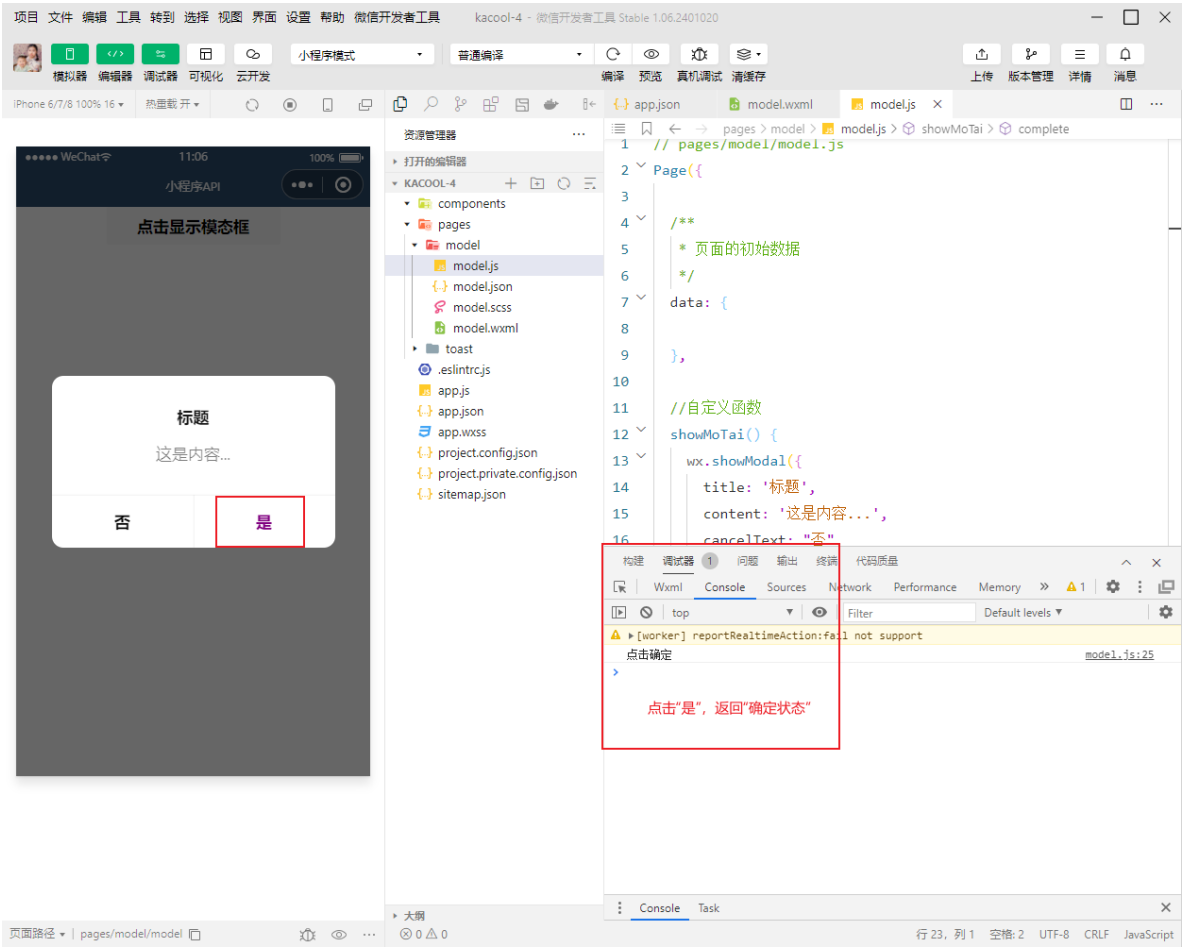
      if (res.confirm) {
         console.log("点击确定");
      }
    }
  })
}
```

效果:


WeChat
11:05
100%
小程序API
点击显示模态框
点击按钮，显示模态框
标题
这是内容...
否
是


构建
调试器
问题
输出
终端
Wxml
Console
Sources
top
点击取消

点击“否”，返回“取消状态”



4.1.3、wx.showLoading

显示加载框

属性	类型	默认值	必填	说明
title	string		是	提示的内容
mask	boolean	false	否	是否显示透明蒙层，防止触摸穿透
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

4.1.4、wx.hideLoadding

隐藏加载框

属性	类型	默认值	必填	说明	最低版本
noConflict	boolean	false	否	目前 toast 和 loading 相关接口可以相互混用，此参数可用于取消混用特性	2.22.1
success	function		否	接口调用成功的回调函数	
fail	function		否	接口调用失败的回调函数	
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）	

代码：

```
<!-- API交互 -->
<button bindtap="showLoad"  type="primary">点击显示Loading</button>
<button bindtap="hideLoad">点击隐藏Loading</button>
```

```
//自定义函数
showLoad(){
  wx.showLoading({
    title: '加载中...',

  })
}
,
hideLoad(){
  wx.hideLoading()
}
```

效果：


●●●●● WeChat
11:12
100%
小程序API
点击显示Loading
点击隐藏Loading
加载中...
点击显示按钮，弹出加载框
并不自动消失。


●●●●● WeChat
11:13
100%
小程序API
点击显示Loading
点击隐藏Loading



4.2节、导航栏API

4.2.1、wx.showTabBar

4.2.2、wx.hideTabBar

属性	类型	默认值	必填	说明
animation	boolean	false	否	是否需要动画效果
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

代码：

```
<!-- 导航栏API -->
<button bindtap="hideTab">点击隐藏TabBar</button>

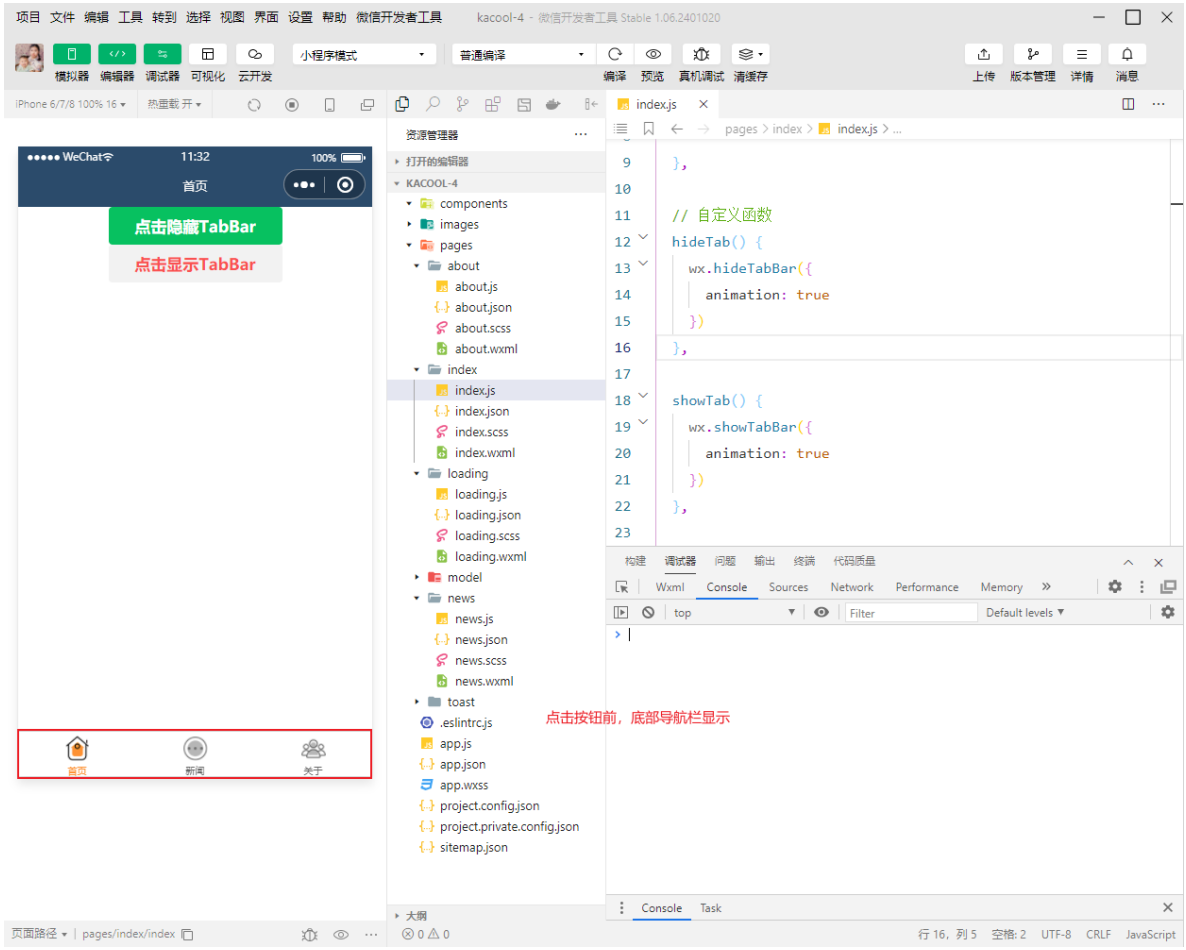
<button bindtap="showTab">点击显示TabBar</button>

```

```
// 自定义函数
hideTab(){
  wx.hideTabBar({
    animation:true
  })
},

showTab(){
  wx.showTabBar({
    animation:true
  })
}
```

效果:




WeChat
11:33
100%
首页
点击隐藏TabBar
点击显示TabBar
点击隐藏按钮，导航栏消失


WeChat
11:33
100%
首页
点击隐藏TabBar
点击显示TabBar



4.3节、下拉刷新API

4.3.1、wx.startPullDownRefresh

4.3.2、wx.stopPullDownRefresh

对于下拉刷新，一般是手动下滑来实现，首先需要开启允许下拉刷新：

`"enablePullDownRefresh": true`

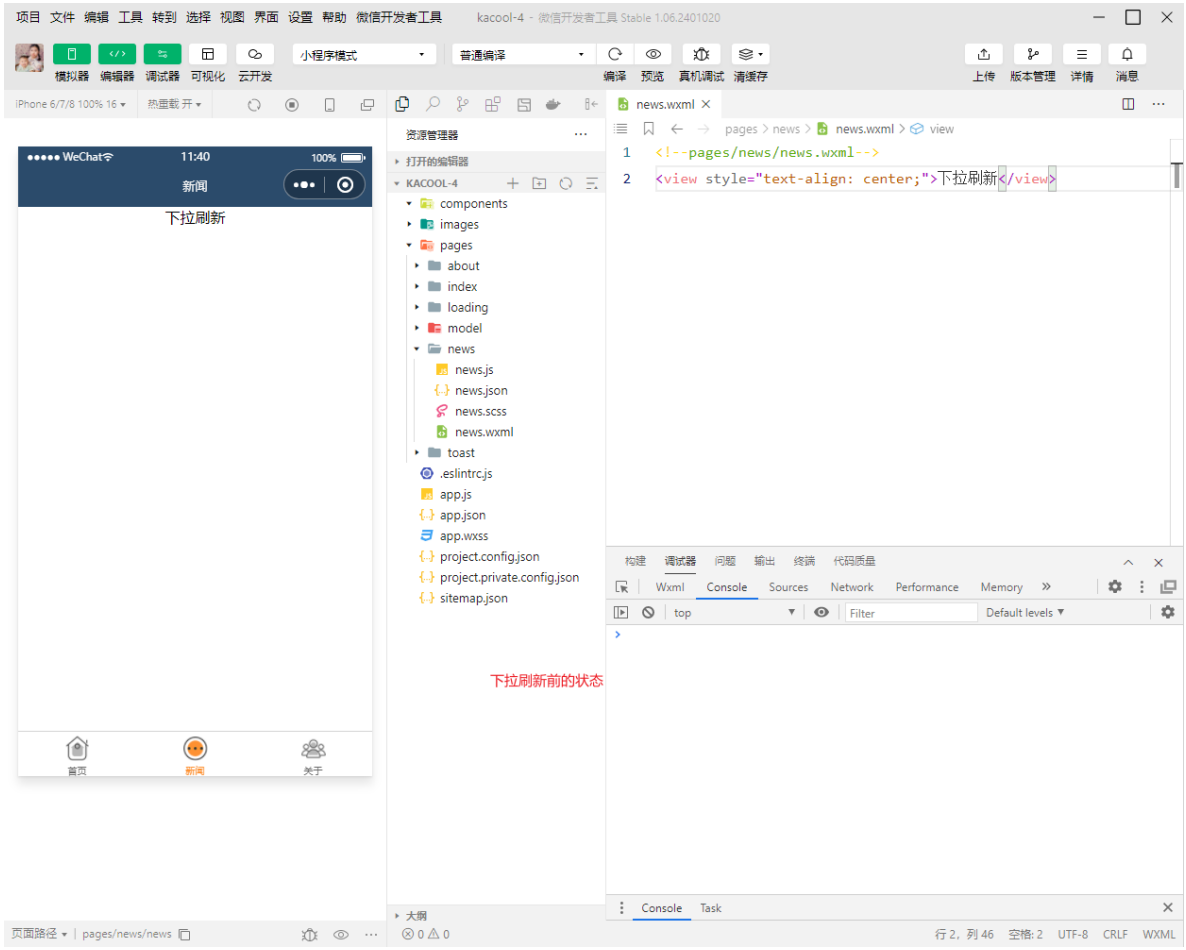
如果在app.json中开启，表示全局开启下拉刷新，如果在页面.json文件中开启，则仅表示在该页面开启。

但是在开启之后，还需要手动关闭下拉刷新。

```
news.json配置

{
  "usingComponents": {},
  "navigationBarTitleText": "新闻",
  "enablePullDownRefresh": true,
  "backgroundColor": "#fe9232",
  "backgroundTextStyle":"dark"
}
```

```
news.js
/**
* 页面相关事件处理函数--监听用户下拉动作
*/
onPullDownRefresh: function () {
    console.log("开始了");
    setTimeout(() => {
        wx.stopPullDownRefresh()
    }, 2000);
}
```


项目 文件 编辑 工具 转到 选择 视图 界面 设置 帮助 微信开发者工具
kacool-4 - 微信开发者工具 Stable 1.06.2401020
模拟器 编辑器 调试器 可视化 云开发
小程序模式
普通编译
编译 预览 真机调试 清缓存
上传 版本管理 详情 消息
iPhone 6/7/8 100% 16
热重载 开
WeChat 11:40 100%
新闻
下拉刷新
首页 新闻 关于
资源管理器
打开的编辑器
KACOOL-4
components
images
pages
about
index
loading
model
news
news.js
news.json
news.scss
news.wxml
toast
.eslintrc.js
app.js
app.json
app.wxss
project.config.json
project.private.config.json
sitemap.json
news.wxml
pages > news > news.wxml > view
1 <!--pages/news/news.wxml-->
2 <view style="text-align: center;">下拉刷新</view>
构建 调试器 问题 输出 终端 代码质量
Wxml Console Sources Network Performance Memory
top
Filter
Default levels
下拉刷新前的状态
Console Task
大纲
页面路径 pages/news/news
行 2，列 46 空格: 2 UTF-8 CRLF WXML



说明：

对于下拉刷新，一般用于初始化页面数据，尤其是加载了很多数据后，进行下拉刷新就会把容器中的数据恢复到首次页面打开时候的状态。

更多知识学习：https://developers.weixin.qq.com/miniprogram/dev/api/ui/pull-down-refresh/wx.startPullDownRefresh.html

4.4节、上拉加载API

上拉加载，也就是常见的触底加载，一般用于请求出更多的数据。它不需要开启，只要在触底函数中，调用即可。

但是一般会要求传递给后端部分的数据，比如当前第几次请求，每次请求的数据条数等信息。

```
/**
* 页面上拉触底事件的处理函数
*/
onReachBottom: function () {
    console.log("触底了");
}
```

说明：

对于下拉加载或者触底加载，一般为了防止多次快速请求，我们需要进行截流处理，也就是在数据没有加载完毕前，不能开启下一次的下拉加载动作。具体过程，详见综合案例。

4.5节、网络请求API

4.5.1、wx.request

属性	类型	默认值	必填	说明	最低版本
url	string		是	开发者服务器接口地址	
data	string/object/ArrayBuffer		否	请求的参数	
header	Object		否	设置请求的 header，header 中不能设置 Referer。 `content-type` 默认为 `application/json`	
timeout	number		否	超时时间，单位为毫秒。默认值为 60000	2.10.0
method	string	GET	否	HTTP 请求方法	
dataType	string	json	否	返回的数据格式	
responseType	string	text	否	响应的数据类型	1.7.0
success	function		否	接口调用成功的回调函数	
fail	function		否	接口调用失败的回调函数	
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）	

method的属性值：

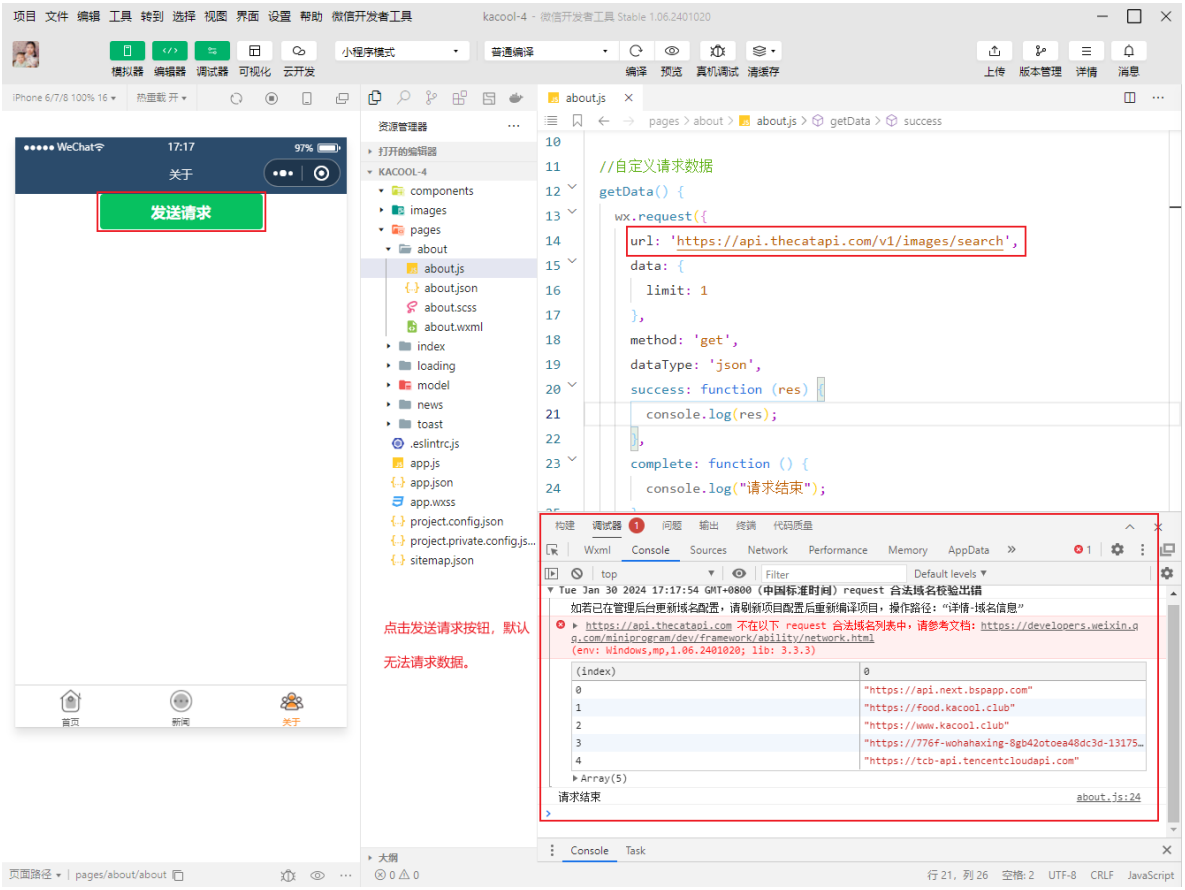
合法值	说明
OPTIONS	HTTP 请求 OPTIONS
GET	HTTP 请求 GET
HEAD	HTTP 请求 HEAD
POST	HTTP 请求 POST
PUT	HTTP 请求 PUT
DELETE	HTTP 请求 DELETE
TRACE	HTTP 请求 TRACE
CONNECT	HTTP 请求 CONNECT

代码:

```
<!-- 网络请求API -->
<button bindtap="getData" type="primary">发送请求</button>
```

```
//自定义请求数据
getData(){
    wx.request({
        url: 'https://api.thecatapi.com/v1/images/search',
        data:{
            limit:1
        },
        method:'get',
        dataType:'json',
        success:function(res){
            console.log(res);
        },
        complete:function(){
            console.log("请求结束");
        }
    })
}
```

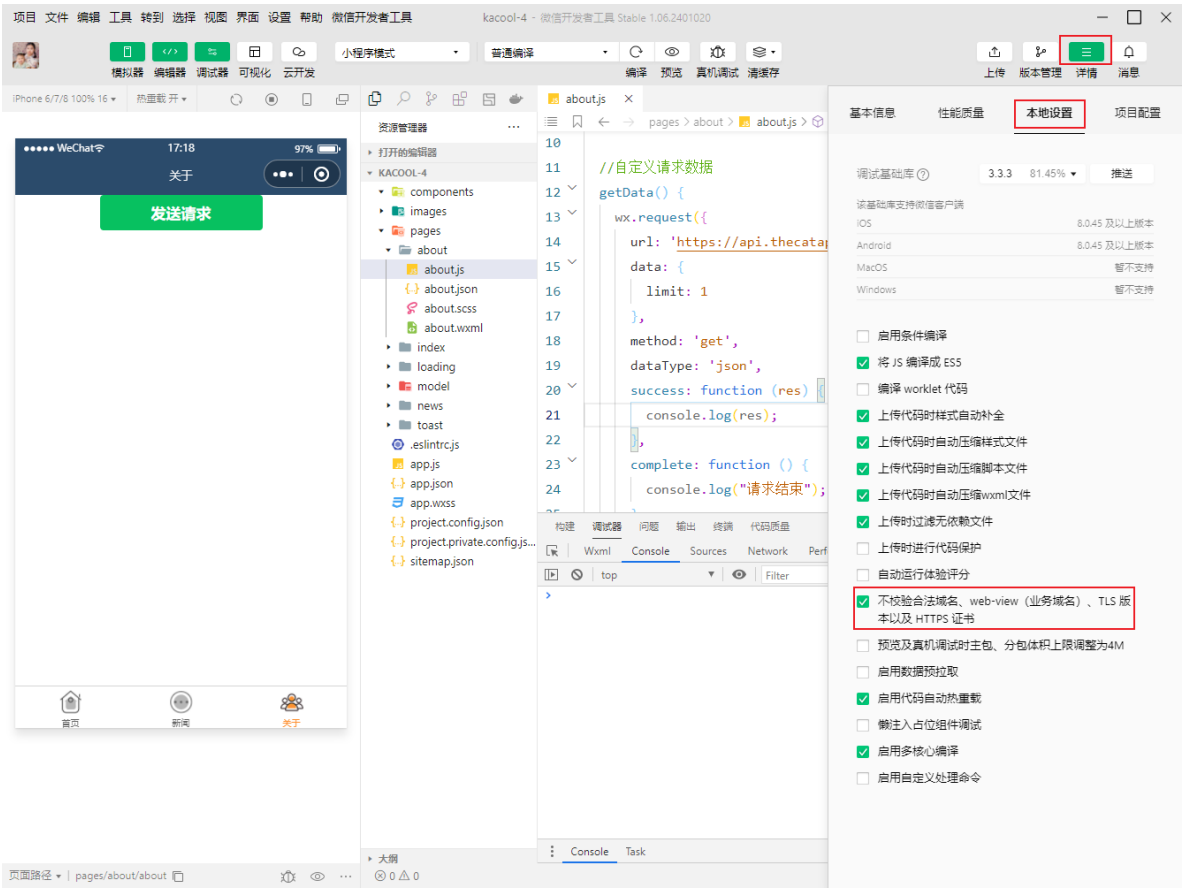
效果:



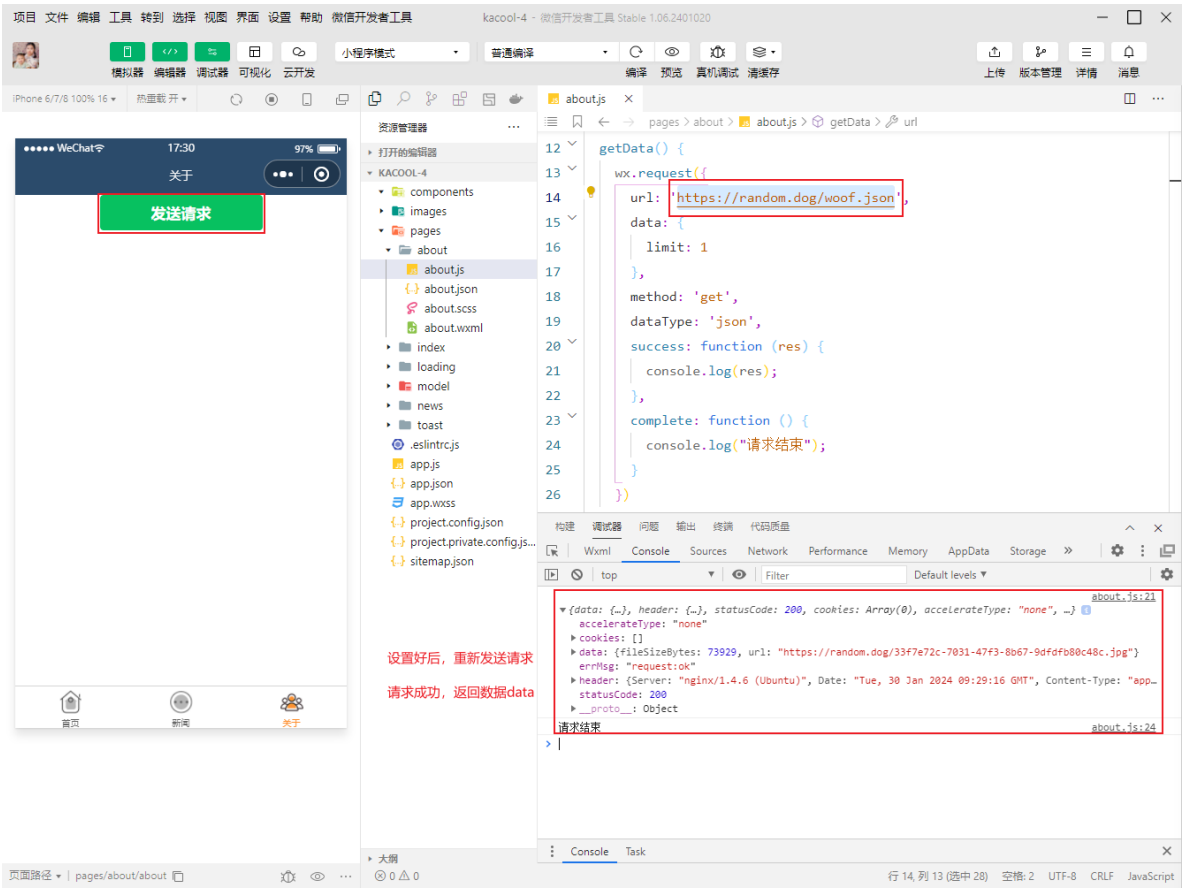
解决方法：

1、取消域名安全校验

2、配置合法域名



再次发送请求，数据已成功获取，至于方法2：配置合法域名，后续讲解



更多知识学习：https://developers.weixin.qq.com/miniprogram/dev/api/network/request/wx.request.html

4.6节、媒体API

4.6.1、wx.chooseImage

1 选择图片或者拍照API

属性	类型	默认值	必填	说明
count	number	9	否	最多可以选择的图片张数
sizeType	Array.	['original', 'compressed']	否	所选的图片的尺寸(原图、压缩图)
sourceType	Array.	['album', 'camera']	否	选择图片的来源(相册、相机)
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

代码：

```
<!-- 媒体API -->
<button bindtap="getImg">选择相片</button>

getImg(){
  wx.chooseImage({
    count: 1,
    sizeType: ['original', 'compressed'],
    sourceType: ['album', 'camera'],
    success (res) {
      console.log( res.tempFilePaths)
    }
  })
}
```

4.6.2、wx.chooseMedia

```
多媒体API，已完全取代了wx.chooseImage。
```

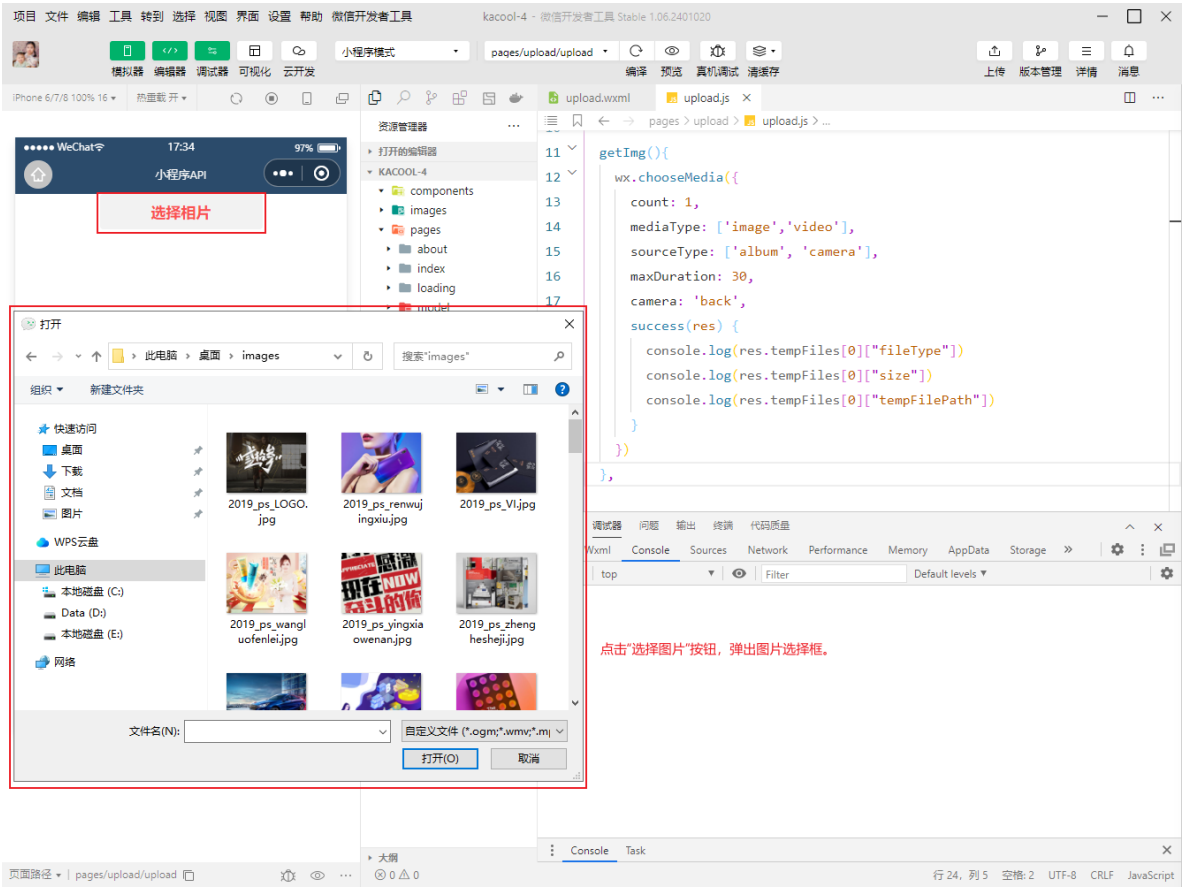
属性	类型	默认值	必填	说明
count	number	9	否	最多可以选择的文件个数，基础库2.25.0前，最多可支持9个文件，2.25.0及以后最多可支持20个文件
mediaType	Array.	['image', 'video']	否	文件类型(相片、视频)
sourceType	Array.	['album', 'camera']	否	图片和视频选择的来源(相册、相机)
maxDuration	number	10	否	拍摄视频最长拍摄时间，单位秒。时间范围为 3s 至 60s 之间。不限制相册。

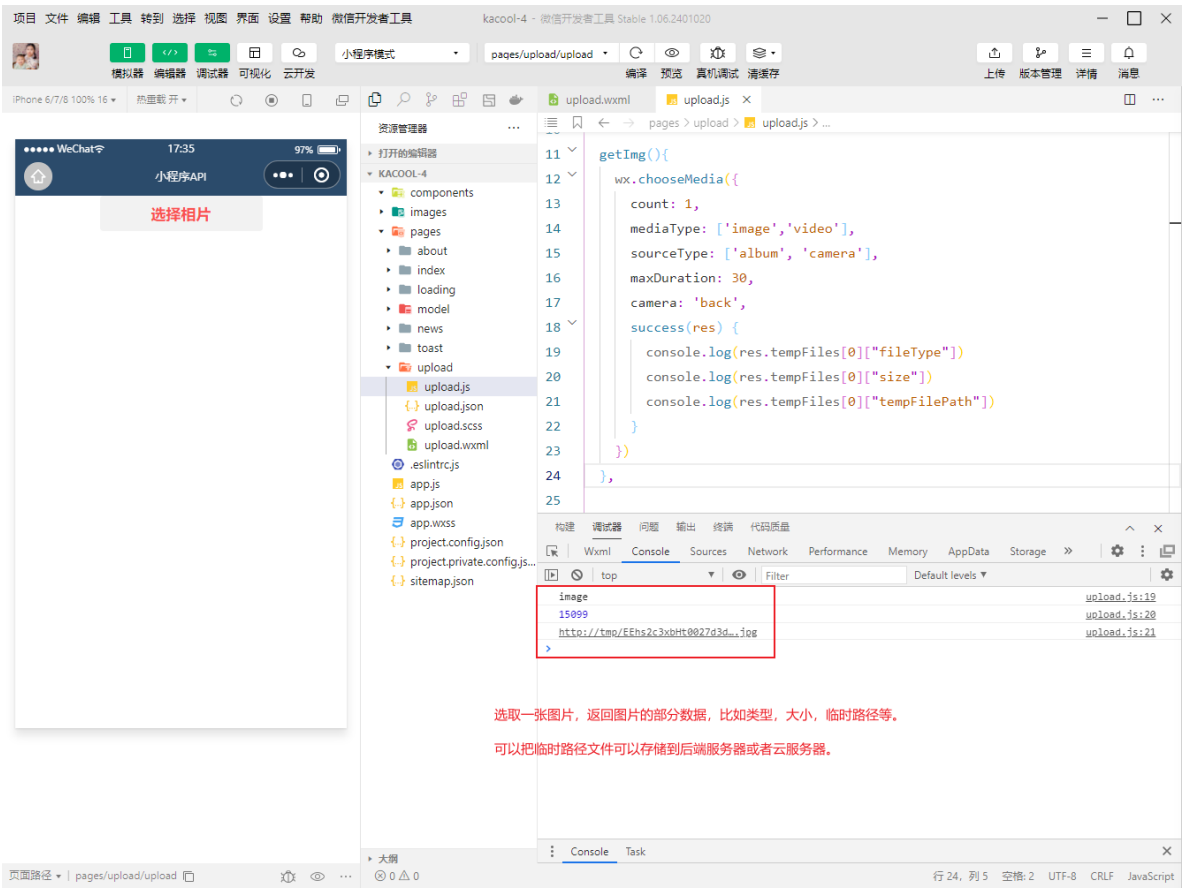
属性	类型	默认值	必填	说明
sizeType	Array.	['original', 'compressed']	否	是否压缩所选文件（原图、压缩图）
camera	string	'back'	否	仅在 sourceType 为 camera 时生效，使用前置或后置摄像头
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

代码：

```
//自定义选择图片函数
getImg(){
  wx.chooseMedia({
    count: 1,
    mediaType: ['image','video'],
    sourceType: ['album', 'camera'],
    maxDuration: 30,
    camera: 'back',
    success(res) {
      console.log(res.tempFiles[0]["fileType"])
      console.log(res.tempFiles[0]["size"])
      console.log(res.tempFiles[0]["tempFilePath"])
    }
  })
}
```

效果：





说明：

在这里，我们就可以通过编写部分代码，将选择的图片或者视频资源上传到云服务器或者后端服务器。

更多知识学习：https://developers.weixin.qq.com/miniprogram/dev/api/media/video/wx.chooseMedia.html

4.7节、综合案例-随机图

实现效果：


WeChat
19:12
97%
随机猫咪
动态加载猫咪，显示加载框，并开启下拉刷新，下拉触底
默认首次获取10张图
构建 2
调试器
问题
输出
终端
代码质量
Wxml
Console
Sources
Network
Performance
Memory
AppData
Pages
pages/rand...
Tree
Editing...
object {2}
catArr [10]
0 {4}
1 {4}
2 {4}
3 {4}
4 {4}
5 {4}
6 {4}
7 {4}
8 {4}
9 {4}
数组中包含
10个对象。
__webviewId__ : 122

下拉刷新：



动态加载猫咪，显示加载框，并开启下拉刷新，下拉触底

数据请求中...

下拉刷新，加载新的10张图

并显示加载框。当数据加载

完毕，加载框消失。

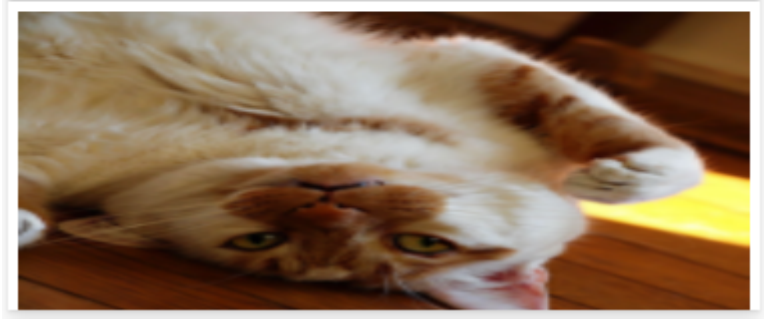


动态加载猫咪，显示加载框，并开启下拉刷新，下拉触底









上拉触底：



上拉触底，再次发送请求

构建 2 调试器 4 问题 输出 终端 代码质量

Wxml Console Sources Network Performance Memory

Pages

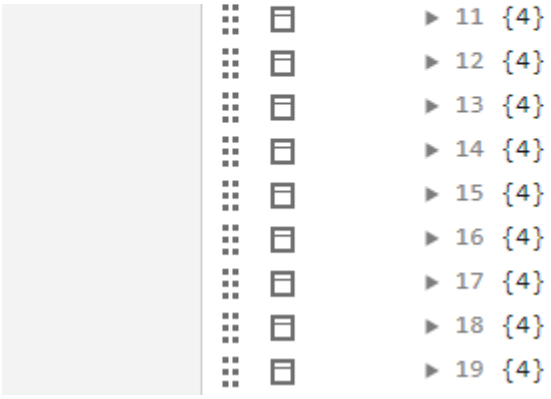
pages/rand...

Tree

```
▼ object {2}
  ▼ catArr [20]
      ▶ 0 {4}
      ▶ 1 {4}
      ▶ 2 {4}
      ▶ 3 {4}
      ▶ 4 {4}
      ▶ 5 {4}
      ▶ 6 {4}
      ▶ 7 {4}
      ▶ 8 {4}
      ▶ 9 {4}
      ▶ 10 {4}
```

触底后，数据量增加10条

每触底一次，增加10条。



案例相关代码：

```
randomCat.json

{
  "usingComponents": {},
  "navigationBarTitleText": "随机猫咪",
  "enablePullDownRefresh": true,
  "backgroundTextStyle":"dark"
}
```

```
<!--pages/randomCat/randomCat.wxml-->

<view class="catlist">
  <view class="title">动态加载猫咪，显示加载框，并开启下拉刷新，下拉触底</view>
  <view class="item" wx:for="{{catArr}}" wx:key="index">
      <image src="{{item.url}}" mode=""/>
  </view>
</view>
```

```
/* pages/randomCat/randomCat.wxss */

.catlist{
  box-sizing: border-box;
}
.catlist .title{
  font-size: 28rpx;
  text-align: center;
  color: palevioletred;
  padding: 20rpx 0rpx;
}
.catlist .item{
  width: 100%;
  box-sizing: border-box;
  height: 400rpx;
  margin-bottom: 10rpx;
  padding: 10rpx;
  box-shadow: 0rpx 0rpx 6rpx #888;
}
.catlist image{
  width: 100%;
```

```
  height: 100%;
}
```

```
// pages/randomCat/randomCat.js

Page({

  /**
   * 页面的初始数据
   */
  data: {
    catArr:[],
    flag:true
  },

  /**
   * 生命周期函数--监听页面加载
   */
  onLoad(options) {
    this.getCat()
  },

  getCat(){
    //如果标识为真，表示正在加载，不可进行下一次请求。
    if(!this.data.flag){
      return false
    }

    //数据请求中，将标识设置为假
    this.setData({
       flag:false
    })

    wx.showLoading({
      title: '数据请求中...',
      mask:true
    })
    wx.request({
      url: 'https://api.thecatapi.com/v1/images/search',
      data:{
        limit:2
      },
      method:'GET',
      dataType:'json',
      success:res=>{
        this.setData({
          catArr:this.data.catArr.concat(res.data)
        })
      },
      complete:()=>{
        //不管请求是否成功，完毕后将标识设置为真
        this.setData({
          flag:true
        })
        setTimeout(() => {
```

```
            wx.hideLoading()
            wx.stopPullDownRefresh()
          }, 1000);
        }
    })
    },

    /**
     * 生命周期函数--监听页面初次渲染完成
     */
    onReady() {

    },

    /**
     * 生命周期函数--监听页面显示
     */
    onShow() {

    },

    /**
     * 生命周期函数--监听页面隐藏
     */
    onHide() {

    },

    /**
     * 生命周期函数--监听页面卸载
     */
    onUnload() {

    },

    /**
     * 页面相关事件处理函数--监听用户下拉动作
     */
    onPullDownRefresh() {
      this.setData({
        catArr:[]
      })
      this.getCat()
    },

    /**
     * 页面上拉触底事件的处理函数
     */
    onReachBottom() {
      this.getCat()
    },

    /**
     * 用户点击右上角分享
     */
    onShareAppMessage() {
```

```
109
110     }
111   })
```

第5章、自定义组件

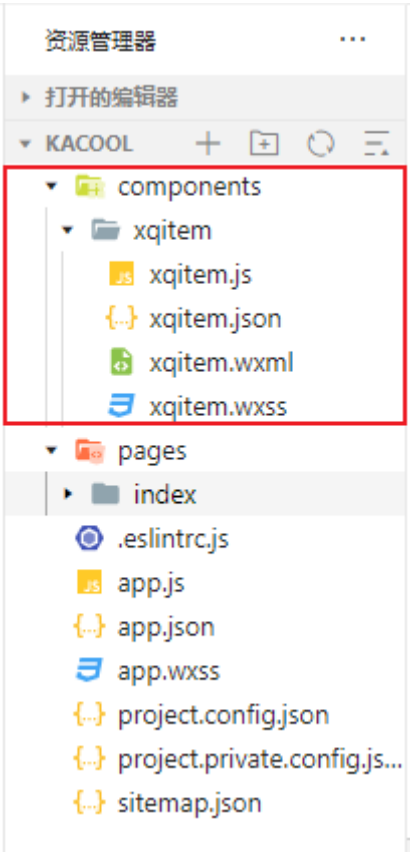
5.1节、组件概述

在小程序中，既有我们在创建页面时，所使用的小程序内置基本组件，也有用户自定义组件。自定义组件称为：components，它的本质跟普通页面没有大的区别，一般用于复现数据，比如商品列表等。

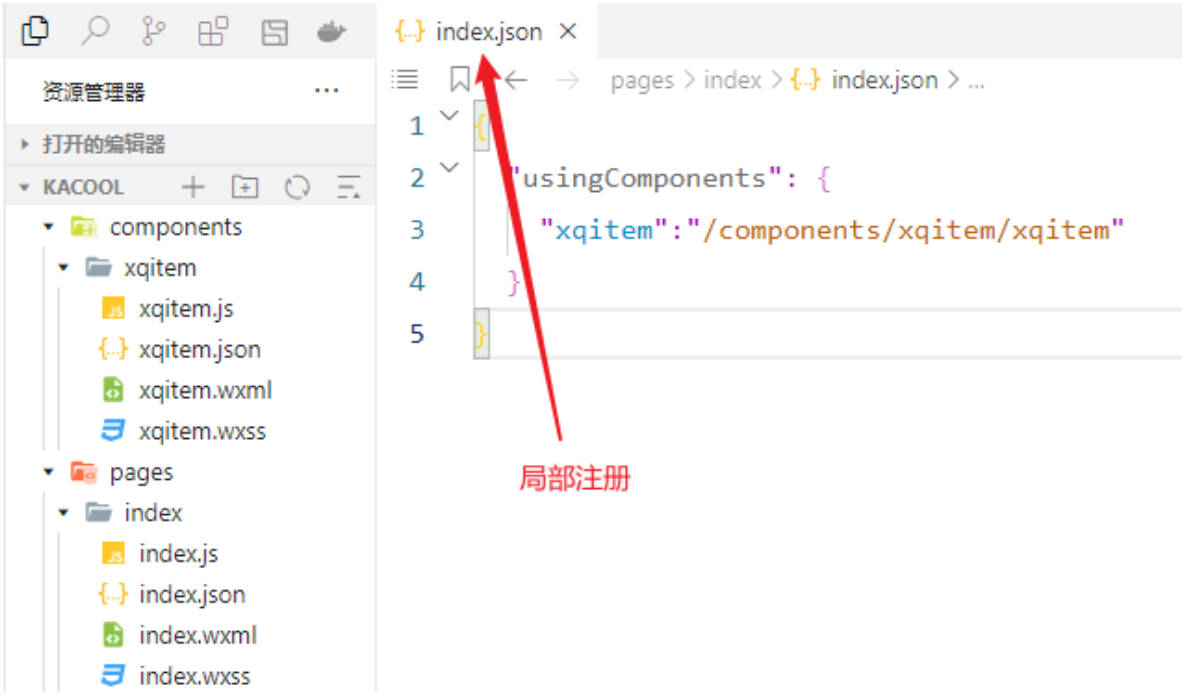
要想正常使用组件，需要按照以下步骤进行：

1. 小程序根目录创建 "components" 目录
2. 在该目录下，创建一个组件
3. 注册创建好的组件(全局或者局部注册)
4. 使用注册的组件

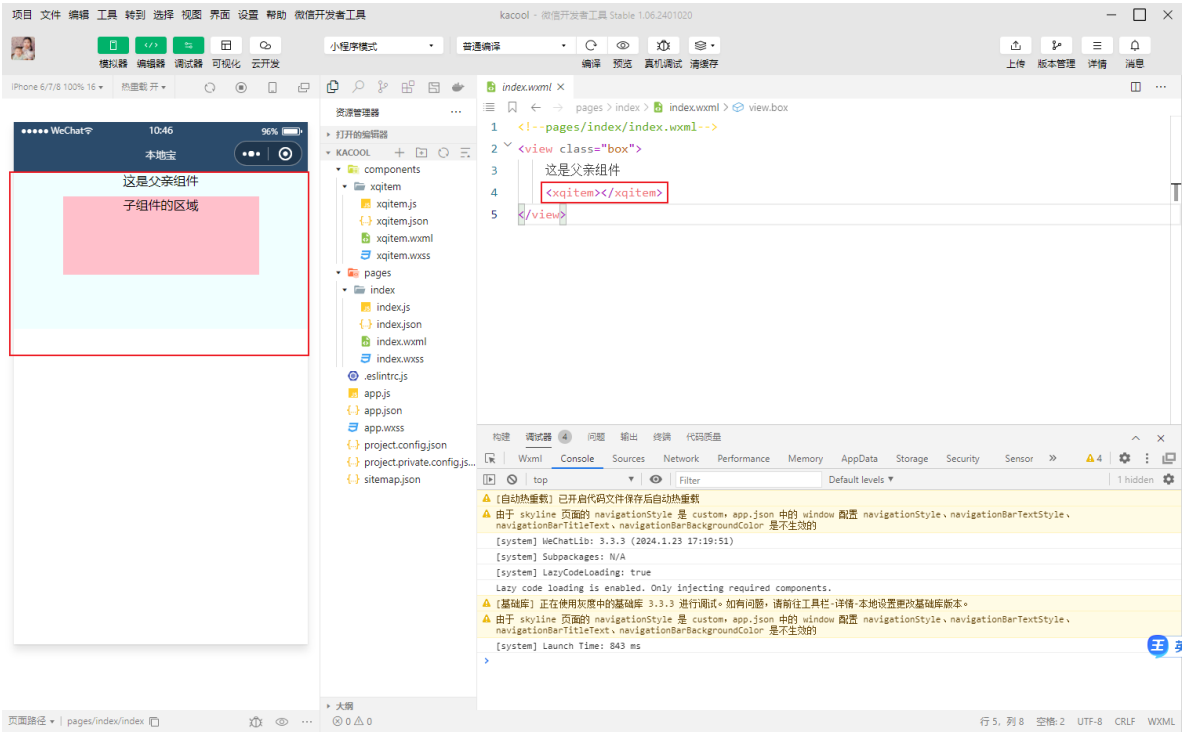
```
1  根目录创建--components目录
2  创建一个组件
```




```
1  注册组件
```



1 使用组件



5.2节、组件传值

对于组件，既然是为了多次重复使用，因此肯定是需要向子组件传递数据的，我们既可以 向子组件传递数据 （主流），也可以是子组件向父组件传回数值。

4.2.1、父传子

父传子，就是父亲组件向子组件传递数据，其传递方式就是通过 属性 传递，子组件通过在 properties 中进行数值接收。

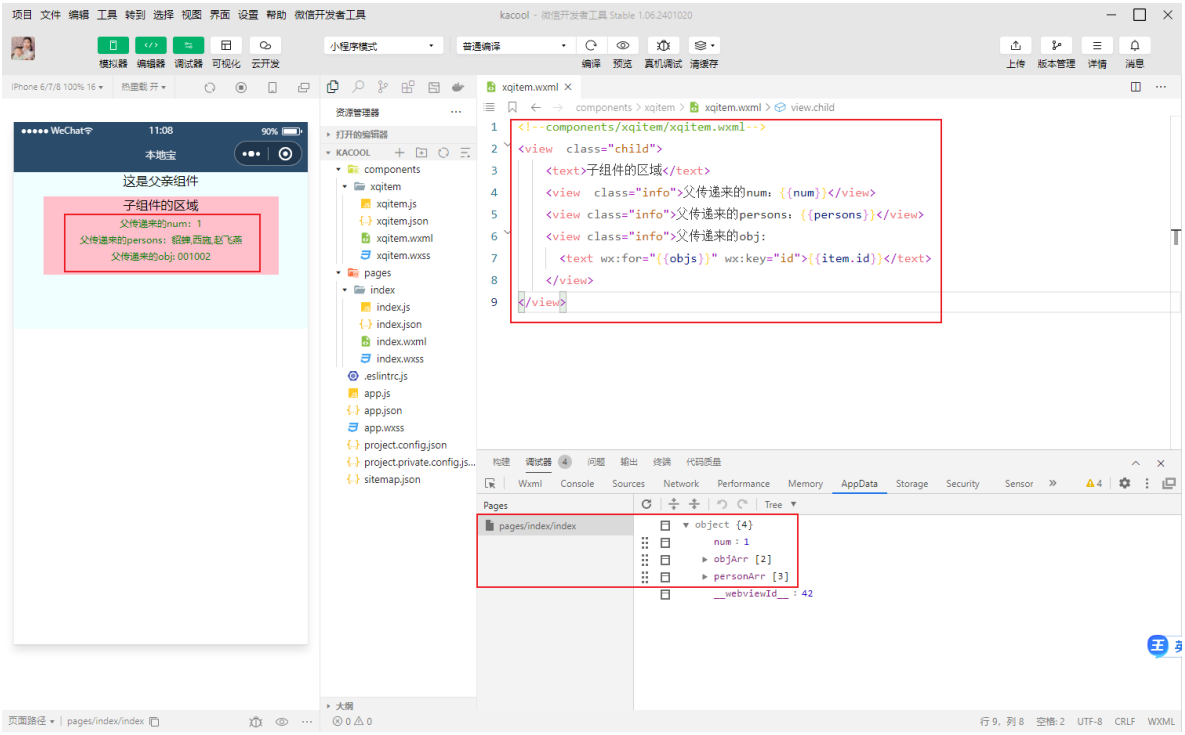
```
<!--pages/index/index.wxml-->
```

```
<view class="box">
    这是父亲组件
    <xqitem num="{{num}}" persons="{{personArr}}" objs="
{{objArr}}"></xqitem>
</view>

/**
* 页面的初始数据
*/
data: {
    num:1,
    personArr:["貂蝉","西施","赵飞燕"],
    objArr:[{
        id:"001",
        title:"这是标题1"
    },
    {
        id:"002",
        title:"这是标题2"
    }]
}
```

```
<!--components/xqitem/xqitem.wxml-->
<view  class="child">
    <text>子组件的区域</text>
    <view  class="info">父传递来的num: {{num}}</view>
    <view class="info">父传递来的persons: {{persons}}</view>
    <view class="info">父传递来的obj:
      <text wx:for="{{objs}}" wx:key="id">{{item.id}}</text>
    </view>
</view>

/**
* 组件的属性列表
*/
properties: {
    num:{
        type:Number,
        value:0
    },
    persons:{
        type:Array,
        value:[]
    },
    objs:{
        type:Object,
        value:{}
    }
}
```



4.2.2、子传父

> 父传子的意思就是通过子组件，改变父组件中的数据，一般通过自定义事件的方式进行。

1. 父亲定义自定义函数，并和初始数值传递给子组件

```
<!--pages/index/index.wxml-->
<view class="box">
    这是父亲组件:num = {{num}}
    <xqitem num="{{num}}" bind:cNum="changeNum"></xqitem>
</view>

/**
* 页面的初始数据
*/
data: {
    num:1
},

//自定义函数
changeNum(e){
    this.setData({
        num:e.detail
    })
}
```

> 说明：
>
> cNum为自定义事件名，触发该事件后，执行自定义函数changeNum，其内部有一个e，就是事件对象，由它带回子组件中传递回来的数值。

2. 子组件触发事件，并传递回数值

```
<!--components/xqitem/xqitem.wxml-->
<view  class="child">
    <text>子组件的区域</text>
    <view  class="info">父传递来的num: {{num}}</view>
    <view>
      <button type="primary" size="mini"
bind:tap="btnChange">+2</button>
    </view>
</view>

/**
* 组件的属性列表
*/
properties: {
    num:{
        type:Number,
        value:0
    }
},

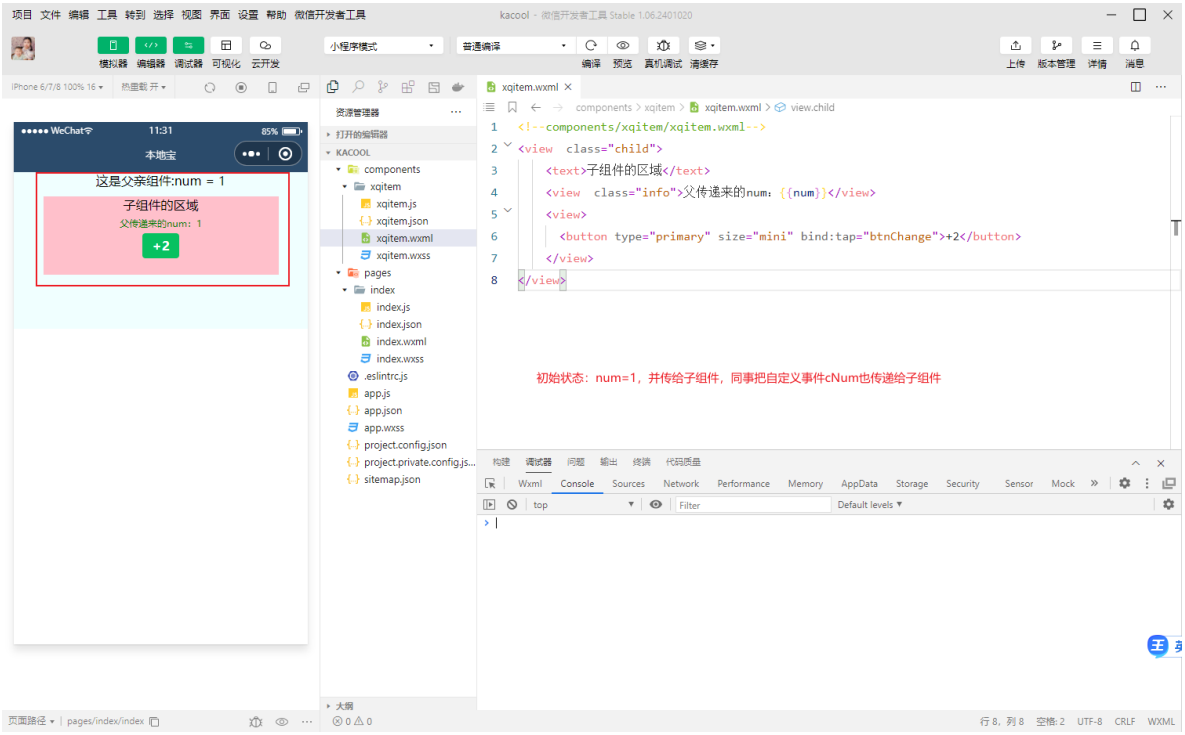
/**
* 组件的方法列表
*/
methods: {
    btnChange(){
        this.setData({
            num:this.properties.num + 2
        })
        this.triggerEvent("cNum",this.properties.num)
        }
}

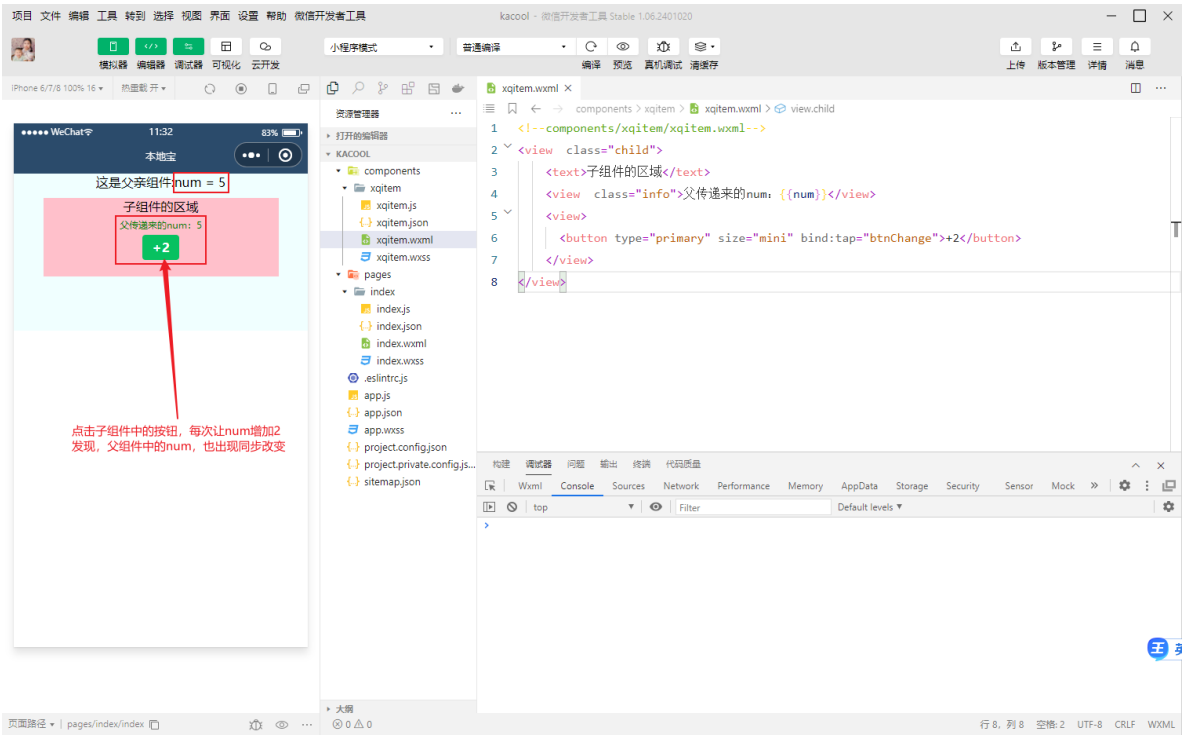
```

说明:

在子组件中，绑定按钮点击事件，当点击按钮时，先改变子组件接收到的num数值，

然后通过：`this.triggerEvent("cNum",this.properties.num)` 代码去触发自定义事件cNum，并传回修改后Num的数值。





4.2.3、获取实例

除了让子组件主动触发事件以外，我们也可以通过方法，去获取子组件的实例，然后在父组件中，去调用子组件中的任意方法和数据。使用前，给子组件设置一个id或者class。
格式：`this.selectComponent("id或class选择器")`

更改父组件代码

```
<!--pages/index/index.wxml-->
<view class="box">
    这是父亲组件:num = {{num}}
    <view><button type="primary" size="mini"
bind:tap="getChild">获取子组件</button></view>
```

```
    <xqitem num="{{num}}" bind:cNum="changeNum" id="cA">
</xqitem>
</view>

/**
* 页面的初始数据
*/
 data: {
      num:1
},

  //获取子组件
getChild(){
    const child = this.selectComponent("#cA")
    child.setData({
       num:child.properties.num + 2
    })
    child.btnChange()
}
```

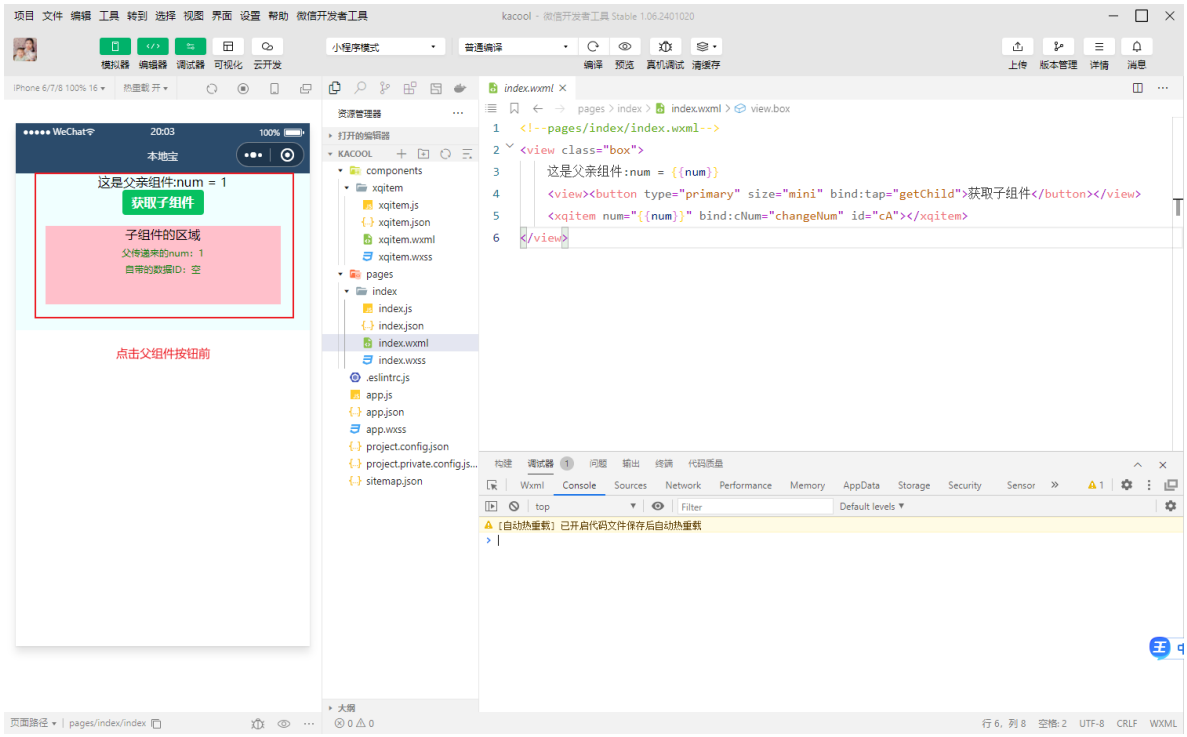
更改子组件代码

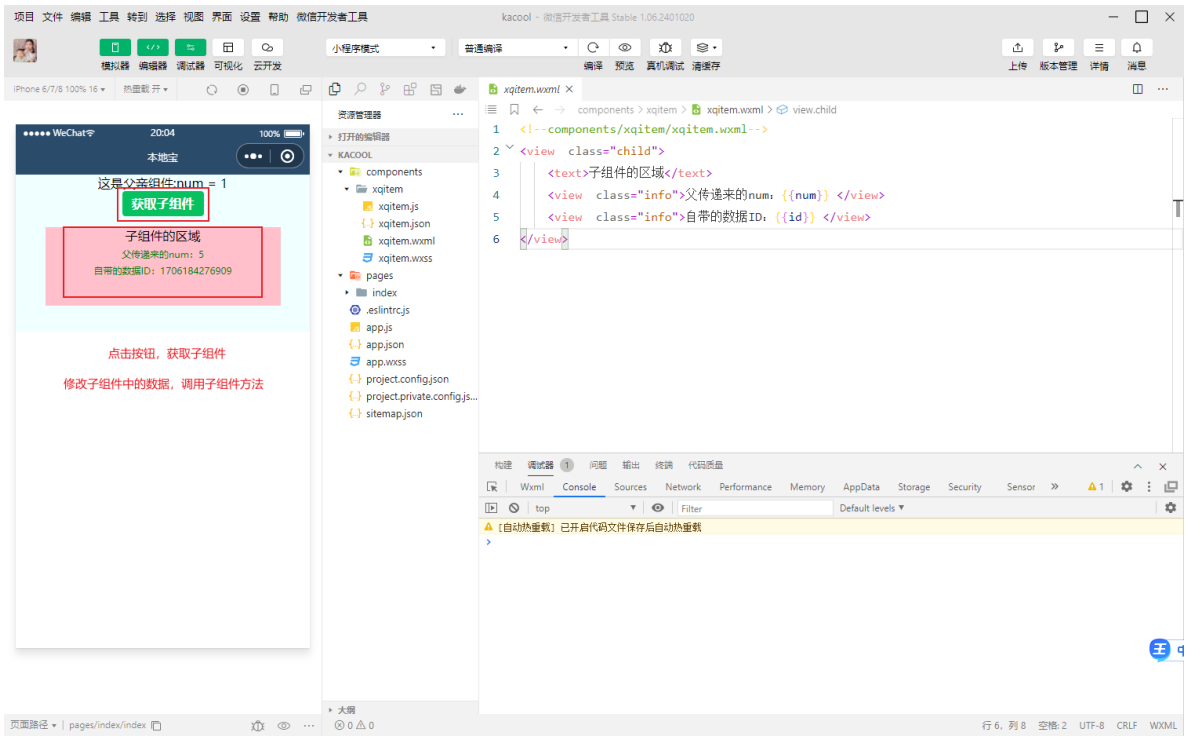
```
<!--components/xqitem/xqitem.wxml-->
<view  class="child">
    <text>子组件的区域</text>
    <view  class="info">父传递来的num：{{num}} </view>
    <view  class="info">自带的数据ID：{{id}} </view>
</view>

/**
* 组件的属性列表
*/
properties: {
    num:{
        type:Number,
        value:0
    }
},

/**
* 组件的初始数据
*/
data: {
    id:'空'
},

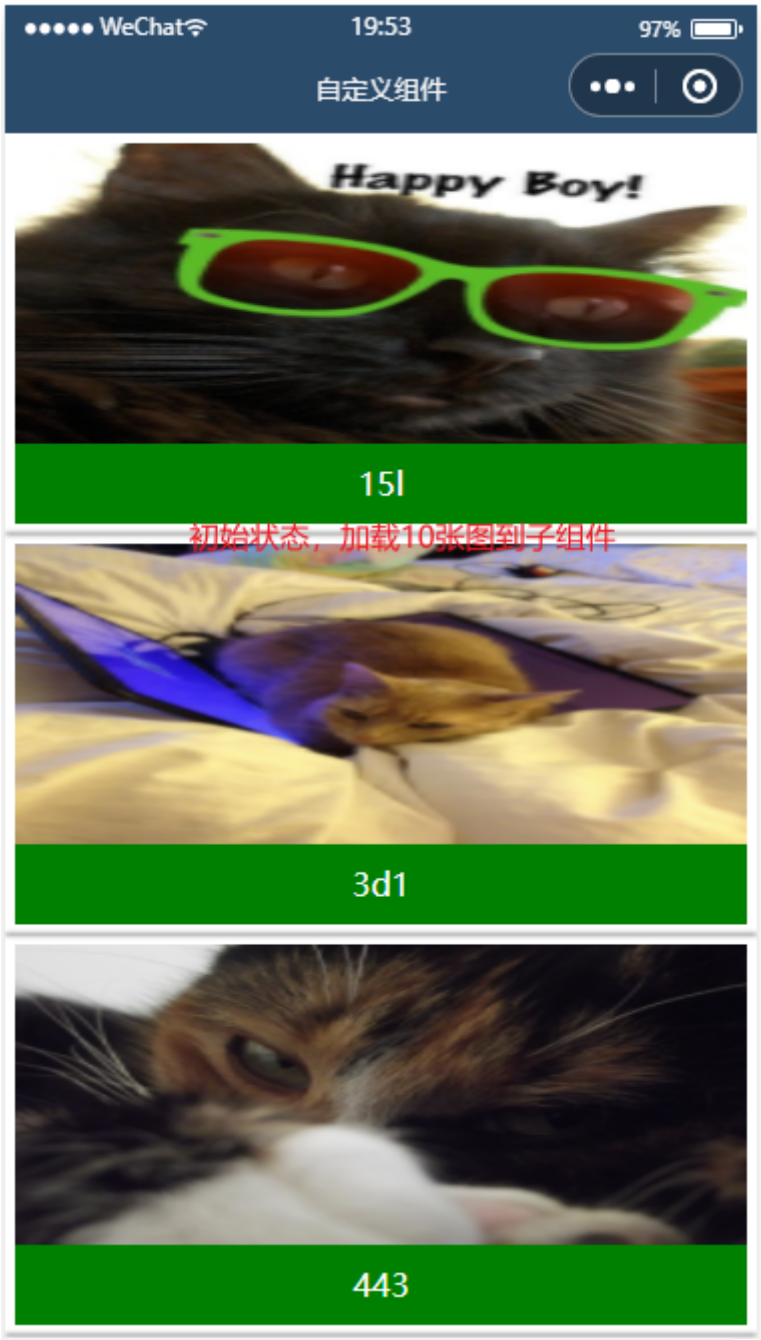
/**
* 组件的方法列表
*/
methods: {
    btnChange(){
        this.setData({
            id:new Date().getTime()
        })
    }
}
```

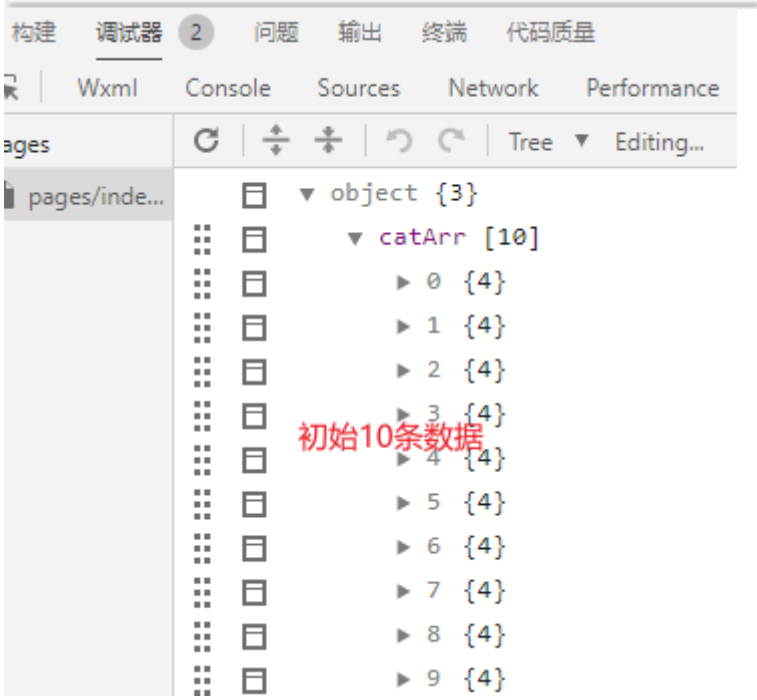




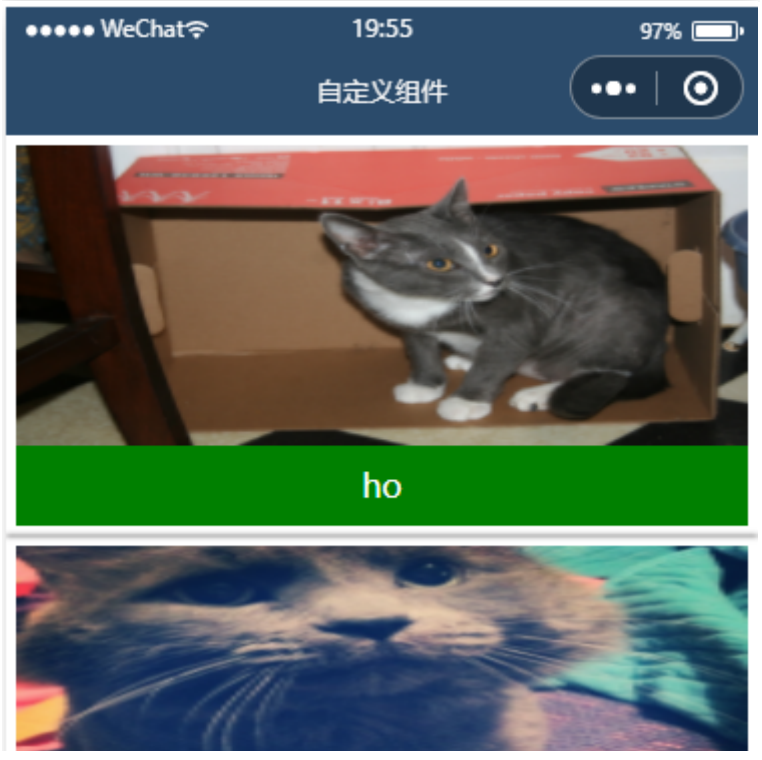
5.3节、综合案例-随机图进阶

实现效果：


●●●●● WeChat
19:53
97%
自定义组件
Happy Boy!
15l
初始状态，加载10张图到子组件
3d1
443


构建
调试器 2
问题
输出
终端
代码质量
Wxml
Console
Sources
Network
Performance
Tree
Editing...
pages/inde...
▼ object {3}
▼ catArr [10]
▶ 0 {4}
▶ 1 {4}
▶ 2 {4}
▶ 3 {4}
▶ 4 {4}
▶ 5 {4}
▶ 6 {4}
▶ 7 {4}
▶ 8 {4}
▶ 9 {4}
初始10条数据


WeChat
19:54
97%
自定义组件
数据请求中...
下拉刷新，重新请求数据
alp


WeChat
19:55
97%
自定义组件
ho



alp



bjk



调试器 2 问题 输出 终端 代码质量

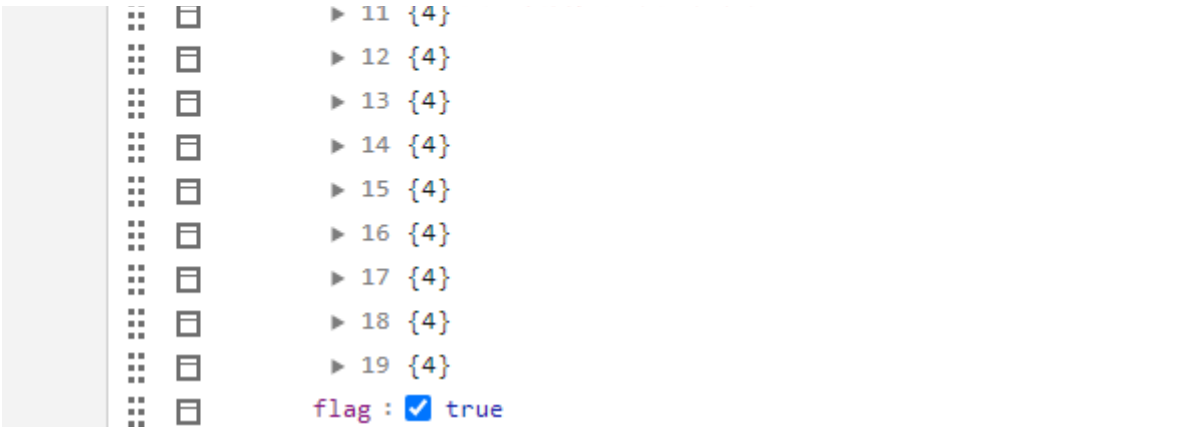
Wxml Console Sources Network Performance Memory

Tree

es/inde...

```
▼ object {3}
  ▼ catArr [20]
    ▶ 0 {4}
    ▶ 1 {4}
    ▶ 2 {4}
    ▶ 3 {4}
    ▶ 4 {4}
    ▶ 5 {4}
    ▶ 6 {4}
    ▶ 7 {4}
    ▶ 8 {4}
    ▶ 9 {4}
    ▶ 10 {4}
```

每触发一次，多请求10张图，并且在每次请求初始时，flag设为假，请求完毕后，又设置为真。



子组件代码：

```
<!--components/xqitem/xqitem.wxml-->

<view  class="child">
    <image src="{{catObj.url}}" mode=""/>
    <view class="info">{{catObj.id}}</view>
</view>
```

```
/* components/xqitem/xqitem.wxss */

.child{
  width: 100%;
  height: 400rpx;
  padding: 10rpx;
  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  box-shadow: 0rpx 0rpx 6rpx #666;
  image{
    width: 100%;
    height: 100%;
  }
  .info{
    text-align: center;
    height: 80rpx;
    line-height: 80rpx;
    background-color: green;
    color: white;
  }
}
```

```
// components/xqitem/xqitem.js

Component({

  /**
   * 组件的属性列表
   */
  properties: {
      catObj:{
```

```
        type:Object,
        value:{}
      }
  },

  /**
   * 组件的初始数据
   */
  data: {

  },

  /**
   * 组件的方法列表
   */
  methods: {

  }
})
```

父组件代码:

```
index.json

{
  "usingComponents": {
    "xqitem":"/components/xqitem/xqitem"
  },
  "navigationBarTitleText": "自定义组件",
  "navigationBarBackgroundColor": "#2b4b6b",
  "navigationBarTextStyle":"white",
  "enablePullDownRefresh": true,
  "onReachBottomDistance": 100
}
```

```
<!--pages/index/index.wxml-->

<view class="cats">
  <view class="list" wx:for="{{catArr}}" wx:key="index">
    <xqitem catObj="{{item}}"></xqitem>
  </view>
</view>
```

```
/* pages/index/index.wxss */

.box{
  width: 750rpx;
}
```

```
// pages/index/index.js

Page({
```

```
  /**
   * 页面的初始数据
   */
  data: {
    catArr:[],
    flag:true
  },

  /**
   * 生命周期函数--监听页面加载
   */
  onLoad(options) {
    this.getCat()
  },

  getCat(){
    //如果标识为真，表示正在加载，不可进行下一次请求。
    if(!this.data.flag){
      return false
    }

    //数据请求中，将标识设置为假
    this.setData({
       flag:false
    })

    wx.showLoading({
      title: '数据请求中...',
      mask:true
    })
    wx.request({
      url: 'https://api.thecatapi.com/v1/images/search',
      data:{
        limit:2
      },
      method:'GET',
      dataType:'json',
      success:res=>{
        this.setData({
          catArr:this.data.catArr.concat(res.data)
        })
      },
      complete:()=>{
        //不管请求是否成功，完毕后将标识设置为真
        this.setData({
          flag:true
        })
        setTimeout(() => {
          wx.hideLoading()
          wx.stopPullDownRefresh()
        }, 1000);
      }
  })
  },

  /**
   * 生命周期函数--监听页面初次渲染完成
```

```
61      */
62     onReady() {
63
64     },
65
66     /**
67      * 生命周期函数--监听页面显示
68      */
69     onShow() {
70
71     },
72
73     /**
74      * 生命周期函数--监听页面隐藏
75      */
76     onHide() {
77
78     },
79
80     /**
81      * 生命周期函数--监听页面卸载
82      */
83     onUnload() {
84
85     },
86
87     /**
88      * 页面相关事件处理函数--监听用户下拉动作
89      */
90     onPullDownRefresh() {
91       this.setData({
92         catArr:[]
93       })
94       this.getCat()
95     },
96
97     /**
98      * 页面上拉触底事件的处理函数
99      */
100    onReachBottom() {
101      this.getCat()
102    },
103
104    /**
105     * 用户点击右上角分享
106     */
107    onShareAppMessage() {
108
109    }
110  })
```

第6章、云开发基础

6.1节、云开发概述

微信云开发 (WeChat Cloud Development) 是微信开放平台提供的一套云端一体化开发平台，旨在简化开发者构建微信小程序和微信公众号的后台服务和前端页面的过程。通过微信云开发，开发者无需搭建和维护自己的服务器，即可快速开发小程序和公众号的全栈应用。

微信云开发提供以下核心功能和特点：

1. 云数据库：提供了一个类似于 NoSQL 数据库的云端数据库，开发者可以直接在小程序或公众号前端操作数据库，实现数据的存储和读取。
2. 云存储：提供了一个云存储空间，用于存储图片、音视频文件等多媒体资源，方便前端页面的展示和使用。
3. 云函数：提供了在云端运行的函数计算能力，开发者可以编写和部署云函数来实现后台的业务逻辑和数据处理，无需自行搭建服务器。
4. 实时数据推送：支持实时数据推送和数据变化监听，可以实时更新前端页面的数据展示。
5. 开发工具和日志管理：提供了微信开发者工具和日志管理功能，方便开发者进行本地调试和远程日志查看。

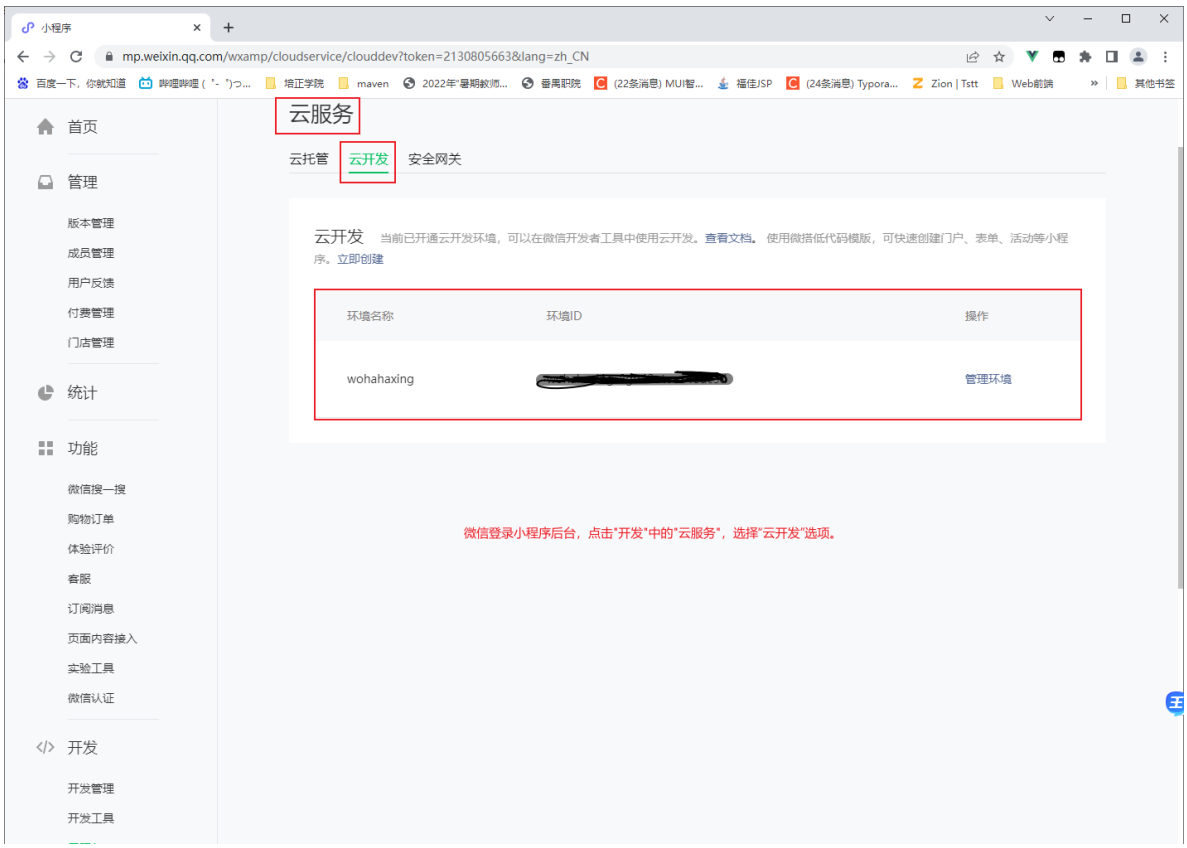
微信云开发的优势在于无需搭建和维护服务器，开发门槛较低，可以更专注于小程序和公众号的业务逻辑和用户体验。同时，微信云开发提供了强大的云端能力，支持快速构建功能丰富、高性能的应用。

如何开通微信云开发：

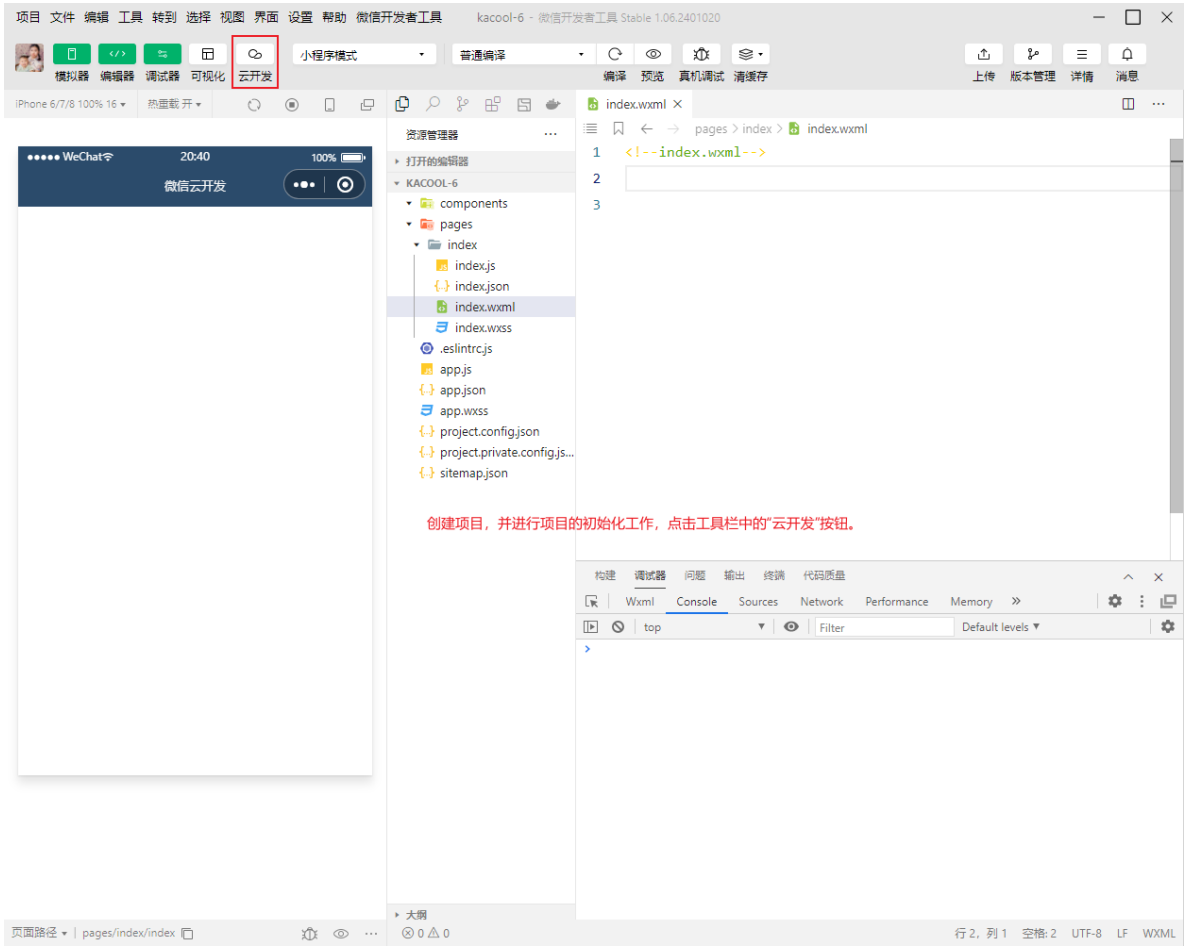
微信云服务器空间是微信云开发的一部分，用于存储小程序和公众号的云端数据和资源。

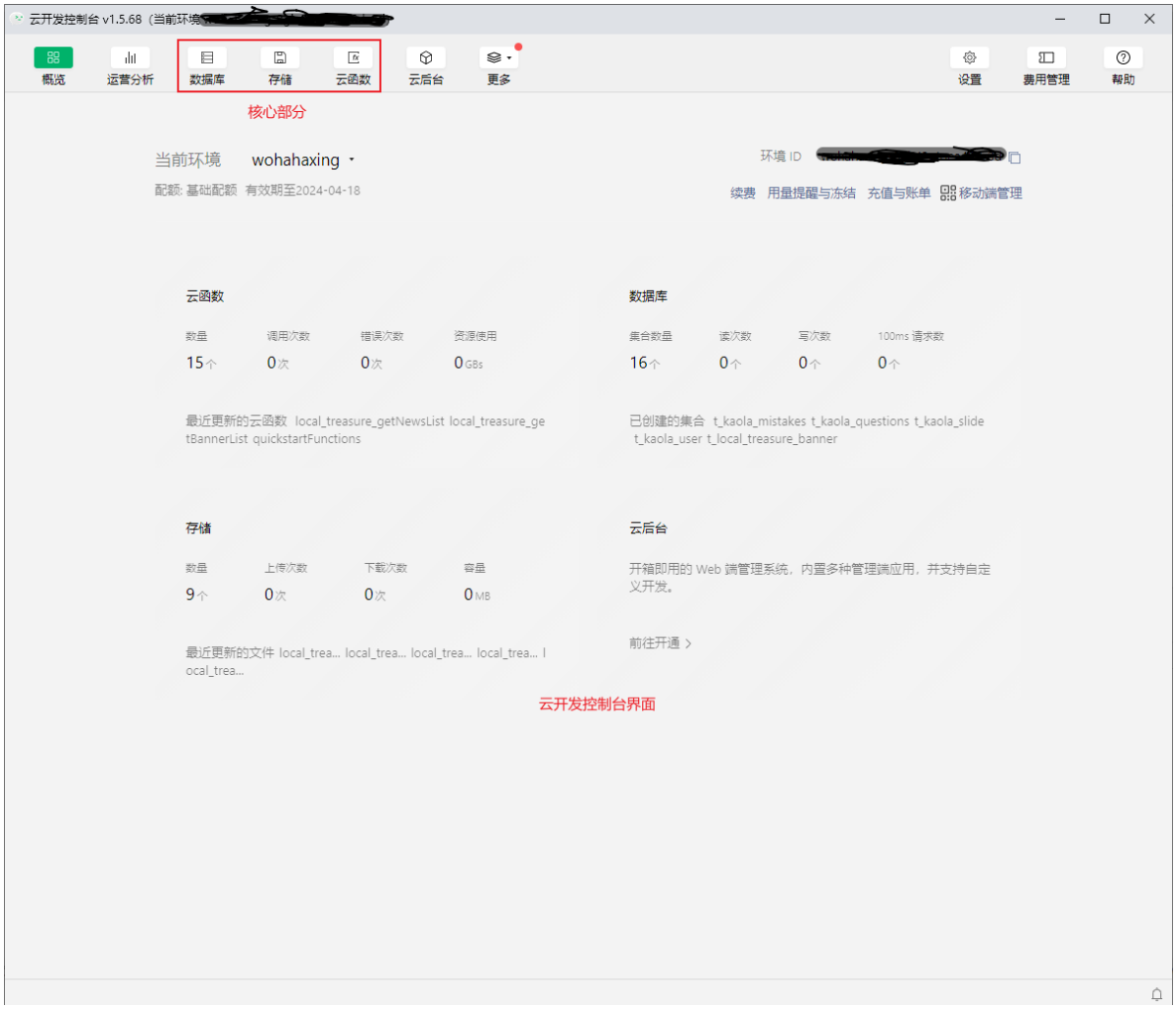
可以通过以下步骤申请微信云服务器空间：

1. 登录微信公众平台或小程序管理后台：使用您的微信公众号或小程序的管理员账号登录到对应的管理后台。
2. 进入云开发控制台：在管理后台中，找到云开发控制台的入口。对于小程序，可以在开发-开发设置中找到；对于公众号，可以在设置-开发者工具中找到。
3. 开通云开发服务：在云开发控制台中，您需要开通云开发服务，按照提示进行操作即可。
4. 创建云环境：开通云开发服务后，您需要创建一个云环境，这是一个隔离的运行环境，用于存储您的云端数据和资源。在云开发控制台中，选择创建新环境，并按照提示填写环境名称和描述。
5. 获取云服务器空间：创建云环境后，您会自动获得一个云服务器空间，用于存储小程序和公众号的云端数据和资源。



也可以通过开发工具，在项目中，点击“云开发”工具栏。





6.2节、云数据库

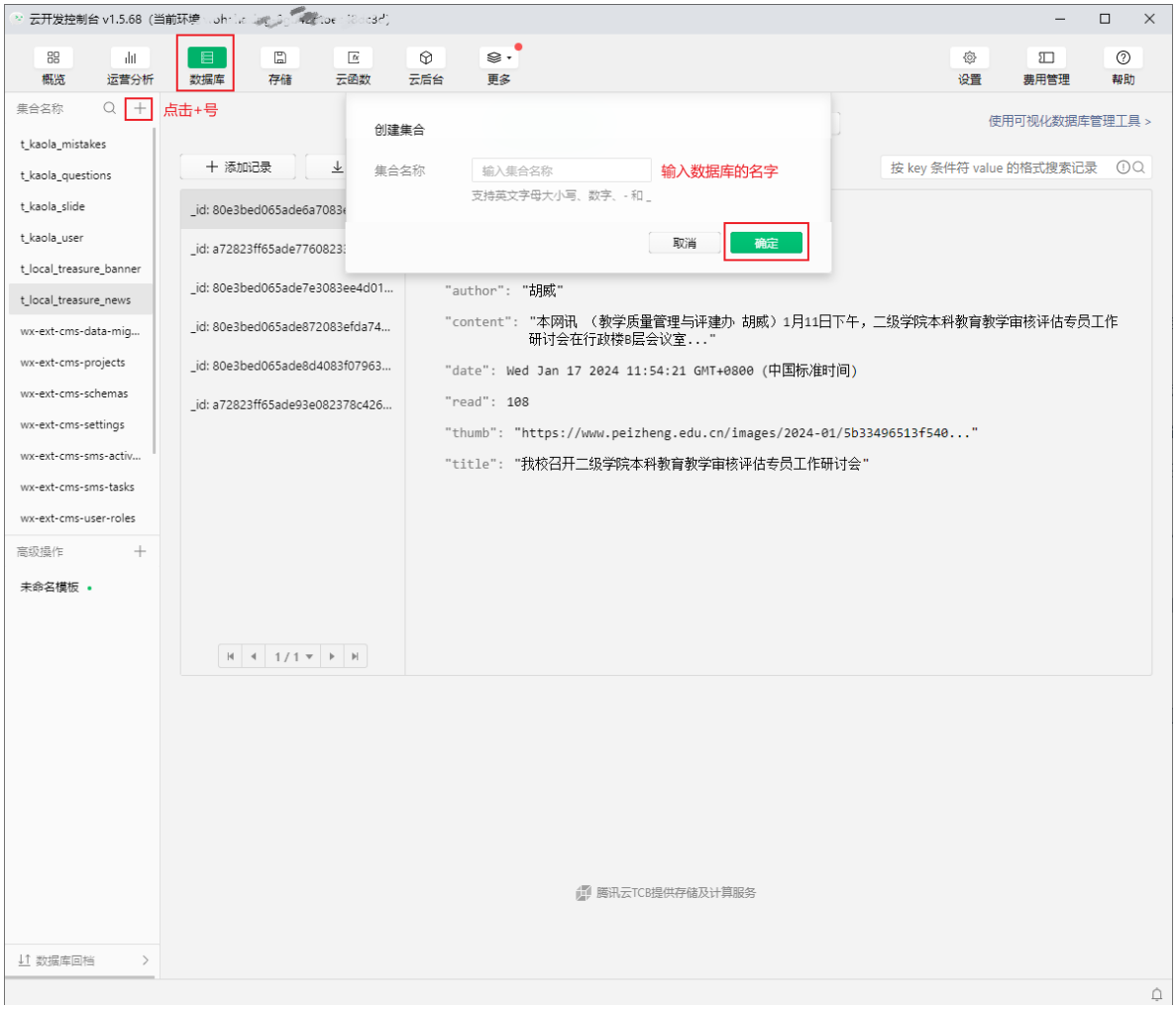
微信云数据库（WeChat Cloud Database）是微信云开发的核心组件之一，它是一种基于 NoSQL 的云端数据库服务，专为微信小程序和微信公众号提供数据存储和读取的能力。

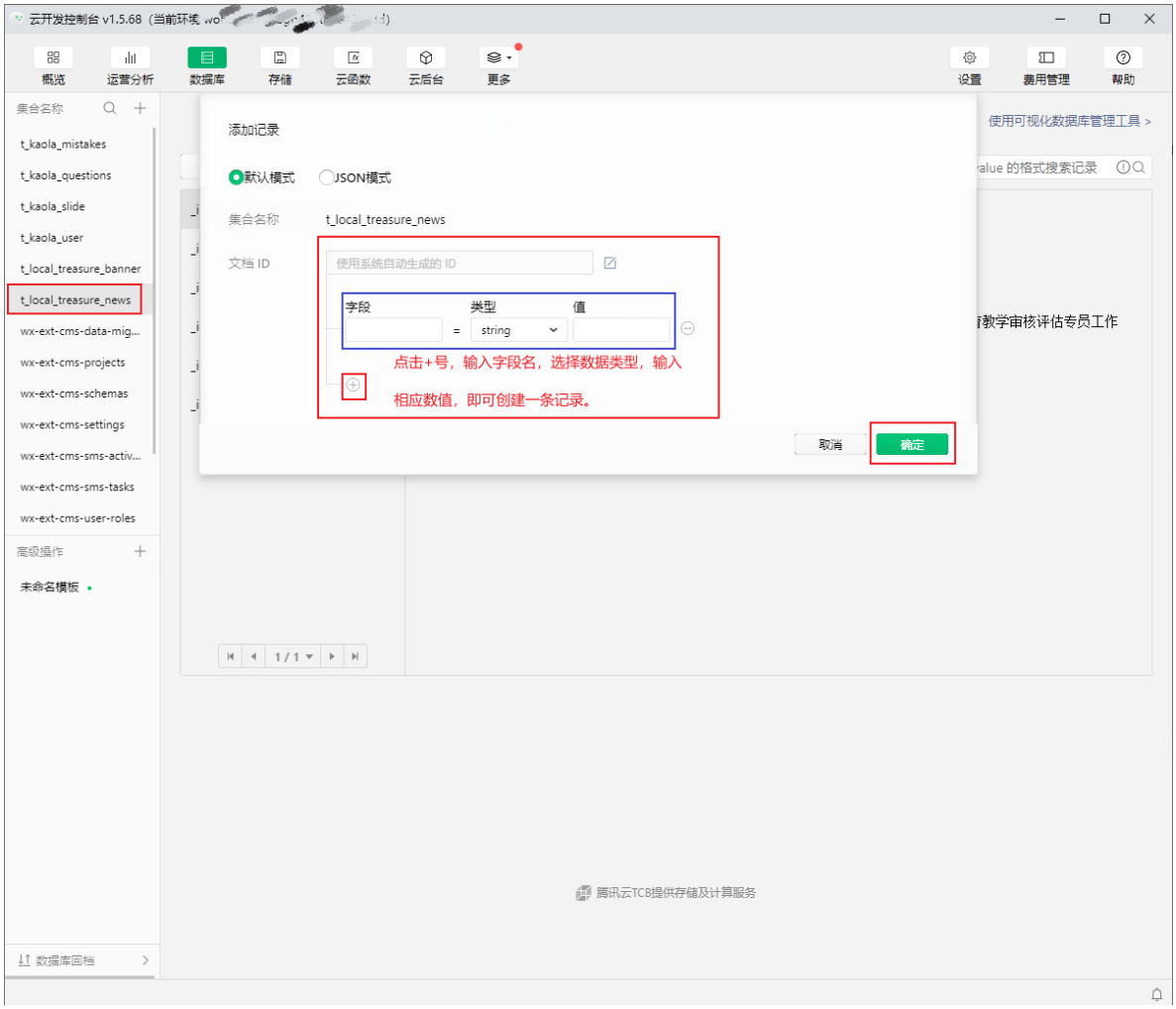
微信云数据库的特点：

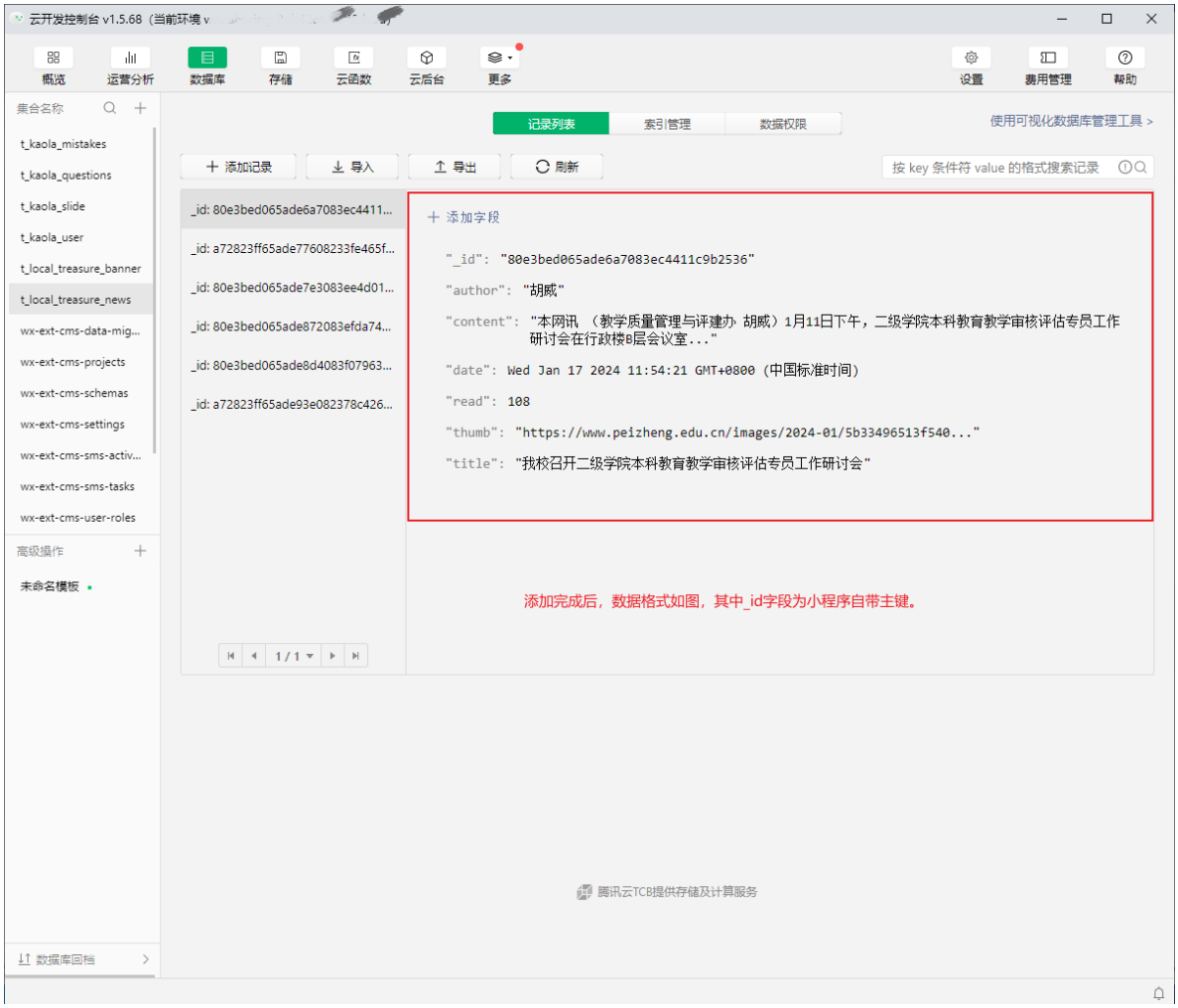
1. 数据库类型：微信云数据库支持多种数据库类型，包括云数据库（文档型数据库）和云数据库 MySQL（关系型数据库），开发者可以根据需求选择合适的数据库类型。
2. 数据库管理：微信云数据库提供了可视化的数据库管理界面，开发者可以在云开发控制台中对数据库进行创建、管理和配置。
3. 数据读写操作：开发者可以直接在小程序或公众号前端通过 API 进行数据的增、删、改、查操作，无需自己搭建后台服务器。
4. 数据查询和筛选：微信云数据库支持复杂的数据查询和筛选操作，例如条件查询、范围查询、排序等，方便开发者根据需求获取所需的数据。
5. 实时数据推送：微信云数据库支持实时数据推送功能，当数据库中的数据发生变化时，可以实时通知到前端页面，方便实时更新数据展示。
6. 数据权限管理：微信云数据库提供了数据权限管理功能，可以根据用户的身份和权限设置数据的读写权限，保护数据的安全性和隐私性。
7. 数据备份和恢复：微信云数据库提供了数据备份和恢复的功能，可以定期备份数据库，并在需要时进行数据的恢复。

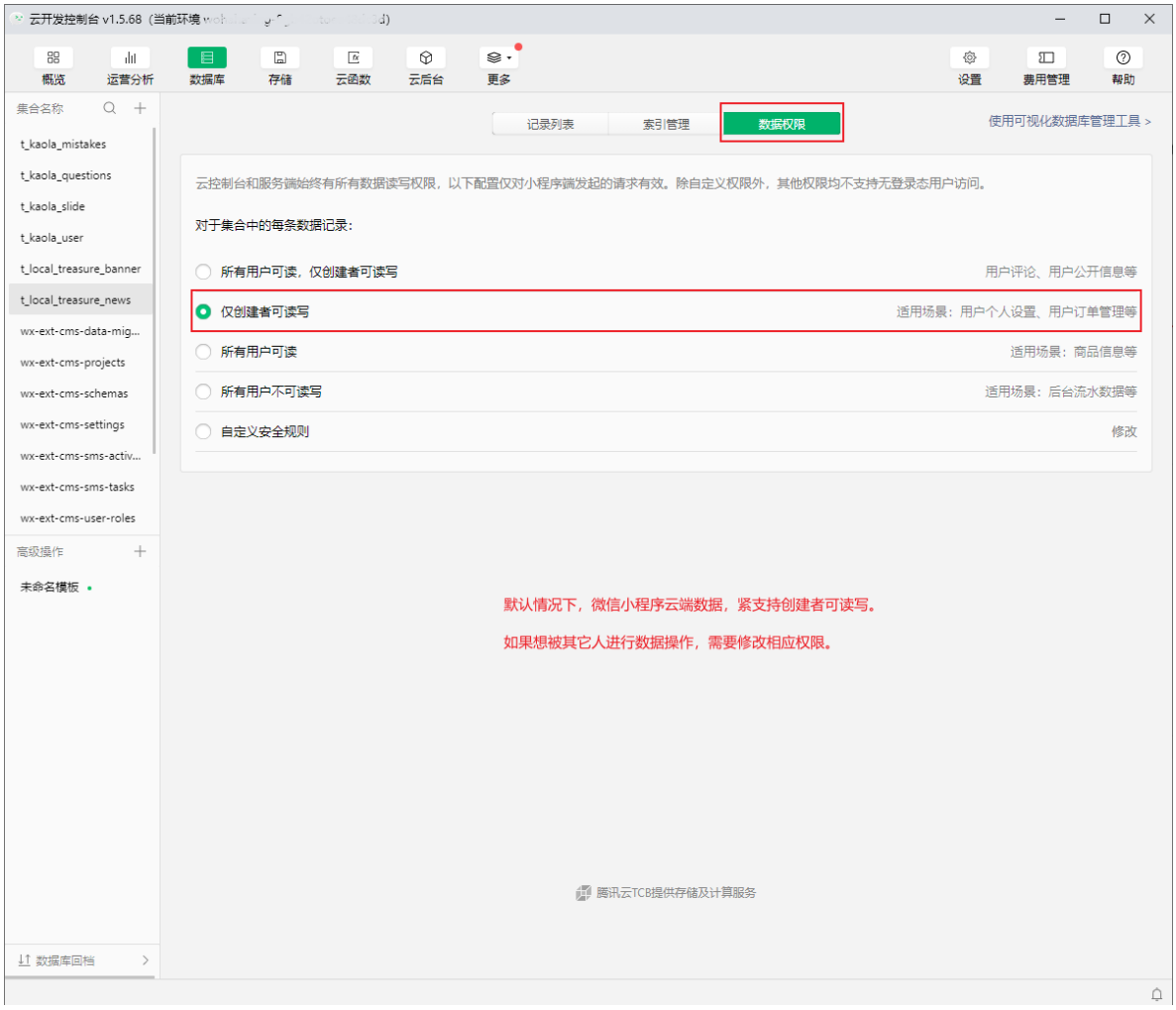
通过微信云数据库，开发者无需搭建和维护自己的数据库服务器，可以快速实现小程序和公众号的数据存储和读取。同时，微信云数据库提供了强大的数据查询和管理功能，满足了各种数据操作的需求。

创建过程：




添加记录
默认模式
JSON模式
集合名称
t_local_treasure_news
文档 ID
使用系统自动生成的 ID
字段
类型
值
string
点击+号，输入字段名，选择数据类型，输入相应数值，即可创建一条记录。
取消
确定


记录列表
索引管理
数据权限
添加记录
导入
导出
刷新
添加字段
"_id": "80e3bed065ade6a7083ec4411c9b2536"
"author": "胡威"
"content": "本网讯 （教学质量管理与评建办 胡威）1月11日下午，二级学院本科教育教学审核评估专员工作研讨会在行政楼B层会议室..."
"date": Wed Jan 17 2024 11:54:21 GMT+0800 (中国标准时间)
"read": 108
"thumb": "https://www.peizheng.edu.cn/images/2024-01/5b33496513f540..."
"title": "我校召开二级学院本科教育教学审核评估专员工作研讨会"
添加完成后，数据格式如图，其中_id字段为小程序自带主键。



6.3节、云函数

要想通过小程序项目，对云端数据进行操作，我们还需要借助云函数来进行。它是一种在云端运行的函数计算服务，通过微信云函数，开发者可以将复杂的业务逻辑和数据处理任务放在云端进行处理，前端页面只需调用云函数即可获取结果，极大地简化了开发流程和减轻了服务器的负担。

6.3.1、初始化云服务

1. 初始化云服务

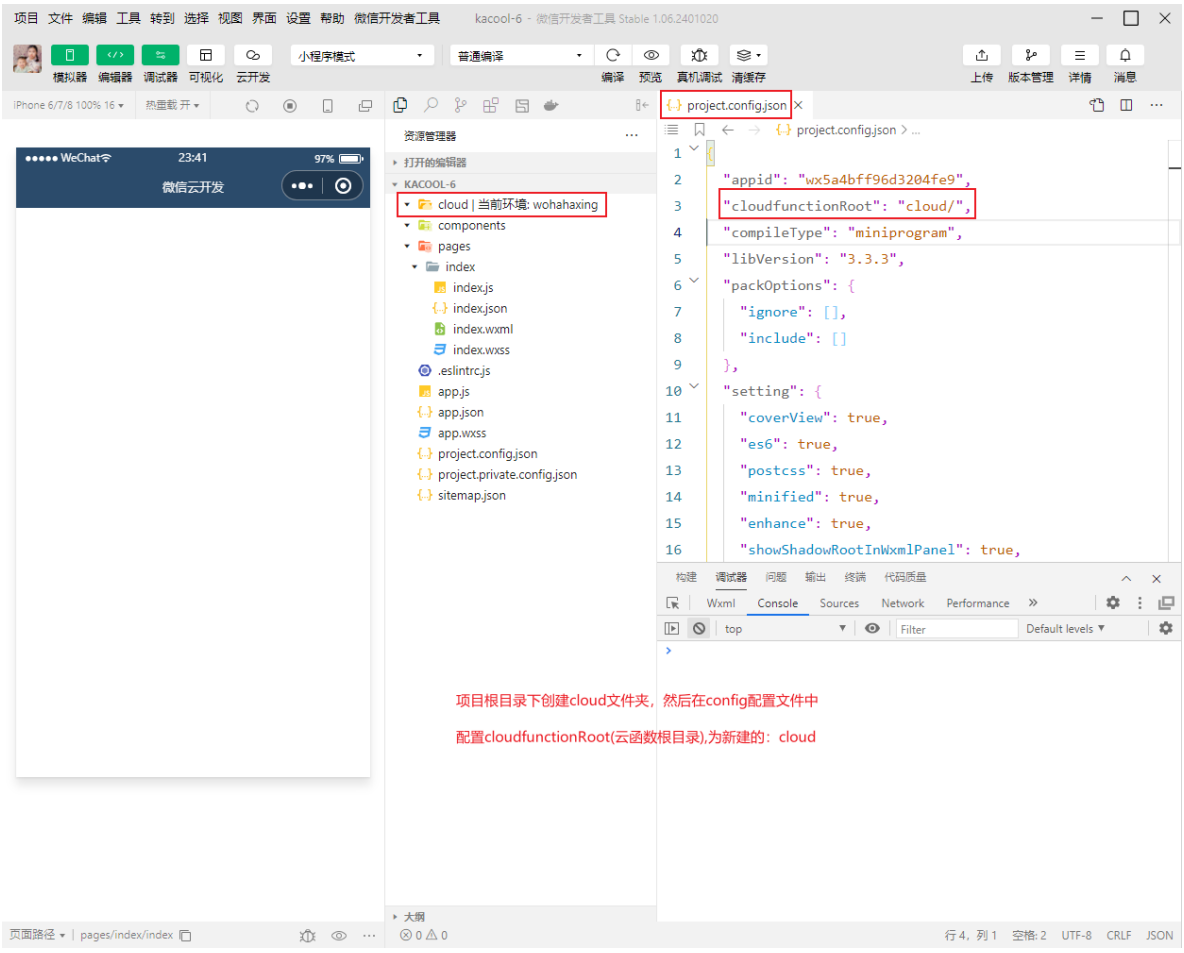
```
app.js

/**
   * 当小程序初始化完成时，会触发 onLaunch（全局只触发一次）
   */
onLaunch: function () {
   wx.cloud.init({
        "env":"云空间ID"
   })
}
```

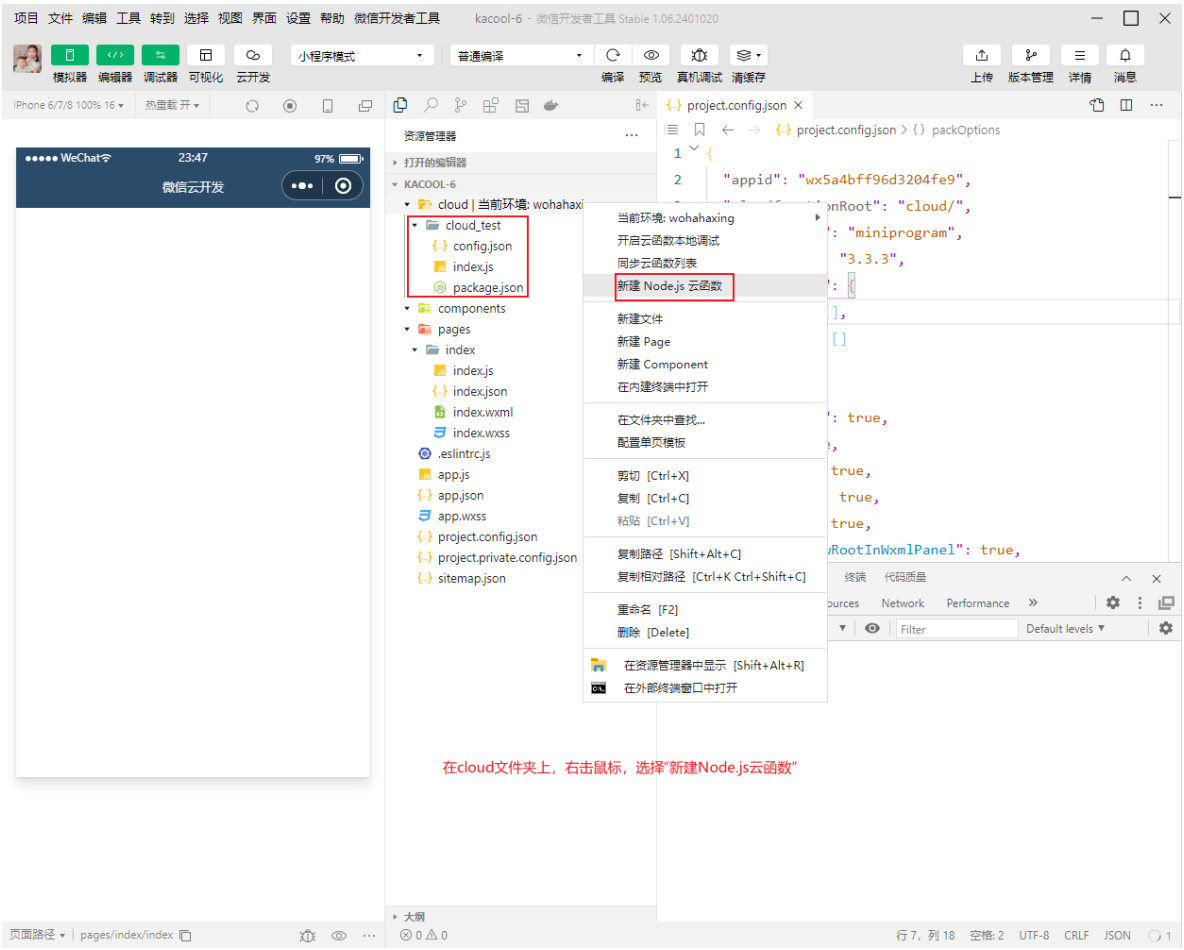
2. 创建云函数目录

在小程序根目录下，创建文件夹：`cloud`,然后在`project.config.json`文件中进行云函数目录配置。

具体如下图所示：



3． 创建云函数



4. 编辑云函数

```
// 云函数入口文件
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
// 云函数入口函数
exports.main = async (event, context) => {
    const res = await
cloud.database().collection('t_local_treasure_news').get()

    return {
      code : 200,
      msg  : '操作成功！',
      data : res.data
    }
}
```

```
// index.js
Page({

  /**
   * 页面的初始数据
   */
  data: {
     newsList:[]
  },

  /**
   * 生命周期函数--监听页面加载
   */
  onLoad: function (options) {
    this.getNews()
  },

  //自定义获取云数据
  getNews(){
      wx.cloud.callFunction({
        "name":"cloud_test"
      }).then(res=>{
        console.log(res);
      })
  },

  /**
   * 生命周期函数--监听页面初次渲染完成
   */
  onReady: function () {

  },

  /**
   * 生命周期函数--监听页面显示
   */
  onShow: function () {

```

```
39      },
40
41      /**
42       * 生命周期函数--监听页面隐藏
43       */
44      onHide: function () {
45
46      },
47
48      /**
49       * 生命周期函数--监听页面卸载
50       */
51      onUnload: function () {
52
53      },
54
55      /**
56       * 页面相关事件处理函数--监听用户下拉动作
57       */
58      onPullDownRefresh: function () {
59
60      },
61
62      /**
63       * 页面上拉触底事件的处理函数
64       */
65      onReachBottom: function () {
66
67      },
68
69      /**
70       * 用户点击右上角分享
71       */
72      onShareAppMessage: function () {
73
74      }
75  })
76
```

5. 使用云函数

> 云函数编写好之后，还需要进行上传、部署操作才能使用。



```
▼result:
    code: 200
  ▶ data: (6) [{…}, {…}, {…}, {…}, {…}, {…}]
    msg: "操作成功！"
  ▶ __proto__: Object
▶ __proto__: Object
```

成功返回了云数据库中的数据

说明：

本节关于云函数的开发，紧是带领同学们入门微信云开发的环境，熟悉云开发的初始化工作及云函数的简单使用，对于如何利用云函数来对数据库进行增、删、查、改操作，则在下一节课程中，再详细的学习。

6.3.2、云函数的CRUD

1. 查询数据

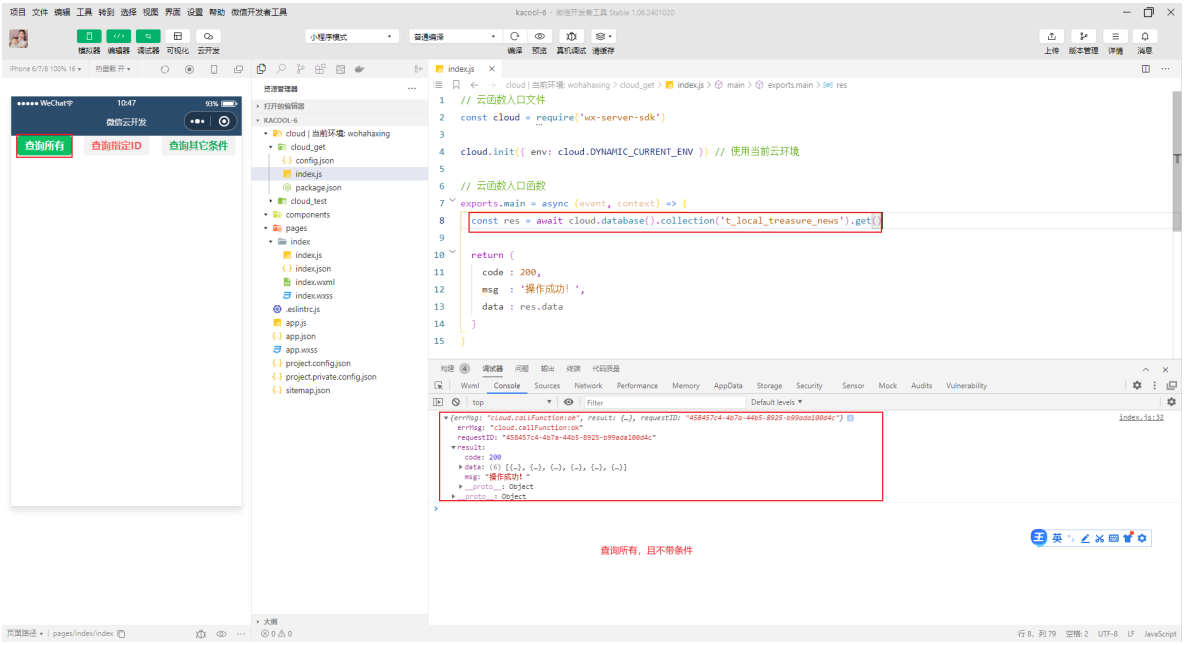
查询数据，可以说是所有项目中，最主要的工作，在云开发中，查询分为以下三种：

get()　　查询所有

doc()　　查询指定_id

where()　　查询指定其它条件

```
1  // 云函数入口文件
2  const cloud = require('wx-server-sdk')
3
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6  // 云函数入口函数
7  exports.main = async (event, context) => {
8    const res = await
   cloud.database().collection('t_local_treasure_news').get()
9
10   return {
11     code : 200,
12     msg  : '操作成功！',
13     data : res.data
14   }
15 }
```



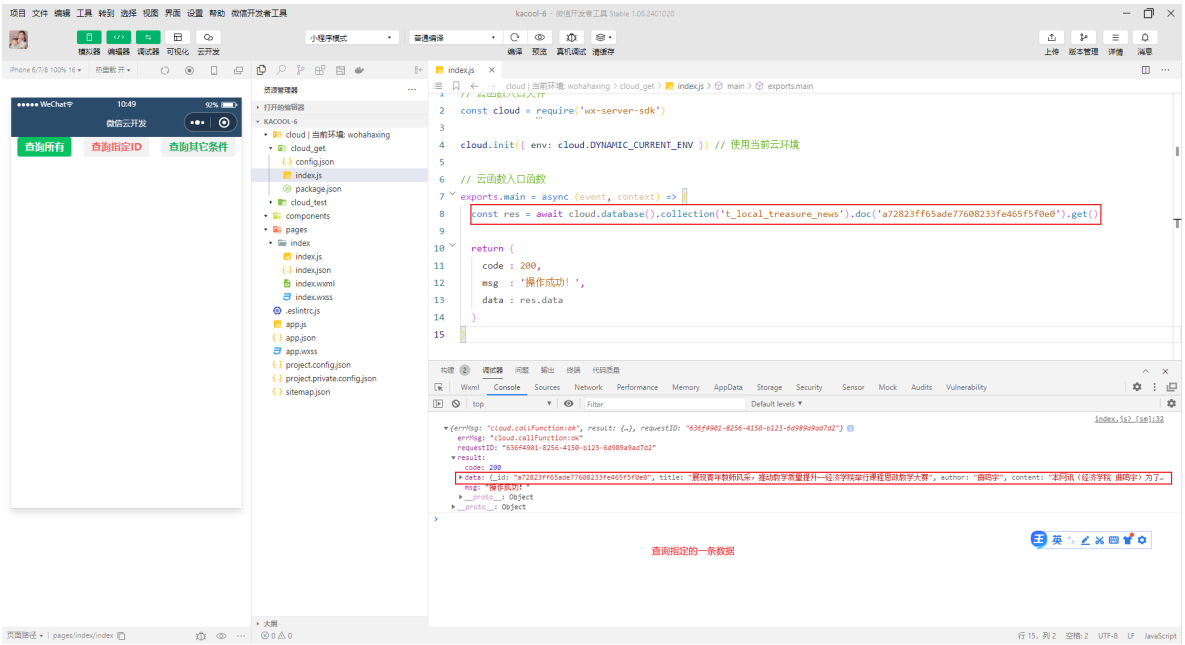
查询所有，且不带条件

```
// 云函数入口文件
const cloud = require('wx-server-sdk')

cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境

// 云函数入口函数
exports.main = async (event, context) => {
  const res = await
cloud.database().collection('t_local_treasure_news').doc('a72823f
f65ade77608233fe465f5f0e0').get()

  return {
    code : 200,
    msg  : '操作成功！',
    data : res.data
  }
}
```

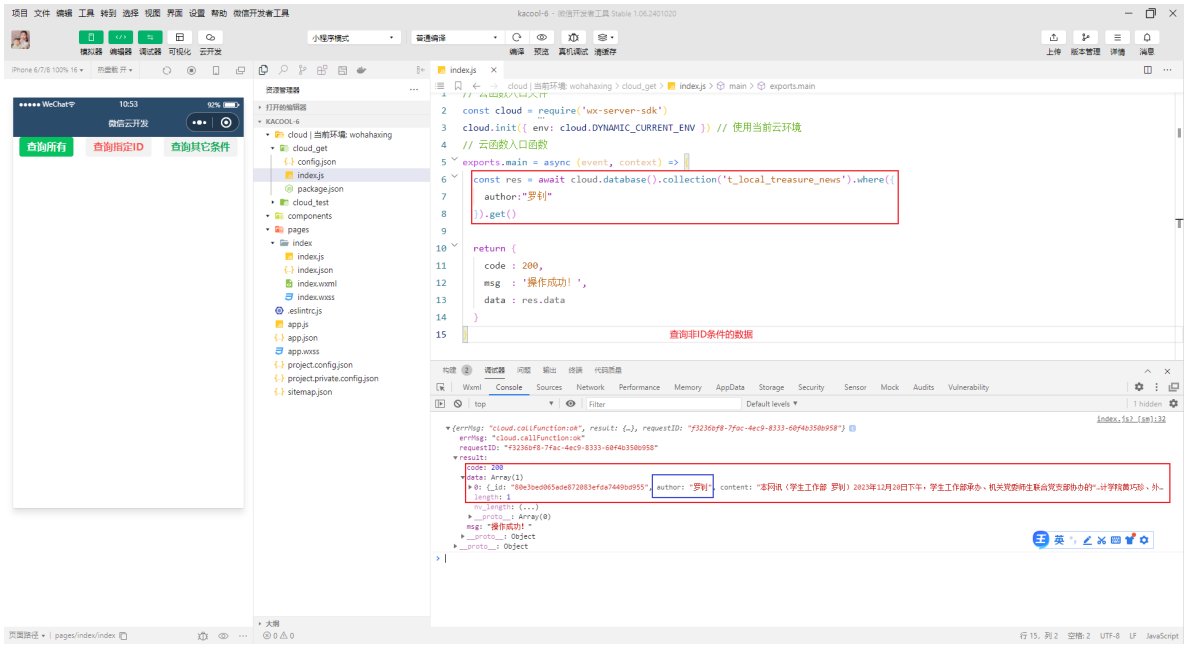


查询指定的一条数据

```
// 云函数入口文件
```

```
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
// 云函数入口函数
exports.main = async (event, context) => {
  const res = await
cloud.database().collection('t_local_treasure_news').where({
    author:"罗钊"
  }).get()

  return {
    code : 200,
    msg  : '操作成功！',
    data : res.data
  }
}
```

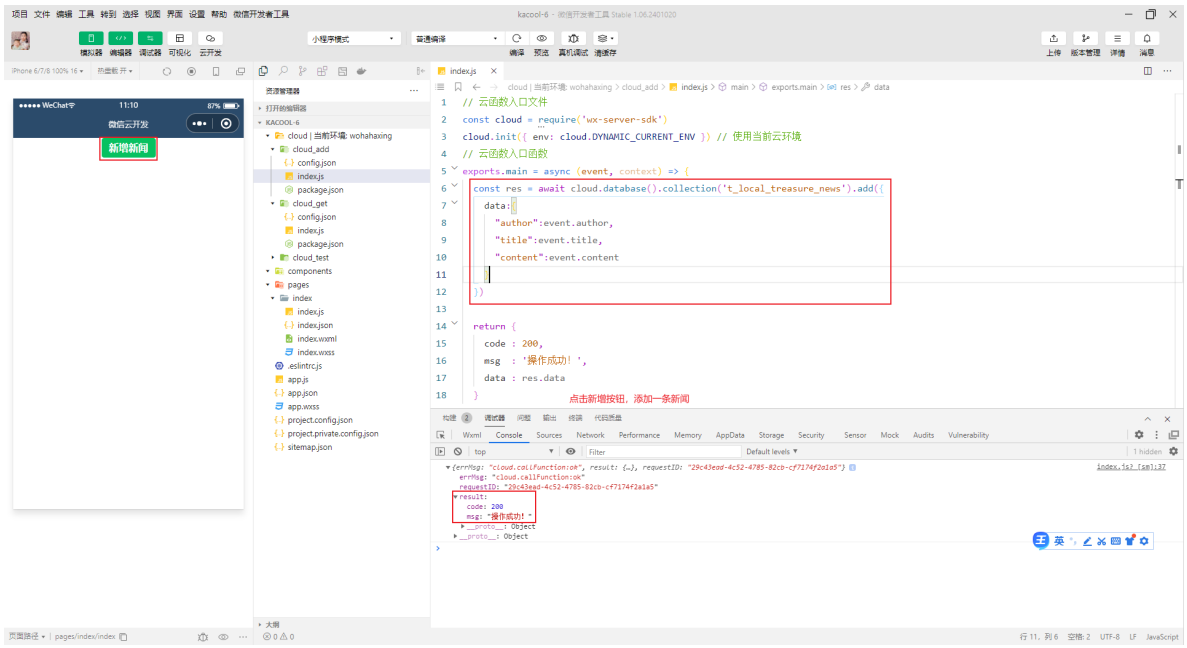


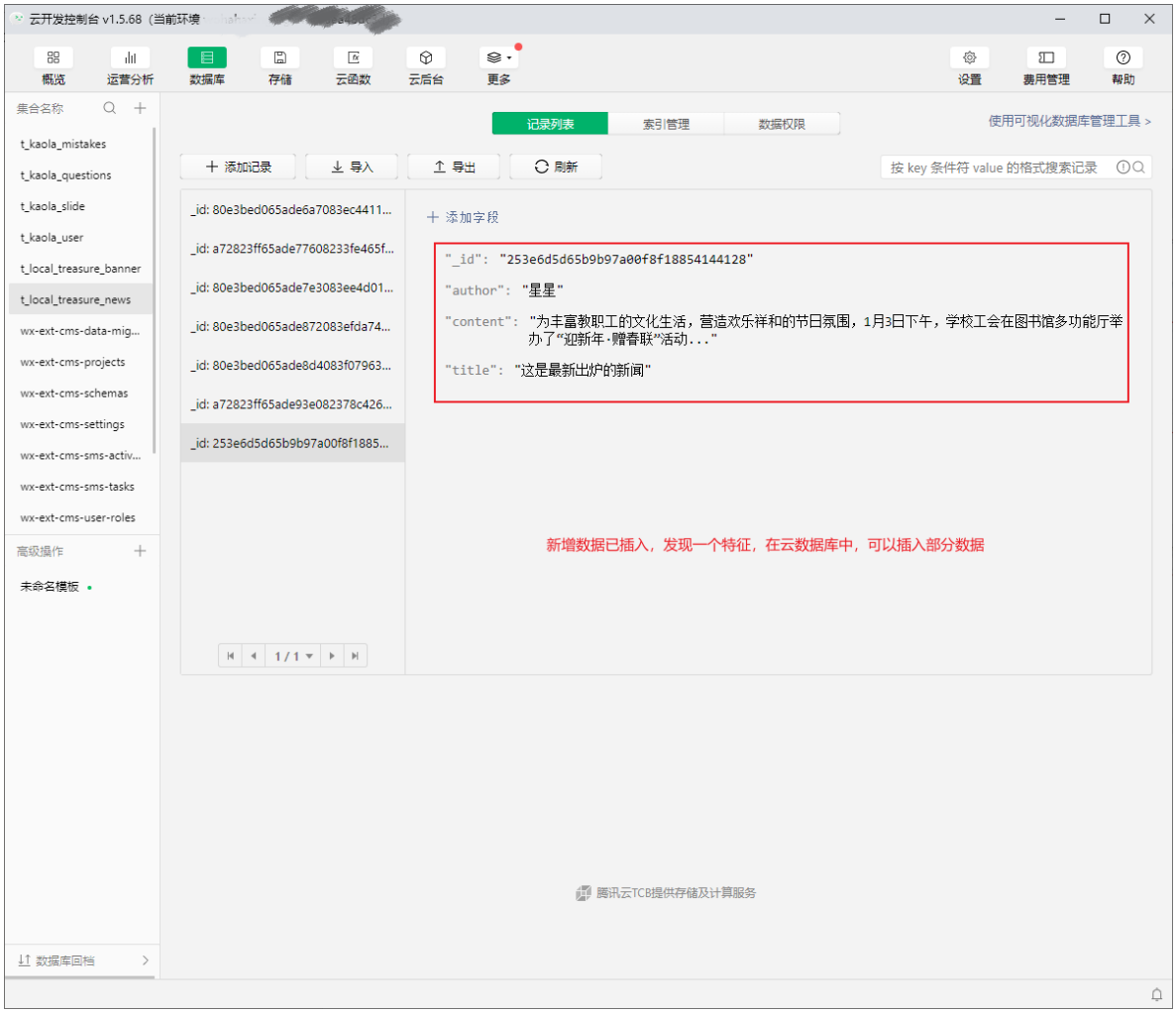
2. 新增数据

```
// 云函数入口文件
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
// 云函数入口函数
exports.main = async (event, context) => {
  const res = await
cloud.database().collection('t_local_treasure_news').add({
    data:{
      "author":event.author,
      "title":event.title,
      "content":event.content
    }
  })

  return {
    code : 200,
    msg  : '操作成功！',
    data : res.data
  }
}
```

```
//自定义函数
  addNews(){
    wx.cloud.callFunction({
      name:"cloud_add",
      data:{
        author:'星星',
        title:'这是最新出炉的新闻',
        content:'为丰富教职工的文化生活，营造欢乐祥和的节日氛围，1月3日下午，学校工会在图书馆多功能厅举办了“迎新年·赠春联”活动，学校党委副书记黄弢'
      }
    }).then(res=>{
      console.log(res);
    })
  }
```



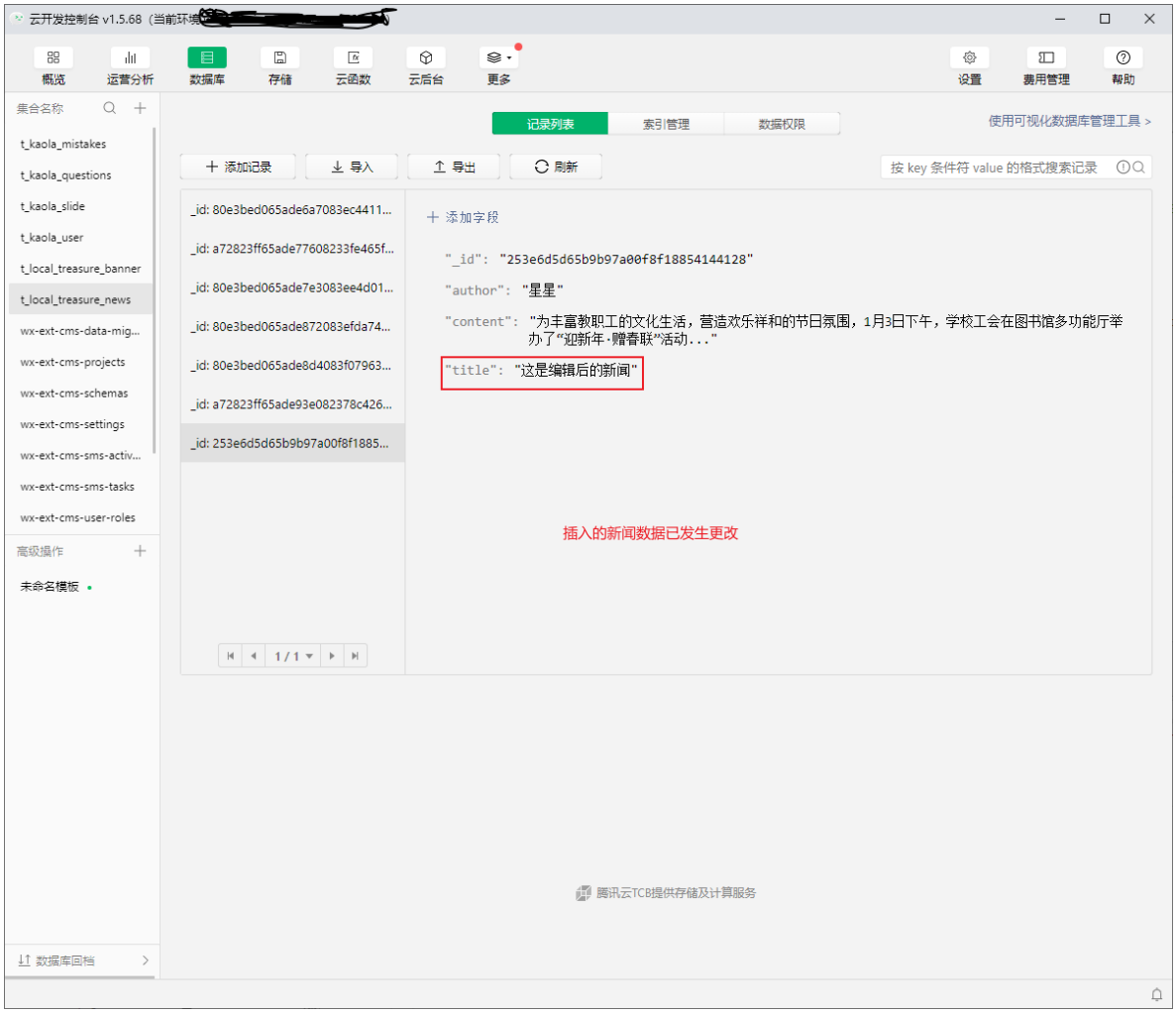


3. 修改数据

```
// 云函数入口文件
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
// 云函数入口函数
exports.main = async (event, context) => {
  const {id,title} = event
  const res = await
cloud.database().collection('t_local_treasure_news').doc(id).upd
ate({
    data:{
      title:title
    }
  })

  return {
    code : 200,
    msg  : '操作成功！',
    data : res.data
  }
}
```

```
wx.cloud.callFunction({
        name:"cloud_update",
        data:{
          id:'253e6d5d65b9b97a00f8f18854144128',
          title:'这是编辑后的新闻'
        }
      })
      .then(res=>{
        console.log(res);
      })
```



4. 删除数据

```
// 云函数入口文件
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境

// 云函数入口函数
exports.main = async (event, context) => {
    const {id} = event
    const res = await
cloud.database().collection('t_local_treasure_news').doc(id).rem
ove()

    return {
      code : 200,
      msg  : '操作成功！',
```

```
      data : res.data
    }
  }
```

```
wx.cloud.callFunction({
        name:"cloud_delete",
        data:{
          id:'253e6d5d65b9b97a00f8f18854144128'
        }
      }).then(res=>{
        console.log(res);
      })
```

可以发现，刚插入的新闻已被成功删除，由于删除后的数据无法恢复，因此在删除前，要做好确认工作。

###